Track D: Finding errors in code

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track D: Finding errors in code 3

D0 Start here: foundations for finding errors 4

D0-L1 Bugs, errors and failures, the words and the method 4

D0-L2 Read code you did not write 8

D1 Reading tracebacks and stack traces 13

D1-L1 Read a Python traceback from the bottom up 13

D1-L2 Spark errors, the condition and the real cause 18

D1-L3 Py4J and Spark Connect errors 22

D2 Debugging tools 27

D2-L1 breakpoint() and pdb, stop and look 27

D2-L2 Logging levels that help at 3 a.m. 31

D2-L3 explain(), the Spark UI and the query profile 36

D3 Silent errors 42

D3-L1 Join row explosion and nulls in filters 42

D3-L2 Type coercion and timezone drift 46

D3-L3 Mutable defaults and the truthiness of zero 51

D4 Data errors 56

D4-L1 Row counts across layers 56

D4-L2 Distribution drift between waves 60

D4-L3 Weight totals and survey checks 64

D5 Errors typical of AI-generated code 68

D5-L1 Invented functions, parameters and syntax 68

D5-L2 Deprecated APIs and version mismatches 72

D5-L3 Five review checks for AI-written code 76

D6 Verifying AI output 82

D6-L1 Read before you run, and check each API 82

D6-L2 Small data with a known answer, failing test first 86

D6-L3 Ask for assumptions, then compare a second approach 90

D7 Review practice 95

D7-L1 Review practice: Python and pandas 95

D7-L2 Review practice: PySpark and Spark SQL 98

D7-L3 Review practice: Databricks SQL and DAX 103
D8 Prompting for correct code 108
D8-L1 State versions and constraints in every prompt 108
D8-L2 State expected outputs and tests in the prompt 112
D8-L3 Iterate on a failing answer 115
Track D best-practice checklist 119
Appendix: solutions and answer keys 120

Track D: Finding errors in code

Find and fix errors fast, including errors in code that Claude and Copilot write.

D0 Start here: foundations for finding errors

Roadmap: Every month. The words and the method of debugging, and how to read code you did not write.

D0-L1 Bugs, errors and failures, the words and the method

Concept

This track is about finding what is wrong in code, quickly and for certain. Start with three kinds of problem, from cheapest to most expensive.

- A **syntax error** stops Python before anything runs. It points at the line.
- An **exception** stops the program while it runs, with a traceback that names the error type and the line, such as `KeyError: 'R2'`.
- A **wrong result** raises nothing. The code finishes and reports the wrong number. These **silent bugs** cost most, because nobody knows to look.

Use precise words. A **defect** (bug) is the mistake in the code. A **failure** is what someone sees: a crash, or a total that is 3% too high. The **symptom** is where it shows. The **root cause** is the first place it went wrong, often far from the symptom.

Work through every bug with the same method:

1. **Reproduce** it, then shrink the input to the smallest one that still fails. Three rows beat three million.
2. **Read the evidence**: the last line of the traceback, or the first value that differs from what you expected.
3. **Form one hypothesis** that would explain everything you see.
4. **Test it** with one small check: an `assert`, a print of a single value, or the debugger (lesson D2-L1).
5. **Write a test** that fails because of the bug, so it cannot return unnoticed.
6. **Fix the root cause**, not the symptom, and watch the test pass.
7. **Look for the same mistake elsewhere.** Copied code copies bugs.

Two habits undo the method. Changing several things at once means you never learn which change mattered. Catching every exception and returning a default, such as `except Exception: return 0.0`, turns a loud failure into a silent one.

Why it matters

A weight loader caught every exception and returned 0.0 so that "the pipeline never breaks". When a supplier changed its file encoding, every read failed, every weight became 0.0, and the gold tables showed zero respondents for three regions. The pipeline reported success for a week. Without the catch-all, the first run would have stopped with a clear `UnicodeDecodeError` naming the file.

Worked example: weak code

```
def mean_weight(path: str) -> float:
    try:
        values = [float(line) for line in open(path)]
        return sum(values) / len(values)
    except Exception:
        return 0.0
```

1. **Line 5.** Catches everything: a missing file, a bad number, an empty file, a typo in the code.
2. **Line 6.** A plausible default. The failure becomes a silent wrong result.

Worked example: strong code

```
from pathlib import Path


def mean_weight(path: Path) -> float:
    """Mean of the weights in a file with one number per line.

    Raises:
        ValueError: If the file holds no weights, or a line is not a number.
    """
    values = [float(line) for line in path.read_text().splitlines() if line.strip()]
    if not values:
        raise ValueError(f"no weights in {path}")
    return sum(values) / len(values)
```

1. **Line 10.** A bad line raises ValueError naming the text, and a missing file raises FileNotFoundError. Both are loud and specific.
2. **Line 12.** The empty case is handled on purpose, with a message that names the file.

Reproduce small, read the evidence, test one hypothesis at a time, pin the bug with a failing test, fix the cause, and never silence an error with a default.

Recap card

- **Three kinds of problem, cheapest first** Syntax error (nothing runs), exception (crash with a traceback), wrong result (silent bug).
- **Defect, failure, root cause** Defect: the mistake in code. Failure: what someone sees. Root cause: the first place it went wrong.
- **The debugging method** Reproduce small, read the evidence, one hypothesis, one check, a failing test, fix the cause, look for the same mistake elsewhere.
- **Why except Exception: return 0.0 is dangerous** It turns a loud, specific failure into a silent wrong result.

Practice

Exercise 1. Multiple choice (level 1)

A nightly job finishes without errors, but the gold table's weight total is 3% higher than the delivery's. What kind of problem is this?

- a) A syntax error
- b) An exception
- c) A wrong result, a silent bug
- d) Not a bug, because nothing failed

Why?

- r1) The code ran to the end and produced a wrong number, with no error to point at it.
- r2) Silent bugs are always caused by bad data.

Exercise 2. Put in order (level 2)

Put the debugging method in order.

- Write a test that fails because of the bug.
- Fix the root cause and watch the test pass.
- Test the hypothesis with one small check.
- Form one hypothesis that explains everything you see.
- Read the evidence: the last traceback line or the first wrong value.
- Reproduce the failure and shrink the input to the smallest one that still fails.

Exercise 3. Predict the output (level 2)

What does this print?

```python
weights = {"R1": 1.0}
try:
    print(weights["R2"])
except KeyError as err:
    print("missing key:", err)
```

Exercise 4. Write the function (level 3)

Step 1 of the method is to shrink a failing input. Write `minimise(rows: list[int], fails: Callable[[list[int]], bool]) -> list[int]`.

- Raise `ValueError` if `fails(rows)` is `False`: there is nothing to shrink.
- Otherwise, repeatedly try removing one element at a time, from the left. Whenever the shorter list still fails, keep it and start again.
- Stop when no single removal still fails, and return that list.

Keep the original order, and do not change `rows`.

Your code must pass these checks:

- For 'any negative value fails', [3, -1, 4, -5] shrinks to [-5] when you remove from the left and restart after each success.
- For 'sum above 10 fails', [4, 1, 9, 2, 6] shrinks to [9, 6]. Restart from the left after each removal.
- Raise ValueError when the full input does not fail.
- Work on a copy: leave the list passed in unchanged.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Shrink a failing input by removing one element at a time.
def minimise(
    rows: list[int],
    fails: Callable[[list[int]], bool],
) -> list[int]: ...
```

Starter code:

```python
def minimise(rows, fails):
    return rows
```

Exercise 5. Spot the bug (level 3)

A file of weights with one corrupted line makes this function report a total of 0.0, and the pipeline carries on. Tap the **two** lines that turn the failure into a silent wrong result, then fix the function so bad input raises.

```python
def load_total(lines: list[str]) -> float:
    """Total weight from the lines of a delivery file."""
    try:
        return sum(float(line) for line in lines)
```

```
    except Exception:
        return 0.0
```

Write down the line number of the bug. Then write the fix.

Sources

- A missing dict key raises KeyError whose message is the key's repr ('R2'); float("n/a") raises ValueError. (verified: Run on Python 3.11 and 3.13.)

D0-L2 Read code you did not write

Concept

Most code you debug was written by someone else: a colleague, your past self or an AI assistant. Read it in a fixed order, so you understand it before you trust it or change it.

1. **The contract.** Read the name, the signature and the docstring. What does it promise to take and return? `get_unprocessed_waves(...) -> list[str]` promises a list and no side effects.
2. **Inputs and outputs.** Which values come in, and what leaves: the return value, a written table, a printed line?
3. **Side effects.** Look for anything that changes the world: writing files or tables, network calls, `print`, and changes to arguments, such as `list.remove` or `df[col] = ...` on a frame passed in. A function named `get_` or `check_` should have none.
4. **Trace by hand.** Pick a tiny input, three values, and follow the code line by line. Write each variable's value in a small table after every step. It is slow for one minute and saves an hour.
5. **Edges.** Ask what happens with an empty input, a zero, a `None`, a duplicate key and one row. Most bugs live there.
6. **Assumptions.** Note what the code takes for granted: sorted input, unique IDs, a column that is never missing. Each one is a question for its author, or a check to add.

Only then run it, on a small input whose right answer you worked out by hand.

While you read, watch for code that changes a list while looping over it. `for w in weights: weights.remove(w)` skips the element after each removal, so some values survive.

If the code is hard to read, restate it: rename variables, add type hints, and split it into small functions. Rewriting it clearly, with the same behaviour, is often how you find the bug.

Why it matters

A helper called `clean_weights` was reused in four notebooks. It also sorted the caller's list in place, which nobody noticed from its name. One notebook relied on the original order to pair weights with respondent IDs, and every respondent after the first got someone else's weight. Reading the helper for side effects, step 3 above, finds the `sort()` in seconds.

Worked example: weak code

```
1  def f(a, b):
2      r = []
3      for i in range(len(a)):
4          if a[i] is not None and b[i] > 0:
5              r.append(a[i] / b[i])
6      return r
```

1. **Line 1.** No contract: the name, the parameters and the result tell a reader nothing.
2. **Line 3.** Assumes b is at least as long as a, and says nothing if it is longer.
3. **Line 4.** Silently drops pairs, so the result can be shorter than the inputs.

Worked example: strong code

```
1  def ratios(numerators: list[float | None], denominators: list[float]) ->
       list[float]:
2      """Ratio of each pair, skipping missing numerators and non-positive
       denominators.
3
4      Raises:
5          ValueError: If the lists differ in length.
6      """
7      return [
8          num / den
9          for num, den in zip(numerators, denominators, strict=True)
10         if num is not None and den > 0
11     ]
```

1. **Line 1.** The restated contract: names and types say what goes in and what comes out.
2. **Line 9.** strict=True raises on unequal lengths, an assumption the original never checked.
3. **Line 10.** The skipping rule, now visible and documented.

Read the contract, the inputs and outputs and the side effects first, trace a tiny input by hand, then probe the edges and list the assumptions.

Recap card

- **Order for reading unfamiliar code** Contract, inputs and outputs, side effects, a hand trace on a tiny input, edges, assumptions. Then run it on a known answer.
- **Side effects to look for** Writes to files or tables, network calls, prints, and changes to arguments, such as list.remove or df[col] = ... on a frame passed in.
- **Why not remove items while looping over a list** Each removal shifts the rest left, so the loop skips the next element.
- **How to trace by hand** Pick three values and write each variable's value in a table after every step.

Practice

Exercise 1. Multiple choice (level 1)

You are handed a 60-line function to review. What do you read first?

- a) Its name, signature and docstring: what it promises.
- b) The most complicated loop, because bugs hide there.
- c) Nothing: run it on production data and see what happens.
- d) The last line, because that is what it returns.

Why?

- r1) You cannot judge whether code is right until you know what it is meant to do.
- r2) The signature is the shortest part.

Exercise 2. Predict the output (level 2)

Trace this by hand. What does it print?

```
def running_max(values):
    best = None
    out = []
    for v in values:
        if best is None or v > best:
            best = v
        out.append(best)
    return out


```

```
print(running_max([3, 1, 4, 1, 5]))
```

Exercise 3. Fill the gap (level 2)

Trace this code by hand and fill in `total` after each step.

```
total = 0
for v in [2, 5, 1]:
    total = total * 2 + v
```

```
step 1: v = 2, total = _____(1)
step 2: v = 5, total = _____(2)
step 3: v = 1, total = _____(3)
```

Exercise 4. Refactor (level 3)

Restate `f` clearly, with the same behaviour, as `ratios(numerators, denominators)`:

- type hints and a docstring,
- `zip(..., strict=True)` instead of indexing, so lists of different lengths raise `ValueError`,
- a list comprehension that skips a missing numerator or a denominator that is not positive.

Your code must pass these checks:

- Skip a missing numerator and a non-positive denominator, and divide the rest: [0.5, 0.5].
- A numerator of 0.0 is a real value. Test for a missing one with `is None`.
- Empty inputs give an empty list.
- Lists of different lengths must raise ValueError. zip(..., strict=True) does that.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Ratio of each pair, skipping missing numerators and non-positive
    denominators.
def ratios(
    numerators: list[float | None],
    denominators: list[float],
) -> list[float]: ...
```

Starter code:

```
def f(a, b):
    r = []
    for i in range(len(a)):
        if a[i] is not None and b[i] > 0:
```

```
            r.append(a[i] / b[i])
    return r
```

Exercise 5. Spot the bug (level 3)

`drop_negative([-1, -2, 3])` returns `[-2, 3]`, and it changes the caller's list too. Tap the line with the bug, then fix it so it returns a new list and leaves the input alone.

```
def drop_negative(weights: list[float]) -> list[float]:
    """Return the weights without negative values."""
    for w in weights:
        if w < 0:
            weights.remove(w)
    return weights
```

Write down the line number of the bug. Then write the fix.

Sources

- Removing items from a list inside a for loop over it skips elements: drop_negative([-1, -2, 3]) returns [-2, 3]; zip(..., strict=True) raises ValueError on unequal lengths (Python 3.10 and later). (verified: Run on Python 3.11 and 3.13.)

D1 Reading tracebacks and stack traces

Roadmap: Every month. Find the root cause in Python and Spark errors.

D1-L1 Read a Python traceback from the bottom up

Concept

A traceback lists the calls that led to an error, oldest first. The line that matters most is at the **bottom**. Read it in this order:

1. **The last line** names the exception type and message: `NameError: name 'uuid' is not defined`. That is what went wrong.
2. **The frame just above it** shows where: file, line number, function, and the source line. Python 3.11 and later add `^` or `~` marks under the exact expression.
3. **Walk up** until you reach the first frame in **your own code**. Frames inside `site-packages` or the standard library are rarely the cause. The call from your code that fed them bad input usually is.

Here is your reference `log_pipeline_run`, cut down, on Python 3.13:

```
Traceback (most recent call last):
  File "observability.py", line 7, in <module>
    main()
    ~~~~^^
  File "observability.py", line 5, in main
    log_run()
    ~~~~~~~^^
  File "observability.py", line 2, in log_run
    return str(uuid.uuid4())
               ^^^^
NameError: name 'uuid' is not defined. Did you forget to import 'uuid'?
```

Python 3.12 added the "Did you forget to import" hint. Python 3.11 prints the same error without it.

Chained exceptions print two tracebacks. The joining sentence tells you how they relate:

- "**The above exception was the direct cause of the following exception**" means someone wrote `raise NewError(...) from err`. The first traceback is the root cause.
- "**During handling of the above exception, another exception occurred**" means a second error happened inside an `except` block. Your error handler has its own bug. Fix both.

When you catch an exception to add context, use `raise ... from err`. The original traceback stays attached, so the next person sees both.

Never read only the first line of a traceback, and never read only the message. `KeyError: 'row_count_max'` names the missing key. The frame tells you which dictionary, and so which config file, lacks it.

Why it matters

A contract validator failed with a 60-line traceback. The on-call engineer read the top frames, which pointed at the job scheduler, and restarted the job three times. The last line said `KeyError: 'row_count_max'`, and the frame above it named `_check_row_count`. The new tracker's contract file lacked one key. Reading from the bottom would have found it in a minute.

Worked example: weak code

```
def load_contract(path: str) -> dict[str, object]:
    """Load a contract file."""
    try:
        with open(path) as f:
            return json.load(f)
    except Exception:
        raise ValueError("bad contract")
```

1. **Line 6.** Catches everything, including a typo in your own code.
2. **Line 7.** No path, no cause. The traceback prints 'During handling...', and the reader must guess which error was the real one.

Worked example: strong code

```
import json


def load_contract(path: str) -> dict[str, object]:
    """Load a contract file.

    Raises:
        ValueError: If the file is not valid JSON.
    """
    try:
        with open(path, encoding="utf-8") as f:
            return json.load(f)
    except json.JSONDecodeError as err:
        raise ValueError(f"contract {path} is not valid JSON: line
    {err.lineno}") from err
```

1. **Line 13.** Catch only the error you expect. A missing file still raises FileNotFoundError with its own traceback.

2. **Line 14.** The message names the file and line. `from err` keeps the original traceback as the direct cause.

Read the last line, then the frame above it, then walk up to your own code. When you re-raise, add context and chain with `from err`.

Recap card

- **Order for reading a traceback** Last line (what), frame above it (where), then up to the first frame in your own code.
- **'The above exception was the direct cause...'** Someone chained with raise ... from err. The first traceback is the root cause.
- **'During handling of the above exception...'** The except block raised a new error. Fix the handler and the original error.
- **Re-raising with context** Catch the narrow exception, add facts to the message, and chain with from err.

Practice

Exercise 1. Multiple choice (level 1)

A traceback is 40 lines long. Where do you start reading?

- a) The last line: exception type and message
- b) The first line after 'Traceback (most recent call last)'
- c) The longest frame
- d) The first frame inside site-packages

Why?

- r1) Frames run oldest first. The exception and the frame that raised it are at the bottom.
- r2) Python prints the most important frame last to save space.

Exercise 2. Spot the bug (level 2)

This traceback came from Python 3.13. Tap the frame line in **your own code** where you would start fixing. Your code is in `/repo/pipelines/`.

```
Traceback (most recent call last):
  File "/repo/pipelines/run_qa.py", line 31, in <module>
    report = check_weights(df, "final_weight")
  File "/repo/pipelines/qa_statistical.py", line 19, in check_weights
    "mean": round(stats["mean"], 4),
            ~~~~~^^^^^^^^^^^^^^^^^^
TypeError: type NoneType doesn't define __round__ method
```

Write down the line number of the bug.

Exercise 3. Multiple choice (level 2)

A traceback shows a `KeyError`, then the sentence "During handling of the above exception, another exception occurred", then a `NameError`. What does that tell you?

- a) The except block that handled the KeyError has its own bug
- b) The code used raise ... from err
- c) Only the NameError matters
- d) Python retried the code

Why?

- r1) The second exception was raised while handling the first, without an explicit chain. Both need a fix.
- r2) NameError always hides a KeyError.

Exercise 4. Write the function (level 3)

Write `first_own_frame(traceback_text: str, project_dir: str) -> tuple[str, int] | None`. Return the file path and line number of the **last** frame line (`File "<path>", line <n>, in <name>`) whose path starts with `project_dir`. That is the first frame in your code when you read from the bottom. Return `None` if no frame is in your code. For chained tracebacks, use the last traceback in the text.

Your code must pass these checks:

- Skip library frames and return the last frame inside project_dir, not the first.
- Return None when no frame is in your code.
- Use the last traceback in the text, and its last frame in your code.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
FRAME_PATTERN = ...

# Find the frame to start from when reading a traceback bottom up.
def first_own_frame(
    traceback_text: str,
    project_dir: str,
) -> tuple[str, int] | None: ...
```

Starter code:

```python
def first_own_frame(traceback_text, project_dir):
    for line in traceback_text.splitlines():
        if project_dir in line:
```

```
            return line
```

Exercise 5. Refactor (level 4)

Refactor `row_count_bounds` so a missing key raises `ValueError` naming the contract id and the missing key, chained to the original `KeyError` with `from`. Add type hints and a docstring.

Your code must pass these checks:

- Return (min, max) for a complete contract.
- Raise ValueError naming the contract and the key, with `from err` so the KeyError is the direct cause.
- Catch only KeyError. Other errors, such as passing None, must surface unchanged.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Row count bounds from a data contract.
def row_count_bounds(
    contract: dict[str, object],
) -> tuple[object, object]: ...
```

Starter code:

```
def row_count_bounds(contract):
    try:
        return contract["row_count_min"], contract["row_count_max"]
    except Exception:
        raise ValueError("bad contract")
```

Sources

- Python 3.11+ prints caret or tilde markers under the failing expression; 3.12 and 3.13 add 'Did you forget to import' to NameError for standard modules; 3.11 does not. (verified: Local runs on CPython 3.11.15, 3.12.3 and 3.13.12, September 2026.)
- raise ... from err prints 'The above exception was the direct cause of the following exception'; an error inside an except block prints 'During handling of the above exception, another exception occurred'. (verified: Local runs on CPython 3.11.15 and 3.13.12, September 2026.)
- round(None, 4) raises TypeError: type NoneType doesn't define **round** method. (verified: Local run on CPython 3.13.12, September 2026.)

D1-L2 Spark errors, the condition and the real cause

Concept

A Spark error message is long, but it has a fixed shape. In PySpark 4.0 (Databricks Runtime 17.3 LTS) most errors start with an **error condition** in square brackets and end with a **SQLSTATE**:

```
[UNRESOLVED_COLUMN.WITH_SUGGESTION] A column, variable, or function
parameter with name `final_wieght` cannot be resolved. Did you mean one
of the following? [`final_weight`, `age`, `wave_id`, `respondent_id`].
SQLSTATE: 42703
```

Read three things:

- **The condition** names the problem. Search for it in the Databricks error conditions reference.
- **The SQLSTATE class**, its first two characters. Class **42** is a syntax error or access rule violation: your code names something that does not exist. Class **22** is a data exception: the code is valid but a value is not.
- **The origin line.** Runtime errors show `== DataFrame ==` and `"cast" was called from ...` with your file and line.

When the error appears. Transformations are lazy, so a data error appears at the **action** (`collect`, `count`, `write`), not at the line that caused it. The origin line points you back. In classic Spark, a missing column fails at once, when you call `select`.

Python UDF errors arrive as `PythonException` wrapping a normal Python traceback. Read that inner traceback bottom up (lesson D1-L1).

Shadowed built-ins give a misleading message. Your `data_contracts.py` imports `sum` from `pyspark.sql.functions`. Then `sum(1 for c in checks)` raises `[NOT_COLUMN_OR_STR] Argument col should be a Column or str, got generator`. The code never meant to call Spark. Import `functions as F`.

ANSI mode is on by default in Spark 4.0 and Databricks Runtime 17.0 and above. A cast of `'abc'` to `INT` now raises `[CAST_INVALID_INPUT]` instead of returning null. Use `try_cast` where bad values are expected, and count the nulls it produces.

In code, catch `pyspark.errors.AnalysisException` and read `err.getCondition()` and `err.getSqlState()`, not the message text.

Why it matters

After a runtime upgrade to Databricks Runtime 17.3 LTS, a silver job that had run for two years failed with `[CAST_INVALID_INPUT] The value 'n/a' ... cannot be cast to "INT"`. The engineer's first thought was a Spark bug. The SQLSTATE started with 22, a data exception: the data had always held 'n/a', and the old runtime had turned it into null without a word. The new error exposed a data quality problem that was two years old.

Worked example: weak code

```
1 from pyspark.sql.functions import col, sum
2
3 silver = bronze.withColumn("age", col("age").cast("int"))
4 passed = sum(1 for c in checks if c.status == "PASS")
5 silver.write.mode("append").saveAsTable("research_silver.nat_respondents")
```

1. **Line 1.** Shadows Python's sum. The next line fails with NOT_COLUMN_OR_STR.
2. **Line 3.** Under ANSI mode, one bad value fails the whole write, several lines later.

Worked example: strong code

```
1 from pyspark.sql import functions as F
2
3 silver = bronze.withColumn("age", F.col("age").try_cast("int"))
4 bad_ages = bronze.filter(F.col("age").isNotNull() &
      F.col("age").try_cast("int").isNull()).count()
5 passed = sum(1 for c in checks if c.status == "PASS")
6 silver.write.mode("append").saveAsTable("research_silver.nat_respondents")
```

1. **Line 1.** Python's sum stays Python's sum.
2. **Line 3.** Bad values become null on purpose.
3. **Line 4.** Then you count them, so nothing disappears silently.

Read the condition, the SQLSTATE class and the origin line. Class 42 means fix the code. Class 22 means look at the data.

Recap card

- **Shape of a Spark 4 error message** [ERROR_CONDITION] message ... SQLSTATE: xxxxx, often with an origin line under == DataFrame ==.
- **SQLSTATE class 42 vs 22** 42: syntax error or access rule violation, fix the code. 22: data exception, look at the data.

- **Why a data error appears at write()** Transformations are lazy. The action runs the plan. The origin line shows where the expression was written.
- **NOT_COLUMN_OR_STR ... got generator** Python's sum is shadowed by pyspark.sql.functions.sum. Import functions as F.

Practice

Exercise 1. Multiple choice (level 1)

A job fails with `[UNRESOLVED_COLUMN.WITH_SUGGESTION] ... name` final_wieght `cannot be resolved. Did you mean one of the following? [` final_weight `, ...]. SQLSTATE: 42703`. What do you fix?

- a) The column name in the code: final_wieght is a typo for final_weight
- b) The data: some rows lack final_weight
- c) The cluster: restart it
- d) The table permissions

Why?

- r1) Class 42 errors mean the code names something that does not exist. The message suggests the right name.
- r2) Spark always suggests the column you meant.

Exercise 2. Multiple choice (level 2)

`pass_rate` in your `data_contracts.py` runs `sum(1 for c in self.checks if c.status == "PASS")` and fails with `[NOT_COLUMN_OR_STR] Argument` col `should be a Column or str, got generator`. What is the cause?

- a) `sum` is PySpark's sum, imported by name, which shadows Python's built-in
- b) self.checks is a DataFrame
- c) Generators are not allowed in dataclasses
- d) The status column is missing

Why?

- r1) `from pyspark.sql.functions import sum` replaces the built-in in that module, so the generator goes to Spark's sum.
- r2) PySpark disables Python built-ins in notebooks.

Exercise 3. Spot the bug (level 3)

This job fails on line 5 with `[CAST_INVALID_INPUT] The value 'n/a' of the type "STRING" cannot be cast to "INT"` and `"cast" was called from ... line 3`. Tap the line that causes it, then fix it so bad ages become null. Keep `functions as F`.

```
1 from pyspark.sql import functions as F
2
3 silver = bronze.withColumn("age", F.col("age").cast("int"))
4 silver = silver.withColumn("wave_id", F.lit(wave_id))
5 silver.write.mode("append").saveAsTable("research_silver.nat_respondents")
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 3)

Write `triage(message: str) -> tuple[str | None, str]` for Spark error messages. Extract the error condition, the text in the first `[...]` at the start of the message, or `None`. Then classify by the SQLSTATE at the end (`SQLSTATE: 42703`): class `"42"` gives `"code"`, class `"22"` gives `"data"`, anything else or no SQLSTATE gives `"unknown"`.

Your code must pass these checks:

- Class 42 is a code error. Extract the condition inside the first brackets.
- Class 22 is a data exception.
- Read the class from the SQLSTATE, not from any '42' in the message.
- No brackets and no SQLSTATE: (None, 'unknown').

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
1  # Module constant.
2  CONDITION_PATTERN = ...
3
4  # Module constant.
5  SQLSTATE_PATTERN = ...
6
7  # Module constant.
8  CLASS_LABELS = ...
9
10 # Error condition and a first-pass category for a Spark error message.
11 def triage(message: str) -> tuple[str | None, str]: ...
```

Starter code:

```
1 def triage(message):
2     if "42" in message:
3         return None, "code"
4     return None, "data"
```

Exercise 5. Architecture design (level 4)

Write the first-response steps for your team's runbook when a Spark job fails.

Options: The error condition, the SQLSTATE and the origin line, The Java stack trace from the top.

1. First thing to read
2. SQLSTATE class 22 after a runtime upgrade
3. Handling errors in code

Sources

- PySpark 4.0.4 error texts: UNRESOLVED_COLUMN.WITH_SUGGESTION with SQLSTATE 42703; CAST_INVALID_INPUT with SQLSTATE 22018 and an origin line; PythonException wrapping a Python traceback; NOT_COLUMN_OR_STR for a generator passed to shadowed sum; getCondition() and getSqlState() on AnalysisException; Column.try_cast exists. (verified: Local runs on PySpark 4.0.4, September 2026.)
- ANSI mode is enabled by default in Apache Spark 4.0 and Databricks Runtime 17.0 and above. (verified: https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-ansi-compliance (checked 2026-09-25))
- SQLSTATE class 22 is data exception; class 42 is syntax error or access rule violation. (verified: https://docs.databricks.com/aws/en/error-messages/sqlstates (checked 2026-09-25))

D1-L3 Py4J and Spark Connect errors

Concept

PySpark talks to the Spark engine in one of two ways, and each shapes the errors you see. Facts below were checked on PySpark 4.0.4 and the Databricks docs in September 2026.

Spark Classic runs your Python next to a JVM driver through **Py4J**. PySpark converts most engine errors into Python classes such as `AnalysisException`. Some still arrive raw, as `py4j.protocol.Py4JJavaError`:

```
Py4JJavaError: An error occurred while calling
z:org.apache.spark.api.python.PythonRDD.collectAndServe.
: org.apache.spark.SparkException: Job aborted due to stage failure ...
Caused by: org.apache.spark.api.python.PythonException: Traceback ...
  File "/repo/job.py", line 5, in <lambda>
ZeroDivisionError: division by zero
```

Skip the Java frames. Search for the **last** `Caused by:` line and the Python traceback inside it. That is your bug.

Spark Connect sends a query plan to a remote server over gRPC. Databricks **serverless compute uses Spark Connect**, and supports only Spark Connect APIs. Three differences matter when you debug:

- **Analysis is deferred.** Classic Spark rejects a misspelt column when you call `select`. Spark Connect accepts the `select` and raises `AnalysisException` at the action, or when you read `df.columns` or `df.schema`. A `try` around the `select` catches nothing.
- **Error classes live in** `pyspark.errors.exceptions.connect`**.** They subclass `SparkConnectGrpcException` and the familiar base classes, so `except pyspark.errors.AnalysisException` still works. The message ends with a `JVM stacktrace:` section from the server.
- **JVM-only attributes fail.** `spark.sparkContext` raises `[JVM_ATTRIBUTE_NOT_SUPPORTED]`. RDD code, such as `sc.parallelize`, does not run on serverless.

Two habits follow. Catch `pyspark.errors` classes, never `Py4JJavaError`, so the same code works on both. And place error handling around the **action**, where both modes raise, not around the transformation.

Why it matters

A notebook wrapped `df.select(col(name))` in `try/except AnalysisException` to skip columns missing from older waves. It worked on a classic cluster. After the team moved the job to serverless, the `except` never fired: the `select` succeeded, and the `write` failed with the same `AnalysisException` two cells later. Checking `name in df.columns` before the `select` works in both modes.

Worked example: weak code

```
1 from py4j.protocol import Py4JJavaError
2 from pyspark.sql import functions as F
3
4 try:
2     selected = df.select(F.col(optional_column))
6 except Py4JJavaError:
7     selected = df
8 selected.write.mode("append").saveAsTable(target_table)
```

1. **Line 1.** Py4J exists only in Spark Classic. Serverless has no Py4J.
2. **Line 5.** Under Spark Connect this line never raises. The error arrives at the write.

Worked example: strong code

```
1 from pyspark.errors import AnalysisException
2 from pyspark.sql import functions as F
3
(1) columns = [c for c in (key_column, optional_column) if c in df.columns]
5 selected = df.select(*[F.col(c) for c in columns])
6 try:
7     selected.write.mode("append").saveAsTable(target_table)
(2) except AnalysisException as err:
9     raise RuntimeError(f"write to {target_table} failed:
    {err.getCondition()}") from err
```

1. **Line 4.** Check the schema explicitly. Works the same in Classic and Connect.
2. **Line 8.** A pyspark.errors class, caught around the action, where both modes raise.

Find the last "Caused by" in a Py4J error. Under Spark Connect, expect errors at the action, and catch pyspark.errors classes.

Recap card

- **Finding the cause in a Py4JJavaError** Search for the last 'Caused by:'. A Python UDF error holds a Python traceback there.
- **Spark Connect and missing columns** Analysis is deferred. The error comes at the action or when you read df.columns or df.schema.
- **Exception class that works on Classic and Connect** pyspark.errors.AnalysisException (and other pyspark.errors classes). Not Py4JJavaError.
- **Serverless compute and the Spark API** Spark Connect only. No RDD APIs. spark.sparkContext raises JVM_ATTRIBUTE_NOT_SUPPORTED.

Practice

Exercise 1. Multiple choice (level 1)

A raw `Py4JJavaError` fills 200 lines, mostly Java frames. Where is the cause of a Python UDF failure?

- a) After the last 'Caused by:', in the Python traceback it contains
- b) In the first Java frame
- c) In the line 'An error occurred while calling'
- d) It is not shown. You must read the executor logs

Why?

- r1) The JVM wraps the Python worker's exception. The innermost cause holds the Python traceback.

- r2) Java always prints the cause first.

Exercise 2. Predict the output (level 2)

This runs on **serverless** (Spark Connect). The DataFrame has no column `region`. What does it print?

```
from pyspark.errors import AnalysisException
from pyspark.sql import functions as F

try:
    out = df.select(F.col("region"))
    print("select ok")
except AnalysisException:
    print("caught")
```

Exercise 3. Multiple choice (level 2)

Which `except` clause catches a missing-column error on both a classic cluster and serverless?

- a) except pyspark.errors.AnalysisException
- b) except py4j.protocol.Py4JJavaError
- c) except pyspark.errors.exceptions.connect.SparkConnectGrpcException
- d) except KeyError

Why?

- r1) Both modes raise a subclass of pyspark.errors.AnalysisException.
- r2) AnalysisException is a Py4J class.

Exercise 4. Write the function (level 3)

Write `python_cause(message: str) -> str | None`. Given the text of a Py4J error, return the last line of the Python traceback in the **last** `Caused by: org.apache.spark.api.python.PythonException` section: the first non-blank line that does not start with whitespace, `Traceback` or `File`, after that marker. Return `None` if there is no such section.

Your code must pass these checks:

- Return the exception line after the PythonException marker, skipping frames and Java lines.
- Use the last PythonException section.
- No PythonException section: return None.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
```

```
MARKER = ...

# Python exception line inside the last PythonException of a Py4J error.
def python_cause(message: str) -> str | None: ...
```

Starter code:

```
def python_cause(message):
    return message.splitlines()[-1]
```

Exercise 5. Architecture design (level 4)

A notebook written for a classic cluster moves to serverless. Plan the changes to its error handling.

Options: Check `name in df.columns` before selecting, Wrap each select in try/except.

1. Optional columns in older waves
2. Exception class to catch
3. A helper that uses sc.parallelize

Sources

- Classic: raw Py4JJavaError for RDD Python failures with 'Caused by: org.apache.spark.api.python.PythonException' holding the Python traceback; eager analysis at select. Connect: analysis deferred to action or schema access; errors in pyspark.errors.exceptions.connect subclass SparkConnectGrpcException and are caught by pyspark.errors.AnalysisException; message includes 'JVM stacktrace:'; spark.sparkContext raises JVM_ATTRIBUTE_NOT_SUPPORTED. (verified: Local runs on PySpark 4.0.4, classic local[1] and remote('local[1]'), September 2026.)
- Serverless compute supports only Spark Connect APIs, not RDD APIs, and defers analysis and name resolution to execution time. (verified: https://docs.databricks.com/aws/en/compute/serverless/limitations and https://docs.databricks.com/aws/en/spark/connect-vs-classic (checked 2026-09-25))

D2 Debugging tools

Roadmap: Every month. breakpoint(), pdb, logging levels, Spark UI and explain().

D2-L1 breakpoint() and pdb, stop and look

Concept

A traceback shows where code failed. A **debugger** shows the values at that moment. Python ships one: **pdb**.

Stop at a line. Put `breakpoint()` in your code. When Python reaches it, pdb opens a prompt, `(Pdb)`, with every local variable in scope. Setting the environment variable `PYTHONBREAKPOINT=0` turns every `breakpoint()` call into a no-op, so a forgotten one cannot hang a scheduled job. Remove them before you commit anyway.

Stop where it crashed. Run a script under pdb with post-mortem on:

```
python -m pdb -c continue run_qa.py
...
ZeroDivisionError: float division by zero
Uncaught exception. Entering post mortem debugging
> /repo/run_qa.py(2)weighted_pct()
-> ratio = reach / base
(Pdb) p reach, base
(5.0, 0.0)
```

You land in the frame that raised, with its variables intact. Checked on Python 3.13.

The commands you use most:

Command	Does
`p expr`, `pp expr`	Print, or pretty-print, a value
`w`	Show the stack, newest frame last
`u`, `d`	Move up to the caller, or down again
`n`	Run the next line, stepping over calls
`s`	Step into a call
`c`	Continue to the next breakpoint
`l`	List source around the current line
`q`	Quit

A useful pattern: after a crash, `p` the inputs of the failing frame. If they look wrong, `u` to the caller and `p` what it passed. Repeat until you find the first frame where a value went bad. That is the frame to fix.

In Databricks notebooks, the built-in interactive debugger gives breakpoints, stepping and a variable view for Python cells. It runs on serverless and on recent runtimes, after you turn it on in your editor settings.

In Spark code a debugger stops only the driver. Code inside a UDF runs on executors, where your breakpoint never fires. Test UDF logic as a plain Python function first.

Why it matters

A weighted-reach function returned 0.0 for one subgroup. An analyst added eleven print statements over two hours and still missed it. A post-mortem pdb session on a forced failure showed in one command that `base` was the unweighted count, passed in by a caller that swapped two arguments. `u`, then `p`, found the caller.

Worked example: weak code

```
1  def weighted_pct(reach: float, base: float) -> float:
2      """Weighted percentage."""
(1)    print("DEBUG reach", reach)
4      print("DEBUG base", base)
(2)    breakpoint()
6      return round(reach / base * 100, 1)
```

1. **Line 3.** Print debugging leaves noise in job logs and shows only what you guessed to print.
2. **Line 5.** Committed by mistake, it waits for input in a job that nobody is watching.

Worked example: strong code

```
1  def weighted_pct(reach: float, base: float) -> float | None:
2      """Weighted percentage, or None when the base is zero."""
(1)    if base == 0:
4          return None
5      return round(reach / base * 100, 1)
6
(2) # To investigate a failure, run: python -m pdb -c continue run_qa.py
8  # then: p reach, base   u   p rows
```

1. **Line 3.** The fix the debugging session pointed to: guard the division.
2. **Line 7.** Debug from outside the code. Nothing to remove before you commit.

Let the crash open the debugger, print the failing frame's inputs, then walk up until the values go wrong.

Recap card

- **Open pdb where a script crashed** python -m pdb -c continue script.py. You land in the frame that raised.
- **pdb: print, stack, up, next, step, continue** p, w, u, n, s, c.
- **PYTHONBREAKPOINT=0** Makes breakpoint() a no-op. A safety net for jobs, not a reason to commit breakpoints.
- **Why a breakpoint in a Spark UDF never fires** UDFs run on executors. The debugger stops only the driver. Test UDF logic as plain Python.

Practice

Exercise 1. Multiple choice (level 1)

A scheduled job runs with `PYTHONBREAKPOINT=0`. The code contains `breakpoint()`. What happens at that line?

- a) Nothing: the call does nothing and the job continues
- b) The job waits for input forever
- c) The job fails with a RuntimeError
- d) pdb opens and quits at once

Why?

- r1) PYTHONBREAKPOINT=0 makes breakpoint() a no-op.
- r2) Scheduled jobs never run pdb.

Exercise 2. Fill the gap (level 2)

You are in a post-mortem pdb session in `weighted_pct`. Complete the commands.

```
(Pdb) _____(1) reach, base        # print the inputs
(Pdb) _____(2)                    # move to the caller's frame
(Pdb) p rows
```

Exercise 3. Put in order (level 2)

Order a debugging session for a function that returns a wrong number.

- Stop in the function with breakpoint() or post-mortem pdb
- Print the inputs, walk up with u until a value is first wrong
- Reproduce with the smallest input that gives the wrong number
- Fix that frame and add a test with the small input

- Remove the breakpoint and run the tests

Exercise 4. Write the function (level 3)

Post-mortem debugging reads the local variables of the frame that raised. Write `crash_locals(fn: Callable[..., object], *args: object) -> dict[str, object] | None` that calls `fn(*args)`. If it raises, return a copy of the local variables of the **innermost** frame of the traceback (the one pdb lands in). If it does not raise, return `None`. Import `Callable` from `collections.abc`.

What you need to know.

An exception carries its traceback in `err.__traceback__`. A traceback is a chain of frames, outermost first: `tb.tb_next` moves one call deeper and is `None` at the innermost frame. `tb.tb_frame` is the frame itself, and `tb.tb_frame.f_locals` holds its local variables as a dict.

```
def inner():
    raise ValueError

def outer():
    count = 3
    inner()

try:
    outer()
except Exception as err:
    tb = err.__traceback__              # the frame that called outer()
    tb.tb_next.tb_frame.f_locals        # {'count': 3}: outer's locals
```

Your code must pass these checks:

- Return the locals of the innermost frame, where the error was raised, not the caller's or your own.
- Return None when the call succeeds.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Local variables of the frame that raised, as pdb post-mortem shows them.
def crash_locals(
    fn: Callable[..., object],
    *args: object,
) -> dict[str, object] | None: ...
```

Starter code:

```
def crash_locals(fn, *args):
    try:
        fn(*args)
    except Exception:
        return locals()
```

Exercise 5. Spot the bug (level 3)

This function is ready for a scheduled job, except for one line. Tap it, then remove it and anything else left from debugging.

```
1 def unweighted_base(rows: list[dict[str, object]]) -> int:
2     """Count in-scope respondents."""
3     in_scope = [r for r in rows if r["in_scope"] is True]
4     breakpoint()
5     return len(in_scope)
```

Write down the line number of the bug. Then write the fix.

Sources

- python -m pdb -c continue enters post-mortem at the raising frame and prints 'Uncaught exception. Entering post mortem debugging'; PYTHONBREAKPOINT=0 disables breakpoint(); commands p, w, u, d, n, s, c, l, q. (verified: Local runs on CPython 3.13.12, September 2026.)
- Databricks notebooks have a built-in interactive debugger for Python, available on serverless and recent runtimes, turned on in editor settings. (verified: https://docs.databricks.com/aws/en/notebooks/debugger (checked 2026-09-25))
- ruff rule T100 flags debugger calls such as breakpoint(). (verified: ruff 0.16.9 run in this repo reports 'T100 Trace found: `breakpoint` used'.)

D2-L2 Logging levels that help at 3 a.m.

Concept

A job that failed overnight leaves you its logs and nothing else. Good logs answer three questions fast: what ran, with what inputs, and where it stopped. Python's `logging` module does this better than `print`.

One logger per module:

```
1 import logging
2
3 logger = logging.getLogger(__name__)
```

Levels, lowest to highest, with their numbers:

Level	Number	Use it for
DEBUG	10	Detail you need only while investigating
INFO	20	Normal progress: step started, rows written
WARNING	30	Unexpected but handled: a WARN check

Level	Number	Use it for
ERROR	40	A step failed
CRITICAL	50	The pipeline cannot continue

A logger passes on records at or above its effective level. The root logger's default is **WARNING**, so `logger.info(...)` prints nothing until you configure logging. Checked on Python 3.13:

- `logging.basicConfig(level=logging.INFO)` configures the root logger **once**. A second call does nothing, unless you pass `force=True`.
- `logger.exception("...")` inside an `except` block logs at ERROR and appends the traceback.
- Pass values as arguments: `logger.info("wrote %d rows to %s", n, table)`. The string is built only if the record is emitted. ruff rule G004 flags f-strings in logging calls.

Logging is not control flow. `logger.critical(...)` records a message. It does not stop anything. Your standard says a CRITICAL check **raises**. Log, then raise.

Logging is not your run log. Text logs are for people debugging. The Delta run log table, with one structured row per run, is for queries and dashboards. Write both.

Every log line in a pipeline should carry the identifiers that let you find the data: tracker, wave, table, run id.

Why it matters

A silver job logged "Processing..." and "Done" at INFO for every wave, and nothing else. When wave W14 produced 30% fewer rows, the logs could not say which step dropped them. Adding one INFO line per step, with the row count, tracker and wave, turned the next investigation from a day of reruns into a two-minute log search.

Worked example: weak code

```
import logging

def run_checks(results: list[dict[str, str]], wave_id: str) -> None:
    """Report check results."""
    for r in results:
        if r["status"] == "FAIL" and r["severity"] == "CRITICAL":
            logging.critical(f"check failed: {r['check']}")
```

```
(2)          print("checked", r["check"])
```

1. **Line 7.** Logs CRITICAL and carries on. The pipeline continues with bad data. The f-string is built even when filtered (G004).
2. **Line 8.** print has no level, no logger name and no wave id.

Worked example: strong code

```
 1 import logging
 2
(1) logger = logging.getLogger(__name__)
 4
 5
 6 class CriticalCheckFailed(RuntimeError):
 7     """A CRITICAL data check failed and the pipeline must stop."""
 8
 9
10 def run_checks(results: list[dict[str, str]], wave_id: str) -> None:
11     """Log each check result and stop on a CRITICAL failure."""
12     for r in results:
(2)          logger.info("wave %s check %s status %s", wave_id, r["check"],
   r["status"])
14         if r["status"] == "FAIL" and r["severity"] == "CRITICAL":
15             logger.critical("wave %s CRITICAL check %s failed", wave_id,
   r["check"])
(3)             raise CriticalCheckFailed(f"{r['check']} failed for wave
   {wave_id}")
```

1. **Line 3.** A named logger per module. The name shows up in every record.
2. **Line 13.** Identifiers in every line, formatted only when emitted.
3. **Line 16.** Log, then raise. The level describes. The exception stops.

One named logger per module, levels that match severity, identifiers in every line, and a raise after every CRITICAL.

Recap card

- **Logging levels and numbers** DEBUG 10, INFO 20, WARNING 30, ERROR 40, CRITICAL 50. Default effective level: WARNING.
- **basicConfig called twice** The second call does nothing, unless force=True.
- **logger.exception** Logs at ERROR with the traceback. Call it inside an except block.
- **Does logger.critical stop the pipeline?** No. Log, then raise.

Practice

Exercise 1. Predict the output (level 1)

Nothing has configured logging. What does this print?

```
import logging

logger = logging.getLogger("qa")
logger.info("rows written")
logger.warning("weight sum low")
```

Exercise 2. Multiple choice (level 2)

A notebook calls `logging.basicConfig(level=logging.WARNING)` in its first cell, then `logging.basicConfig(level=logging.DEBUG)` later. What is the root level after the second call?

- a) WARNING: the second call does nothing
- b) DEBUG
- c) An error is raised
- d) Both handlers are active

Why?

- r1) basicConfig does nothing if the root logger already has handlers, unless force=True.
- r2) Logging levels can never be lowered.

Exercise 3. Fill the gap (level 2)

Log the failure with its traceback, then stop the pipeline.

```
try:
    validate(df)
except ContractError:
    logger._____(1)("contract failed for wave %s", wave_id)
    _____(2)
```

Exercise 4. Write the function (level 3)

Write `log_check(logger: logging.Logger, check: str, status: str, severity: str, wave_id: str) -> None`. Log at INFO for `"PASS"`, WARNING for `"WARN"`, ERROR for `"FAIL"` with non-CRITICAL severity, and CRITICAL for `"FAIL"` with `"CRITICAL"` severity, then raise `RuntimeError`. Every message contains the wave id and check name. Use %-style arguments, not f-strings, in the logging calls. Raise `ValueError` for an unknown status.

What you need to know.

`logging.Logger` is the type of the object `logging.getLogger(name)` returns. Each level is a number: `logging.INFO` is 20, `WARNING` 30, `ERROR` 40, `CRITICAL` 50. `logger.log(level, message, *args)` logs at a level you pass in, so a dict can map each status to its level.

Pass values as arguments after a `%s` template. Logging fills them in only when the message is actually emitted:

```
logger.log(logging.WARNING, "wave %s has %d rows", "W1", 0)
# WARNING wave W1 has 0 rows
```

Your code must pass these checks:

- PASS is INFO, WARN is WARNING, a non-critical FAIL is ERROR. Include the wave and check in each message.
- A CRITICAL failure logs at CRITICAL and then raises RuntimeError.
- Unknown statuses raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
STATUS_LEVELS = ...

# Log one check result at a level matching its outcome.
def log_check(
    logger: logging.Logger,
    check: str,
    status: str,
    severity: str,
    wave_id: str,
) -> None: ...
```

Starter code:

```
import logging


def log_check(logger, check, status, severity, wave_id):
    logger.info(f"{check} {status}")
```

Exercise 5. Architecture design (level 4)

Set the logging for a silver job that runs nightly on Databricks.

Options: INFO, with DEBUG switched on by a job parameter when investigating, DEBUG always.

1. Level in production
2. Each step's log line
3. Run history for dashboards

Sources

- Default effective level WARNING; unconfigured output is the bare message; basicConfig second call no-op unless force=True; logger.exception logs at ERROR with traceback; level numbers 10 to 50. (verified: Local run on CPython 3.13.12, September 2026.)
- ruff rule G004 flags f-strings in logging calls. (verified: ruff 0.16.9 run in this repo reports 'G004 Logging statement uses f-string'.)

D2-L3 explain(), the Spark UI and the query profile

Concept

A slow Spark job often shows why in its **plan**. Read it before you tune anything.

`df.explain()` prints the physical plan without running the query. `df.explain("formatted")` numbers each operator and lists its details below. The other modes are `"simple"` (the default), `"extended"`, `"codegen"` and `"cost"`. Read the tree from the **bottom up**: the scan is at the bottom, the result at the top.

What to look for (examples from PySpark 4.0.4):

In the plan	Means
`PartitionFilters: [(wave_id = 3)]`	Only matching partitions are read. Good.
`PushedFilters: [GreaterThan(w, 1.0)]`	The filter runs inside the file reader. Good.
`PushedFilters: []` on a filtered query	Nothing was pushed down. Check the filter's column and type.
`Exchange hashpartitioning(...)`	A shuffle: data moves across the cluster. Each one costs time.
`BroadcastHashJoin`	The small side is copied to every executor. No shuffle of the large side.
`SortMergeJoin`	Both sides are shuffled and sorted. Normal for two large tables.
`CartesianProduct` or `BroadcastNestedLoopJoin`	Every row meets every row. Usually a missing join condition.
`BatchEvalPython`	A Python UDF. Rows leave the JVM for a Python worker.

`AdaptiveSparkPlan isFinalPlan=false` means adaptive query execution will adjust the plan at run time, for example switching a join to broadcast. `explain()` shows the starting plan.

After the run, look at what happened. On classic compute, the **Spark UI** shows jobs, stages, tasks and the SQL tab with row counts per operator. On **serverless**, the Spark UI is not available. Use the **query profile**, which shows each operator with its time spent, rows and memory peak, and a Top operators panel.

In the UI or profile, compare **rows out** of each join with rows in. A join whose output exceeds its larger input is duplicating rows (lesson D3-L1). Look for one task far slower than the rest of its stage: that is skew.

Why it matters

A gold job's runtime grew from 4 minutes to 70 over a month. Nobody had changed the code. `explain()` showed `BroadcastNestedLoopJoin`: a join on a range condition, fine while the dimension held 50 rows, painful at 4,000. The plan named the problem in one line. Rewriting the range join as an equi-join on a precomputed band key brought it back to 5 minutes.

Worked example: weak code

```
1 from pyspark.sql import functions as F
2
1 joined = respondents.crossJoin(regions).filter(
4     F.col("postcode_area") == F.col("area_code")
5 )
2 joined.write.mode("overwrite").saveAsTable("research_gold.nat_regions")
```

1. **Line 3.** A cross join first, then a filter. Check the plan before you trust the optimiser to rewrite it.
2. **Line 6.** Runs without ever reading the plan.

Worked example: strong code

```
1 from pyspark.sql import functions as F
2
3 joined = respondents.join(
1     F.broadcast(regions), respondents["postcode_area"] ==
    regions["area_code"], "left"
5 )
2 joined.explain("formatted")
7 # Expect BroadcastHashJoin, no CartesianProduct, one scan of respondents.
8 joined.write.mode("overwrite").saveAsTable("research_gold.nat_regions")
```

1. **Line 4.** An explicit equi-join with the small side broadcast.

2. **Line 6.** Read the plan once, when you write the code, and again when data volumes change.

Read the plan bottom up, look for pushed filters, shuffles, join types and Python UDFs, then confirm rows per operator in the Spark UI or query profile.

Recap card

- **Read a physical plan in which direction?** Bottom up: scan at the bottom, result at the top.
- **PartitionFilters and PushedFilters** Filters applied while reading. Empty PushedFilters on a filtered query means no pushdown.
- **Plan operators that signal trouble** CartesianProduct or BroadcastNestedLoopJoin (cross join), BatchEvalPython (Python UDF), many Exchange (shuffles).
- **Spark UI on serverless** Not available. Use the query profile: time spent, rows and memory per operator.

Practice

Exercise 1. Multiple choice (level 1)

A job's plan contains `CartesianProduct`. What is the most likely cause?

- a) A join without an equality condition, so every row meets every row
- b) A broadcast join
- c) A Python UDF
- d) Partition pruning

Why?

- r1) A Cartesian product pairs every row with every row. It comes from a cross join or a join condition Spark cannot use as an equi-join.
- r2) Spark uses Cartesian products for all small tables.

Exercise 2. Spot the bug (level 2)

This is the formatted plan for `spark.read.parquet(path).filter(F.col("w") > 1.0).groupBy("wave_id").agg(F.sum("w"))`. Tap the line that shows a shuffle.

```
1 AdaptiveSparkPlan (6)
2 +- HashAggregate (5)
3    +- Exchange (4)
4       +- HashAggregate (3)
5          +- Filter (2)
6             +- Scan parquet  (1)
```

Write down the line number of the bug.

Exercise 3. Multiple choice (level 2)

Your job runs on **serverless** compute. Where do you see time spent and rows per operator after it runs?

- a) The query profile
- b) The Spark UI's SQL tab
- c) df.explain('cost')
- d) The driver logs

Why?

- r1) The Spark UI is not available on serverless. Databricks points you to the query profile.
- r2) The query profile is the only tool that works in Databricks.

Exercise 4. Write the function (level 3)

Write `plan_warnings(plan: str, max_exchanges: int = 2) -> list[str]` that reads the text of `df.explain()` and returns warnings, in this order, each at most once: `"cross join"` if the plan contains `CartesianProduct` or `BroadcastNestedLoopJoin`, `"python udf"` if it contains `BatchEvalPython` or `ArrowEvalPython`, and `"many shuffles"` if the word `Exchange` appears more than `max_exchanges` times, not counting `BroadcastExchange`.

What you need to know.

`re.findall(pattern, text)`, or `compiled.findall(text)`, returns every match as a list, so `len(...)` counts them. Two pattern tools control what counts:

- `\b` marks a word boundary: `\bcat\b` matches `cat` as a whole word only.
- `(?<!con)` is a **negative lookbehind**: the match counts only when `con` does not come right before it.

```
import re

text = "cat concat bobcat"
re.findall(r"cat", text)           # ['cat', 'cat', 'cat']
re.findall(r"(?<!con)cat", text)   # ['cat', 'cat']: not the one in concat
re.findall(r"\bcat\b", text)       # ['cat']
```

Your code must pass these checks:

- BroadcastNestedLoopJoin is a cross join too, and BatchEvalPython is a Python UDF. Keep the order.
- BroadcastExchange is not a shuffle. Count only plain Exchange operators.
- Three shuffles is more than the default of 2.
- No warnings for a normal plan.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
CROSS_JOIN_OPERATORS = ...

# Module constant.
PYTHON_UDF_OPERATORS = ...

# Module constant.
SHUFFLE_PATTERN = ...

# Warnings for common problems in a Spark physical plan.
def plan_warnings(
    plan: str,
    max_exchanges: int = 2,
) -> list[str]: ...
```

Starter code:

```
def plan_warnings(plan, max_exchanges=2):
    warnings = []
    if "Cartesian" in plan:
        warnings.append("cross join")
    return warnings
```

Exercise 5. Architecture design (level 4)

A gold job on classic compute slowed from 5 to 40 minutes. Plan the investigation.

Options: Run explain('formatted') and read the joins and exchanges, Double the cluster size.

1. First step
2. Checking what happened at run time
3. One task in a stage takes 30 minutes, the rest 20 seconds

Sources

- explain modes simple, extended, codegen, cost, formatted; plans show PartitionFilters, PushedFilters, Exchange hashpartitioning, BroadcastHashJoin, SortMergeJoin, CartesianProduct, BatchEvalPython, AdaptiveSparkPlan isFinalPlan=false. (verified: Local runs on PySpark 4.0.4, September 2026.)
- The Spark UI is not available on serverless; use the query profile, which shows time spent, rows and memory per operator and a Top operators panel. (verified: https://docs.databricks.com/aws/en/compute/serverless/

limitations and https://docs.databricks.com/aws/en/sql/user/queries/query-profile (checked 2026-09-25))

D3 Silent errors

Roadmap: Every month. Join explosion, nulls in filters, type coercion, timezone drift, mutable defaults.

D3-L1 Join row explosion and nulls in filters

Concept

Some bugs raise no error. The job succeeds and the numbers are wrong. Two of the most common come from joins and from nulls.

Join row explosion. A left join of respondents to a region lookup should keep one row per respondent. If the lookup holds a key twice, say `N` for "North" and "North (old)", every respondent in `N` appears twice. Weighted totals double for that region. Nothing fails.

Check the grain around every join:

- Count rows before and after. A many-to-one join must not change the count.
- In pandas, pass `validate="many_to_one"`. A duplicate key then raises `MergeError: Merge keys are not unique in right dataset`. Checked on pandas 3.0.2.
- In Spark, count duplicate keys in the lookup before joining: `dim.groupBy(key).count().filter("count > 1")`.

Nulls behave differently in pandas and Spark. Checked on pandas 3.0.2 and PySpark 4.0.4:

Operation	pandas	Spark and SQL
Filter `region != "N"`	Keeps rows where region is missing	**Drops** rows where region is null
Join on a null key	**Matches** null to null	Never matches a null key
`x NOT IN (subquery)` where the subquery holds a null	Not applicable	Returns **no rows**

In Spark and SQL, a comparison with null gives null, not True or False, and a filter keeps only True. So `F.col("region_code") != "N"` silently drops every respondent with no region. Say what you mean:

- Keep nulls: `(F.col("region_code") != "N") | F.col("region_code").isNull()`, or `~F.col("region_code").eqNullSafe("N")`.
- Exclude a list that might contain nulls: use `NOT EXISTS` or a `left_anti` join, not `NOT IN`.

In pandas, the surprise runs the other way: null keys match each other. Drop or fill null keys before a merge unless you want that.

Why it matters

A regional reach table showed the North at 118% weighted reach. The region lookup had gained a second row for code `N` when a boundary changed. The left join doubled every northern respondent. A row count check before and after the join, or `validate="many_to_one"`, would have stopped the job at the join.

Worked example: weak code

```
1  def add_region(respondents: pd.DataFrame, regions: pd.DataFrame) ->
     pd.DataFrame:
2      """Attach region names."""
1      merged = respondents.merge(regions, on="region_code", how="left")
2      return merged[merged["region"] != "Unknown"]
```

1. **Line 3.** No grain check. A duplicate code doubles rows, and null codes match a null row in regions.
2. **Line 4.** A filter that also changes the row count, mixed into the join step.

Worked example: strong code

```
1  def add_region(respondents: pd.DataFrame, regions: pd.DataFrame) ->
     pd.DataFrame:
2      """Attach region names. One output row per respondent.
3
4      Raises:
5          pandas.errors.MergeError: If a region code appears twice in
     regions.
6      """
1      lookup = regions.dropna(subset=["region_code"])
2      merged = respondents.merge(lookup, on="region_code", how="left",
     validate="many_to_one")
3      if len(merged) != len(respondents):
10         raise ValueError(f"join changed rows: {len(respondents)} ->
     {len(merged)}")
```

```
11      return merged
```

1. **Line 7.** pandas matches null keys to each other. Drop them from the lookup.
2. **Line 8.** pandas raises MergeError if the lookup has a duplicate key.
3. **Line 9.** An explicit grain check. It also works where validate is not available.

Check row counts across every join. Know how your engine treats nulls in filters and keys, and write the null case explicitly.

Recap card

- **Check for join row explosion** Row counts before and after. pandas: validate='many_to_one'. Spark: count duplicate keys in the lookup first.
- **col != 'x' with nulls: pandas vs Spark** pandas keeps missing values. Spark and SQL drop nulls, because the comparison is null.
- **Null join keys: pandas vs Spark** pandas merge matches null to null. Spark and SQL never match null keys.
- **NOT IN with a null in the list** Returns no rows. Use NOT EXISTS or a left_anti join.

Practice

Exercise 1. Predict the output (level 1)

What does this print? (pandas 3)

```
1 import pandas as pd
2
3 resp = pd.DataFrame({"id": [1, 2, 3], "code": ["N", "S", "N"]})
4 regions = pd.DataFrame({"code": ["N", "N", "S"], "name": ["North", "North
    (old)", "South"]})
5 print(len(resp.merge(regions, on="code", how="left")))
```

Exercise 2. Multiple choice (level 2)

In PySpark, `respondents.filter(F.col("region_code") != "N")` runs on 1,000 rows: 300 are `N`, 50 are null. How many rows remain?

- a) 650
- b) 700
- c) 350
- d) 1,000

Why?

- r1) null != 'N' evaluates to null, and a filter keeps only rows where the condition is true.
- r2) Spark drops nulls from every DataFrame before filtering.

Exercise 3. Multiple choice (level 2)

`SELECT * FROM respondents WHERE region_code NOT IN (SELECT code FROM excluded_regions)` returns no rows. `excluded_regions` holds `'N'` and one null. Why?

- a) x NOT IN (..., NULL) is never true, because x <> NULL is null
- b) NOT IN is not supported in Spark SQL
- c) The subquery returns too many rows
- d) Every respondent is in region N

Why?

- r1) NOT IN expands to x <> 'N' AND x <> NULL. The second term is null, so the whole condition is never true.
- r2) Subqueries ignore nulls.

Exercise 4. Write the function (level 3)

Write `safe_left_join(facts: pd.DataFrame, lookup: pd.DataFrame, key: str) -> pd.DataFrame`. Drop rows with a null key from `lookup` first. If a key appears more than once in the remaining lookup, raise `ValueError` naming the duplicated keys in sorted order. Return the left join, which must have exactly `len(facts)` rows. Null keys in `facts` must get missing lookup values, not match anything.

Your code must pass these checks:

- Raise ValueError listing duplicated keys in sorted order, like ['N', 'W'].
- pandas matches null keys to each other. Drop null keys from the lookup so a null fact key gets no name.
- Return one row per fact row, in the original order.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Left join that keeps the grain of facts.
def safe_left_join(
    facts: pd.DataFrame,
    lookup: pd.DataFrame,
    key: str,
) -> pd.DataFrame: ...
```

Starter code:

```
import pandas as pd


```

```
def safe_left_join(facts, lookup, key):
    return facts.merge(lookup, on=key, how="left")
```

Exercise 5. Spot the bug (level 3)

This step should remove test respondents (`panel == "test"`) and keep everyone else, including respondents with no panel recorded. 4% of real respondents vanish. Tap the line, then fix it.

```
from pyspark.sql import functions as F

live = respondents.filter(F.col("panel") != "test")
live.write.mode("overwrite").saveAsTable("research_silver.nat_live")
```

Write down the line number of the bug. Then write the fix.

Sources

- pandas 3.0.2: validate='many_to_one' raises MergeError 'Merge keys are not unique in right dataset; not a many-to-one merge'; != keeps missing values; merge matches null keys. (verified: Local run on pandas 3.0.2 (Pyodide pin), September 2026.)
- PySpark 4.0.4: != filter drops nulls; eqNullSafe keeps them when negated; null join keys do not match; NOT IN with a null in the subquery returns no rows; NOT EXISTS and left_anti do not. (verified: Local run on PySpark 4.0.4, September 2026.)

D3-L2 Type coercion and timezone drift

Concept

Two more silent errors: values that change type, and times that change meaning with the zone.

Type coercion. Checked on pandas 3.0.2, PySpark 4.0.4 and Python 3.13:

- `pd.read_csv` turns `respondent_id` `0123` into the integer `123`. The leading zero is gone, and joins to the panel file fail. Pass `dtype={"respondent_id": "string"}` for identifiers.
- An integer column with one missing value becomes `float64` in pandas. `1` becomes `1.0`, and string keys built from it read `"1.0"`.
- Strings sort as text: `sorted(["10", "9", "2"])` gives `['10', '2', '9']`. Wave "W10" sorts before "W2". Pad your ids, or sort on a number.

- Spark's `df.union(other)` matches columns **by position**, not by name. A delivery with `wave_id` and `respondent_id` swapped unions without error, with the values in the wrong columns. Use `unionByName`.
- Under ANSI mode, Spark casts a string column compared with a number: `F.col("age") > 9` raises `CAST_INVALID_INPUT` on the value `'x'`. Cast explicitly with `try_cast` first.

Timezone drift. A timestamp without a zone is a guess.

- `datetime.utcnow()` returns a **naive** datetime and is deprecated from Python 3.12. Comparing naive with aware raises `TypeError: can't compare offset-naive and offset-aware datetimes`. Use `datetime.now(timezone.utc)`.
- Spark's `TIMESTAMP` type is an instant, displayed in the **session time zone** (`spark.sql.session.timeZone`). `F.to_date` on `2026-06-30T23:30:00Z` gives 30 June in UTC and **1 July** in Europe/London. A wave cut-off at midnight moves respondents between waves.
- The same text without an offset, `'2026-06-30 23:30:00'`, parses to instants 4 hours apart in UTC and America/New_York sessions.
- Some local times do not exist. pandas refuses to localise `2026-03-29 01:30` to Europe/London: the clocks skipped that hour.

Store instants in UTC. Convert to local time only for display, or for a business rule that names its zone, such as "fieldwork closes at midnight UK time".

Why it matters

A wave closed at midnight UK time on 30 June. The pipeline ran with the session time zone set to UTC and assigned every interview completed between 23:00 and 00:00 UTC on 30 June to wave W14. In British Summer Time those interviews happened after midnight, in W15. 340 respondents moved wave. Both waves' reach figures shifted by up to 1.5 points.

Worked example: weak code

```
from datetime import datetime

def in_wave(completed_at: datetime, close_at: datetime) -> bool:
    """Was the interview completed before the wave closed?"""
    return completed_at < close_at

close = datetime(2026, 6, 30, 23, 59, 59)
in_wave(datetime.utcnow(), close)
```

1. **Line 7.** Naive: is this UK time or UTC? The code cannot tell, and nor can a reader.

2. **Line 8.** Deprecated since Python 3.12, and naive too.

Worked example: strong code

```
1  from datetime import datetime, timedelta, timezone
2
1  BST = timezone(timedelta(hours=1), "BST")
4
5  def in_wave(completed_at: datetime, close_at: datetime) -> bool:
6      """Was the interview completed before the wave closed?
7
8      Raises:
9          ValueError: If either datetime is naive.
10     """
2      if completed_at.tzinfo is None or close_at.tzinfo is None:
12         raise ValueError("datetimes must carry a time zone")
13     return completed_at < close_at
14
15 close = datetime(2026, 7, 1, 0, 0, tzinfo=BST)
3  in_wave(datetime.now(timezone.utc), close)
```

1. **Line 3.** The business rule names its zone. In production, use zoneinfo.ZoneInfo("Europe/London") to handle the clock change.
2. **Line 11.** Refuse naive values instead of guessing.
3. **Line 16.** Aware UTC now. Aware datetimes compare correctly across zones.

Read identifiers as strings, union by name, and keep every timestamp aware and stored in UTC.

Recap card

- **pd.read_csv and leading zeros** Default parsing makes 0123 into 123. Pass dtype='string' for identifiers.
- **Spark union vs unionByName** union matches columns by position, silently. unionByName matches by name.
- **datetime.utcnow()** Deprecated from Python 3.12 and naive. Use datetime.now(timezone.utc).
- **Spark TIMESTAMP and the session time zone** An instant, shown and converted in spark.sql.session.timeZone. to_date can move a row to another day.

Practice

Exercise 1. Predict the output (level 1)

What does this print?

```
1  waves = ["W10", "W9", "W2"]
2  print(sorted(waves))
```

Exercise 2. Multiple choice (level 2)

A Spark session has `spark.sql.session.timeZone` set to `Europe/London`. An interview's `completed_at` is the instant `2026-06-30T23:30:00Z`. What does `F.to_date("completed_at")` return?

- a) 2026-07-01
- b) 2026-06-30
- c) It depends on the cluster's operating system clock
- d) NULL

Why?

- r1) TIMESTAMP is an instant. to_date uses the session time zone, where 23:30 UTC is 00:30 on 1 July in BST.
- r2) Spark always converts to local time before storing.

Exercise 3. Fill the gap (level 2)

Replace the deprecated call with an aware UTC timestamp.

```
from datetime import datetime, timezone

run_timestamp = datetime._____(1)(_____(2))
```

Exercise 4. Write the function (level 3)

Write `load_delivery(csv_text: str) -> pd.DataFrame`. Read the CSV with `respondent_id` and `postcode_area` as strings, keeping leading zeros. Convert `final_weight` to float. If any non-empty weight fails to parse, raise `ValueError` naming how many. Empty weights stay missing.

What you need to know.

`pd.to_numeric(series, errors="coerce")` converts text to numbers. A value it cannot parse becomes missing instead of raising an error:

```
s = pd.Series(["1.5", "abc", None], dtype="string")
pd.to_numeric(s, errors="coerce")   # 1.5, <NA>, <NA>
```

A value that was present before and missing after failed to parse. Compare `s.notna()` with the parsed result's `.isna()` to find those rows.

Your code must pass these checks:

- Keep identifiers as strings so leading zeros survive.
- Weights are floats. An empty weight stays missing.
- Two weights failed to parse. Raise ValueError with the count, instead of turning them into missing values.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
ID_COLUMNS = ...

# Read a delivery CSV with identifiers as strings and weights as floats.
def load_delivery(csv_text: str) -> pd.DataFrame: ...
```

Starter code:

```
import io

import pandas as pd


def load_delivery(csv_text):
    return pd.read_csv(io.StringIO(csv_text))
```

Exercise 5. Spot the bug (level 3)

A redelivery arrives with its columns in a different order. After this step, some respondent ids read `W14`. Tap the line, then fix it.

```
from pyspark.sql import functions as F

combined = original.union(redelivery)
combined = combined.withColumn("loaded_at", F.current_timestamp())
```

Write down the line number of the bug. Then write the fix.

Sources

- pandas 3.0.2: read_csv drops leading zeros by default; int with None becomes float64; tz_localize rejects a nonexistent DST time. (verified: Local run on pandas 3.0.2, September 2026.)
- Python 3.13: utcnow() raises a DeprecationWarning and is naive; comparing naive and aware datetimes raises TypeError. (verified: Local run on CPython 3.13.12, September 2026.)
- PySpark 4.0.4: union matches by position; to_date depends on session time zone; naive strings parse relative to the session zone; ANSI casts a string compared with an int and raises CAST_INVALID_INPUT on 'x'. (verified: Local run on PySpark 4.0.4, September 2026.)

D3-L3 Mutable defaults and the truthiness of zero

Concept

Two Python habits produce wrong results with no error message. Both appear in code written by people and by AI assistants, including your reference pipeline.

Mutable default arguments. Python evaluates a default value once, when it defines the function. Every call that relies on the default shares that one object. Write `def add_check(result, results=[])`, and each call appends to the same list. The second validation run returns the first run's checks as well.

The fix is a `None` sentinel:

```
def add_check(result: str, results: list[str] | None = None) -> list[str]:
    results = [] if results is None else results
    results.append(result)
    return results
```

ruff flags the pattern as rule B006. Treat that warning as a bug.

Truthiness of numbers. `if value:` is `False` for `None`, and also for `0`, `0.0`, empty strings and empty containers. When zero is a valid measurement, such as a 0% reach, a zero weight or zero rows, a truthiness test treats it as missing.

Your reference `check_wave_stability` does this:

```
if current_pct and prior_pct:
```

A metric that falls from 12% to 0% skips the comparison. The largest possible drop produces no warning.

Test for the case you mean. For missing, write `is None`. For zero, write `== 0`.

```
if current_pct is not None and prior_pct is not None:
```

The same rule covers type hints. `error_message: str = None` claims the value is always a string. Write `str | None = None`, and mypy then makes you handle the `None` case.

Why it matters

A questionnaire routing error removes one question for a whole wave. Its weighted percentage drops from 12% to 0.0%. The wave-over-wave stability check runs `if current_pct and prior_pct:`, treats 0.0 as missing and skips the metric. QA reports PASS. The dashboard publishes 0% reach for that question, and a client asks why the channel "lost its whole audience" in a week.

The check existed to catch exactly this change. One truthiness test disabled it for the one value that mattered most.

Worked example: weak code

```
2  def compare_waves(current: dict, prior: dict, tolerance: float = 0.10,
     flags=[]):
2      """Flag metrics whose share moved more than tolerance between
     waves."""
3      for metric, current_pct in current.items():
4          prior_pct = prior.get(metric)
3          if current_pct and prior_pct:
6              if abs(current_pct - prior_pct) > tolerance:
7                  flags.append(metric)
8      return flags
```

1. **Line 1.** Bare `dict` hides the value type. The values are floats or None.
2. **Line 1.** The default list is created once. Flags from one call leak into the next.
3. **Line 5.** A share of 0.0 is falsy, so the biggest possible drop is never compared.

Worked example: strong code

```
1  def compare_waves(
1      current: dict[str, float | None],
3      prior: dict[str, float | None],
4      tolerance: float = 0.10,
5  ) -> list[str]:
6      """Flag metrics whose share moved more than tolerance between waves.
7
8      A metric is skipped only when a share is missing. A share of 0.0 is
     compared.
9      """
2      flags: list[str] = []
11     for metric, current_pct in current.items():
12         prior_pct = prior.get(metric)
3          if current_pct is None or prior_pct is None:
14             continue
15         if abs(current_pct - prior_pct) > tolerance:
16             flags.append(metric)
17     return flags
```

1. **Line 2.** The hint says a share can be missing, so mypy checks every use.
2. **Line 10.** A new list on every call. No state leaks between runs.
3. **Line 13.** Skips only missing values. 0.0 goes on to the comparison.

Both fixes are one line. Both bugs pass every test that uses only non-zero values and a single call.

Recap card

- **When does Python evaluate a default argument?** Once, when it defines the function. A list or dict default is shared by every call.
- **Safe pattern for an optional list argument** Default to None. Inside: results = [] if results is None else results.
- **What is wrong with `if current_pct and prior_pct:`?** 0.0 is falsy, so a real share of zero is treated as missing. Use `is not None`.
- **Type hint for an optional string that defaults to None** str | None = None. Never str = None.

Practice

Exercise 1. Predict the output (level 1)

What does this print?

```
1 def add_flag(flag, flags=[]):
2     flags.append(flag)
3     return flags
4
5 add_flag("null_weights")
6 print(add_flag("row_count"))
```

Exercise 2. Multiple choice (level 2)

`current_pct = 0.0` and `prior_pct = 0.12`. The tolerance is 0.10. What does `check_wave_stability` from the reference code do with this metric?

- a) Flags it as WARN, because the change is 0.12.
- b) Marks it PASS.
- c) Leaves it out of the results.
- d) Raises a ZeroDivisionError.

Why?

- r1) `if current_pct and prior_pct:` is False when current_pct is 0.0, so the block never runs.
- r2) The function filters out metrics below 1%.

Exercise 3. Fill the gap (level 3)

Fix both signatures. The error message is optional, and the flags list must not be shared between calls.

```
1 def log_run(status: str, error_message: _____(1) = None) -> None: ...
2
```

```
3 def collect(flag: str, flags: list[str] | None = _____(2)) ->
    list[str]: ...
```

Exercise 4. Spot the bug (level 3)

This check should warn when a share moves by more than the tolerance. A drop from 12% to 0% produces no warning. Tap the line with the bug, then fix the function.

```
1 def stability_status(current_pct: float | None, prior_pct: float | None,
    tolerance: float = 0.10) -> str:
2     """Return WARN, PASS, or SKIP when a share is missing."""
3     if current_pct and prior_pct:
4         change = abs(current_pct - prior_pct)
5         return "WARN" if change > tolerance else "PASS"
6     return "SKIP"
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Write the function (level 4)

Write `compare_waves(current, prior, tolerance=0.10)`. Both arguments map metric names to a share or `None`. Return the metrics whose share changed by more than `tolerance`, in the order of `current`. Skip a metric only when either share is `None` or the metric is missing from `prior`. Calling the function twice must not carry results over.

Your code must pass these checks:

- A drop from 0.12 to 0.0 must be flagged. Test for None, not for truth.
- A None share must be skipped, and an unchanged share must not be flagged.
- A metric missing from prior must be skipped, not raise KeyError. Use `prior.get(metric)`.
- The second call returned flags from the first call. Create a new list inside the function.
- Return flags in the order of `current`, without sorting.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
1 # Flag metrics whose share moved more than tolerance between waves.
2 def compare_waves(
3     current: dict[str, float | None],
4     prior: dict[str, float | None],
5     tolerance: float = 0.1,
6 ) -> list[str]: ...
```

Starter code:

```python
def compare_waves(
    current: dict[str, float | None],
    prior: dict[str, float | None],
    tolerance: float = 0.10,
) -> list[str]:
    ...
```

Exercise 6. Refactor (level 5)

A validator collects check results with the helper below. The second contract validation in a job reports the first contract's failures too. Refactor `add_result` so each validation starts empty, keep the signature compatible for callers that pass a list, and add type hints and a docstring.

Your code must pass these checks:

- Two calls without a list must not share results.
- When the caller passes a list, append to that list and return it.
- An empty list from the caller was replaced. `results or []` treats an empty list as missing. Test `results is None`.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Append one check result and return the list.
def add_result(
    name: str,
    passed: bool,
    results: list[dict[str, object]] | None = None,
) -> list[dict[str, object]]: ...
```

Starter code:

```python
def add_result(name, passed, results=[]):
    results.append({"check": name, "passed": passed})
    return results
```

Sources

- ruff rule B006 flags mutable default arguments. (verified: ruff 0.16.9 run in this repo reports B006 on `def f(x=[])`.)

D4 Data errors

Roadmap: Every month. Row counts across layers, distribution drift and weight totals.

D4-L1 Row counts across layers

Concept

Every layer of a medallion pipeline changes the row count for a reason. Write the reasons down as rules, then check them on every run. A count that breaks a rule is a bug, even when no step failed.

For one tracker and one wave:

Check	Rule
Bronze vs delivery	Bronze rows for the delivery = rows in the delivered file or manifest
Silver vs bronze	Silver rows = distinct respondent keys in bronze. Never more than bronze rows
Gold vs silver	Sum of gold `unweighted_n` = silver rows in scope
Redelivery	After MERGE, silver rows = rows before + rows inserted

Where the numbers come from. Count each layer with one `agg` pass, not repeated `count()` calls on an uncached DataFrame: `df.agg(F.count("*"), F.count_distinct("respondent_id"))`. For writes, Delta records metrics in the table history. `DESCRIBE HISTORY` shows `numOutputRows` for a write, and for a MERGE `numSourceRows`, `numTargetRowsInserted` and `numTargetRowsUpdated`. Checked on delta-spark 4.0.1.

Filter both sides the same way. Your reference `get_unprocessed_waves` compares bronze waves **for one tracker** with silver waves **for all trackers**. When another tracker already has wave W14 in silver, this tracker's W14 is never loaded. Every reconciliation query filters bronze, silver and gold on the same tracker and wave.

Store the result. Write each reconciliation as a structured row in the run log: layer, expected, actual, status. A failed CRITICAL rule halts the pipeline before gold is published.

Counts catch lost and duplicated rows. They do not catch wrong values in the right number of rows. Lessons D4-L2 and D4-L3 cover those.

Why it matters

Wave W14 for one tracker never reached silver. The job logged success every night for three weeks, because `get_unprocessed_waves` found W14 in silver, loaded by a different tracker. A rule "silver rows for tracker and wave > 0 when bronze rows > 0" would have failed on the first night.

Worked example: weak code

```
1 bronze_n = bronze.count()
2 silver_n = silver.count()
2 if silver_n > 0:
4     print("silver loaded")
```

1. **Line 1.** All trackers and all waves. The number means nothing for this load.
2. **Line 3.** Any row from any tracker passes this check.

Worked example: strong code

```
1  from pyspark.sql import functions as F
2
3  def layer_counts(df, tracker_id: str, wave_id: str) -> dict[str, int]:
4      """Rows and distinct respondents for one tracker and wave, in one
       pass."""
5      row = (
1          df.filter((F.col("tracker_id") == tracker_id) & (F.col("wave_id")
       == wave_id))
2          .agg(F.count("*").alias("rows"),
       F.count_distinct("respondent_id").alias("keys"))
8          .first()
9      )
10     return {"rows": row["rows"], "keys": row["keys"]}
11
12 bronze_c = layer_counts(bronze, "NAT", "W14")
13 silver_c = layer_counts(silver, "NAT", "W14")
3  if silver_c["rows"] != bronze_c["keys"]:
15     raise RuntimeError(f"silver {silver_c['rows']} != bronze keys
       {bronze_c['keys']}")
```

1. **Line 6.** The same tracker and wave filter on every layer.
2. **Line 7.** Both counts in one pass over the data.
3. **Line 14.** A written rule, checked every run. It halts before gold.

Write the expected relationship between layers as rules, compute counts in one pass on identical filters, and halt when a CRITICAL rule fails.

Recap card

- **Row-count rules: silver vs bronze** Silver rows = distinct respondent keys in bronze for the tracker and wave.

- **Rows after a MERGE** Rows before + numTargetRowsInserted (minus any deletes). Updates add nothing.
- **Counting efficiently** One agg pass: F.count('*') and F.count_distinct(key). Not repeated count() calls.
- **The get_unprocessed_waves bug** Bronze filtered on tracker, silver not. Filter both sides on the same keys.

Practice

Exercise 1. Multiple choice (level 1)

Bronze holds 10,400 rows for tracker NAT, wave W14, with 10,000 distinct respondents. Silver deduplicates on respondent and wave. How many silver rows do you expect?

- a) 10,000
- b) 10,400
- c) Any number up to 10,400
- d) It depends on the weights

Why?

- r1) Silver's grain is one row per respondent per wave, so it holds one row per distinct key.
- r2) Silver always drops 4% of rows.

Exercise 2. Multiple choice (level 2)

A redelivery MERGE into silver reports `numSourceRows = 500`, `numTargetRowsInserted = 120`, `numTargetRowsUpdated = 380`. Silver had 10,000 rows before. How many rows does it hold now?

- a) 10,120
- b) 10,500
- c) 10,380
- d) 10,000

Why?

- r1) Updates replace existing rows. Only inserts add rows.
- r2) MERGE never removes rows, so every source row is added.

Exercise 3. Fill the gap (level 2)

Compute rows and distinct respondents in one pass.

```
1 row = df.agg(
```

```
    F.count("*").alias("rows"),
    F._____(1)("respondent_id").alias("keys"),
).____(2)()
```

Exercise 4. Write the function (level 3)

Write `reconcile(delivered: int, bronze_rows: int, bronze_keys: int, silver_rows: int, gold_n: int, in_scope: int) -> list[dict[str, object]]`. Return one record per rule, in this order, each `{"rule": name, "expected": int, "actual": int, "status": "PASS" or "FAIL"}`: `"bronze_vs_delivery"` (bronze_rows equals delivered), `"silver_vs_bronze_keys"` (silver_rows equals bronze_keys), `"gold_vs_silver_scope"` (gold_n equals in_scope). Raise `ValueError` if any count is negative, or if `in_scope` exceeds `silver_rows`.

Your code must pass these checks:

- Three rules in order, all PASS for consistent counts.
- Silver must equal distinct bronze keys, not bronze rows. Record expected and actual.
- Gold unweighted_n must add up to the in-scope silver rows.
- Negative counts, or more in scope than silver rows, are impossible inputs. Raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Row-count rules across layers for one tracker and wave.
def reconcile(
    delivered: int,
    bronze_rows: int,
    bronze_keys: int,
    silver_rows: int,
    gold_n: int,
    in_scope: int,
) -> list[dict[str, object]]: ...
```

Starter code:

```python
def reconcile(delivered, bronze_rows, bronze_keys, silver_rows, gold_n,
    in_scope):
    return [{"rule": "all", "status": "PASS" if silver_rows > 0 else
    "FAIL"}]
```

Exercise 5. Spot the bug (level 4)

This is your reference `get_unprocessed_waves`, cut down. Tracker NAT's wave W14 is never loaded when another tracker already has W14 in silver. Tap the line with the bug, then fix it.

```
bronze_waves = (
```

```
    spark.table(bronze_table).filter(F.col("tracker_id") == tracker_id)
    .select("wave_id").distinct()
)
silver_waves = spark.table(silver_table).select("wave_id").distinct()
new_waves = [r.wave_id for r in
    bronze_waves.subtract(silver_waves).collect()]
```

Write down the line number of the bug. Then write the fix.

Sources

- Delta MERGE history records numSourceRows, numTargetRowsInserted, numTargetRowsUpdated; writes record numOutputRows. (verified: Local run of DESCRIBE HISTORY on delta-spark 4.0.1 with PySpark 4.0.4, September 2026.)

D4-L2 Distribution drift between waves

Concept

Row counts can be perfect while the data inside has changed. A routing error drops one question. A supplier recodes region. A weighting run uses last year's targets. Each shows up as a **shift in distributions** between waves.

Three checks, from simple to sensitive:

- **Category shares.** For each categorical variable, compare each category's weighted share with the prior wave. Flag a move larger than a tolerance, say 5 percentage points. This is what your `check_wave_stability` does, once its truthiness bug is fixed (lesson D3-L3).
- **New and missing categories.** A category present in one wave and absent in the other is a recode until proven otherwise. `region = "Northern"` appearing beside `"North"` is a supplier change, not a population change.
- **Population stability index (PSI)** summarises a whole distribution in one number:

```
PSI = sum over categories of (actual - expected) x ln(actual / expected)
```

where `expected` and `actual` are shares in the prior and current wave. PSI is 0 when nothing moved. A category with a share of zero makes the logarithm undefined, so replace zero shares with a small floor, such as 0.0001, before computing. A common rule of thumb reads PSI below 0.1 as stable and above 0.25 as a large shift. Those cut-offs are convention, not a standard: calibrate them on your own history of waves.

Guard the divisions. Every share divides by a total. An empty wave, or a variable with no valid answers, gives a total of zero. Your reference `_check_distributions` divides by `total` with no guard and fails on an empty delivery. Check the total first and report "no data" as its own status.

Drift is a question, not a verdict. Real change happens. The check makes a person look before the numbers are published.

Why it matters

A supplier renamed region code `"North"` to `"Northern"` in wave W21. Silver loaded both values, the region lookup matched only `"North"`, and the northern subgroup fell from 1,900 respondents to 12. Every row count passed. A new-and-missing category check on `region` would have flagged `"Northern"` on the first run.

Worked example: weak code

```
def share_changes(prior: dict[str, int], current: dict[str, int]) ->
    dict[str, float]:
    """Change in share per category."""
    p_total = sum(prior.values())
    c_total = sum(current.values())
    return {k: current[k] / c_total - prior[k] / p_total for k in prior}
```

1. **Line 4.** An empty wave makes this 0, and the division fails.
2. **Line 5.** Loops over prior only. A new category in current is ignored, and a missing one raises KeyError.

Worked example: strong code

```
def share_changes(prior: dict[str, int], current: dict[str, int]) ->
    dict[str, float]:
    """Change in share per category, over the union of categories.

    Raises:
        ValueError: If either wave has no observations.
    """
    p_total = sum(prior.values())
    c_total = sum(current.values())
    if p_total == 0 or c_total == 0:
        raise ValueError("a wave has no observations")
    categories = sorted(prior.keys() | current.keys())
    return {
        k: current.get(k, 0) / c_total - prior.get(k, 0) / p_total for k
    in categories
    }
```

1. **Line 9.** An empty wave is reported, not divided by.

2. **Line 11.** New and missing categories both appear, with a share of 0 on the side where they are absent.

Compare shares over the union of categories, flag new and missing values, summarise with PSI, and guard every total.

Recap card

- **Three drift checks** Category shares vs prior wave, new and missing categories, and a PSI summary.
- **PSI formula** Sum of (actual - expected) x ln(actual / expected) over shares, with zero shares floored.
- **PSI thresholds 0.1 and 0.25** A rule of thumb, not a standard. Calibrate on your own wave history.
- **Percentage points vs relative change** 30% to 45% is +15 points and +50% relative. Say which a tolerance uses.

Practice

Exercise 1. Multiple choice (level 1)

Wave W20 has region values North, South, East. Wave W21 has Northern, South, East. Row counts match. What is the most likely cause?

- a) A recode by the supplier
- b) The population of the North moved
- c) A weighting error
- d) Normal sampling variation

Why?

- r1) A category that vanishes while a similar new one appears is a label change until proven otherwise.
- r2) Suppliers always rename categories between waves.

Exercise 2. Predict the output (level 2)

What does this print?

```
prior = {"yes": 30, "no": 70}
current = {"yes": 45, "no": 55}
change = current["yes"] / 100 - prior["yes"] / 100
print(round(change * 100))
```

Exercise 3. Fill the gap (level 2)

Complete the PSI term for one category, with zero shares floored.

```python
import math

e = max(expected_share, FLOOR)
a = max(actual_share, FLOOR)
term = (a - e) * math._____(1)(a / _____(2))
```

Exercise 4. Write the function (level 3)

Write `psi(expected: dict[str, int], actual: dict[str, int], floor: float = 0.0001) -> float`. Both map categories to counts. Use the union of categories, a missing category counting 0. Convert counts to shares, replace shares below `floor` with `floor`, and return the sum of `(a - e) * ln(a / e)`. Raise `ValueError` if either total is 0.

Your code must pass these checks:

- Identical shares give PSI 0, whatever the totals.
- Use shares, not counts, and the natural logarithm.
- Include categories from both waves. Floor a zero share at 0.0001 instead of dividing by zero.
- An empty wave has no shares. Raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Population stability index between two categorical distributions.
def psi(
    expected: dict[str, int],
    actual: dict[str, int],
    floor: float = 0.0001,
) -> float: ...
```

Starter code:

```python
import math


def psi(expected, actual, floor=0.0001):
    return sum((actual[k] - expected[k]) * math.log(actual[k] / expected[k]) for k in expected)
```

Exercise 5. Spot the bug (level 3)

This is your reference `_check_distributions` logic, as a plain function. It fails on an empty delivery. Tap the line, then fix it: return `"NO_DATA"` when the total is 0.

```python
def distribution_status(value_count: int, total: int, min_pct: float, max_pct: float) -> str:
    """PASS if the value's share is within bounds, else WARN."""
    pct = value_count / total
```

```
    return "PASS" if min_pct <= pct <= max_pct else "WARN"
```

Write down the line number of the bug. Then write the fix.

Sources

- PSI = sum (a - e) ln(a / e) over category shares; zero shares need a floor. (verified: Definition checked by the exercise tests in this repo; the formula follows from its terms.)
- PSI thresholds of 0.1 and 0.25 are an informal industry convention. (unverified: Widely quoted in credit-scoring practice; no official standard located. Taught as a rule of thumb only.)

D4-L3 Weight totals and survey checks

Concept

Survey weights carry the whole estimate. A wrong weight gives a plausible but wrong percentage in every table. Check them on every wave, before gold.

Totals and range:

- No null weights, and none zero or negative, unless your design allows zero.
- The weight sum matches its target: the population total for grossing weights, or the sample size for weights normalised to mean 1. Your contract's `weight_sum_min` and `weight_sum_max` encode this.
- The ratio of largest to smallest weight stays below an agreed limit. One respondent standing for 40 others dominates their subgroup.

Efficiency. Unequal weights cost precision. Kish's approximation measures it:

```
deff  = n x sum(w^2) / (sum w)^2 = 1 + CV^2
n_eff = (sum w)^2 / sum(w^2) = n / deff
```

Here CV is the population coefficient of variation of the weights. Weights 0.5, 1.0, 1.5 and 3.0 give a deff of 1.39 and an effective sample of 2.88 out of 4. Track n_eff per subgroup and wave. A sudden fall means the weighting struggled, often because a quota cell was nearly empty.

Spark details that break these checks. Checked on PySpark 4.0.4:

- `F.stddev` is the **sample** standard deviation. It returns null for a single row, and the Kish identity uses the population version, `F.stddev_pop`.

- On an empty DataFrame, `mean` and `stddev` return null. Your reference `check_weight_distribution` then calls `round(None, 4)` and raises `TypeError`.
- `F.sum(F.when(F.col("reach_flag") == 1, 1))` is **null**, not 0, for a subgroup where nobody was reached. Your gold table then shows a blank instead of 0% reach. Wrap it in `F.coalesce(..., F.lit(0))`.

Decide what each result means for the run. A null weight is CRITICAL: halt. A deff above your usual range is WARN: log it and have someone look.

Why it matters

A weighting run used last year's population targets for one age band. The weight sum stayed within tolerance overall, so the contract check passed. The 55+ band's effective sample fell from 800 to 190, and its reach estimate jumped 9 points. A per-subgroup n_eff check with a WARN threshold would have flagged it before the dashboard refreshed.

Worked example: weak code

```
❶ stats = df.agg(F.mean(w).alias("mean"), F.stddev(w).alias("sd")).first()
  2 cv = stats["sd"] / stats["mean"] if stats["mean"] else None
❷ result = {"mean": round(stats["mean"], 4), "cv": round(cv, 4) if cv else
      None}
```

1. **Line 1.** Sample standard deviation, and null for a single row.
2. **Line 3.** round(None, 4) raises TypeError on an empty wave. `if cv` treats a CV of 0 as missing.

Worked example: strong code

```
  1 row = df.agg(
  2     F.count(w).alias("n"),
  3     F.sum(w).alias("total"),
❶     F.sum(F.col(w) * F.col(w)).alias("total_sq"),
  5 ).first()
❷ if row["n"] == 0 or row["total"] is None or row["total"] == 0:
  7     raise ValueError("no weights to check")
  8 deff = row["n"] * row["total_sq"] / row["total"] ** 2
  9 n_eff = row["total"] ** 2 / row["total_sq"]
```

1. **Line 4.** Kish needs only n, sum and sum of squares, in one pass.
2. **Line 6.** Empty and zero totals are reported, never divided by.

Check nulls, range, sum and efficiency per wave and subgroup. Mind Spark's nulls: sample stddev, empty aggregates and sums over no matches.

Recap card

- **Kish design effect and effective sample** deff = n x sum(w^2) / (sum w)^2 = 1 + CV^2. n_eff = (sum w)^2 / sum(w^2).
- **F.stddev in Spark** Sample standard deviation, null for one row. Use F.stddev_pop for 1 + CV^2.
- **sum(when(cond, 1)) with no matches** Null, not 0. Wrap in coalesce(..., lit(0)) or add otherwise(0).
- **Severity for a null weight** CRITICAL: halt before gold.

Practice

Exercise 1. Multiple choice (level 1)

In gold, `F.sum(F.when(F.col("reach_flag") == 1, 1))` gives what for a subgroup where no one was reached?

- a) NULL
- b) 0
- c) An error
- d) The subgroup is dropped

Why?

- r1) when without otherwise gives null for non-matching rows, and the sum of only nulls is null.
- r2) Spark sums return null for any group with a zero.

Exercise 2. Predict the output (level 2)

What does this print?

```
w = [1.0, 1.0, 1.0, 1.0]
n = len(w)
deff = n * sum(x * x for x in w) / sum(w) ** 2
print(deff)
```

Exercise 3. Fill the gap (level 2)

Make the unweighted reach count 0 when no one in the subgroup was reached.

```
F._____(1)(F.sum(F.when(F.col("reach_flag") == 1, 1)),
    F.lit(_____(2))).alias("unweighted_reach")
```

Exercise 4. Write the function (level 3)

Write

`weight_report(weights: list[float | None], max_ratio: float = 20.0) ->`

`dict[str, object]`. Return `{"status": ..., "n": ..., "n_eff": ..., "deff": ...}`. Status is `"CRITICAL"` if the list is empty or any weight is `None` or not positive (and then `n_eff` and `deff` are `None`), `"WARN"` if max/min weight exceeds `max_ratio`, otherwise `"PASS"`. Use `n_eff = (sum w)^2 / sum(w^2)` and `deff = n / n_eff`, each rounded to 4 decimals.

Your code must pass these checks:

- Weights 0.5, 1.0, 1.5, 3.0 give n_eff 2.88 and deff 1.3889.
- Empty lists, None, zero and negative weights are CRITICAL, with no efficiency figures.
- A max/min ratio of 30 exceeds 20: WARN.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Weight checks with Kish's effective sample size and design effect.
def weight_report(
    weights: list[float | None],
    max_ratio: float = 20.0,
) -> dict[str, object]: ...
```

Starter code:

```python
def weight_report(weights, max_ratio=20.0):
    n_eff = sum(weights) ** 2 / sum(w * w for w in weights)
    return {"status": "PASS", "n": len(weights), "n_eff": n_eff, "deff":
    len(weights) / n_eff}
```

Exercise 5. Architecture design (level 4)

Set severities for weight checks in the silver QA step.

Options: CRITICAL: halt before gold, WARN: drop those rows and continue.

1. Any null final_weight
2. Subgroup deff 40% above its usual range
3. Standard deviation for the CV

Sources

- Kish deff = n sum(w^2)/(sum w)^2 = 1 + CV^2 (population CV); n_eff = (sum w)^2 / sum(w^2); example weights give 1.3889 and 2.88. (verified: Computed locally on CPython 3.13, September 2026; identity follows algebraically.)
- PySpark 4.0.4: sum(when(...)) over no matches is null; stddev is the sample version and null for one row; mean and stddev of an empty DataFrame are null. (verified: Local run on PySpark 4.0.4, September 2026.)

D5 Errors typical of AI-generated code

Roadmap: Every month. Invented APIs, deprecated calls, missing edge cases, self-confirming tests.

D5-L1 Invented functions, parameters and syntax

Concept

AI assistants predict plausible code, so they sometimes produce a function, parameter or statement that does not exist, or exists only on another platform. Your reference code has one: `unity_catalog.sql` uses `CREATE ROW ACCESS POLICY`, which is Snowflake syntax. Databricks uses a SQL UDF with `ALTER TABLE ... SET ROW FILTER`.

What invented code does when it runs, checked on PySpark 4.0.4 and pandas 3.0.2:

Invented code	Result
`spark.read.delta(path)`	`AttributeError: 'DataFrameReader' object has no attribute 'delta'`
`df.drop_duplicates(keep="latest")`	`ValueError: keep must be either "first", "last" or False`
`pd.read_csv(path, encodng="utf-8")`	`TypeError: ... unexpected keyword argument 'encodng'. Did you mean 'encoding'?`
`CREATE ROW ACCESS POLICY ...` in Spark SQL	`[PARSE_SYNTAX_ERROR] Syntax error at or near 'ROW'`
`F.col("a").isnotnull()`	`TypeError: 'Column' object is not callable`

The last one is sly. The real method is `isNotNull`. PySpark treats an unknown attribute on a column as a **struct field lookup**, so `F.col("a").isnotnull` builds a valid expression for a field named `isnotnull`. Only the call fails. Without the parentheses, the typo would fail later, at analysis, far from the line that caused it.

Most invented names fail loudly, which makes them cheap to catch **if you run the code**. The expensive cases run: a parameter that exists but means something else, or a function from a newer version than your runtime.

Check every API an assistant gives you:

1. Find it in the official docs **for your version**: Databricks Runtime 17.3 LTS, PySpark 4.0, pandas 3.0.
2. In a notebook, confirm it exists: `hasattr(F, "try_divide")`, `help(pd.DataFrame.merge)`, or `inspect.signature(...)`.
3. For SQL, check the statement in the Databricks SQL reference, not a general SQL tutorial. Syntax from Snowflake, T-SQL and PostgreSQL looks alike and differs.

Treat an unfamiliar name as unverified until a step confirms it.

Why it matters

An assistant proposed `CREATE ROW ACCESS POLICY` to restrict survey rows by region. The engineer pasted it into the migration script without running it. The deployment failed in production at 2 a.m. with a parse error, and the table stayed unrestricted until morning. Checking the statement against the Databricks SQL reference would have taken two minutes.

Worked example: weak code

```
❶ CREATE ROW ACCESS POLICY region_policy
2   AS (region STRING) RETURNS BOOLEAN ->
❷   is_member('analysts_' || region);
4 ALTER TABLE research_silver.nat_respondents ADD ROW ACCESS POLICY
    region_policy ON (region);
```

1. **Line 1.** Snowflake syntax. Databricks fails with PARSE_SYNTAX_ERROR.
2. **Line 3.** is_member checks workspace groups. Unity Catalog guidance favours is_account_group_member.

Worked example: strong code

```
❶ CREATE FUNCTION research_governance.region_filter(region STRING)
2   RETURN is_account_group_member(concat('analysts_', region));
3
4 ALTER TABLE research_silver.nat_respondents
❷   SET ROW FILTER research_governance.region_filter ON (region);
```

1. **Line 1.** A SQL UDF returning a boolean, in a three-level name.
2. **Line 5.** The Databricks statement for row filters (Unity Catalog).

Look up every function, parameter and statement for your exact version, and run it on small data before it reaches a pipeline.

Recap card

- **CREATE ROW ACCESS POLICY** Snowflake syntax. Databricks: SQL UDF plus ALTER TABLE ... SET ROW FILTER.
- **F.col('a').isnotnull()** Unknown attribute becomes a struct field lookup, then 'Column' object is not callable. The method is isNotNull.
- **Three ways to confirm an API exists** Official docs for your version, hasattr or help() in a notebook, inspect.signature for parameters.
- **The costly invented-API case** Code that runs: a real parameter with a different meaning, or a function from a newer version.

Practice

Exercise 1. Multiple choice (level 1)

An assistant writes `spark.read.delta("/Volumes/research/raw/nat")` for Databricks. What happens?

- a) AttributeError: DataFrameReader has no attribute 'delta'
- b) It reads the Delta table
- c) It reads the files as Parquet
- d) A warning, then the read succeeds

Why?

- r1) The reader has format('delta').load(path) and table(name), but no delta() method.
- r2) Delta tables cannot be read by path.

Exercise 2. Predict the output (level 2)

What is printed? (PySpark 4.0)

```
from pyspark.sql import functions as F

expr = F.col("region").isnotnull
print(type(expr).__name__)
```

Exercise 3. Spot the bug (level 2)

An assistant wrote this Databricks SQL to mask the postcode column for non-analysts. Tap the line that does not exist in Databricks, then rewrite it with the Databricks statement.

```
CREATE FUNCTION research_governance.mask_postcode(p STRING)
  RETURN CASE WHEN is_account_group_member('analysts') THEN p ELSE '***'
   END;
```

```
3 CREATE MASKING POLICY postcode_mask ON research_silver.nat_respondents
    (postcode);
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 3)

Write `unknown_params(func: Callable[..., object], params: list[str]) -> list[str] | None`. Use `inspect.signature` to return the names in `params` that `func` does not accept, in the order given. If `func` accepts `**kwargs`, return `None`: you cannot tell from the signature alone.

What you need to know.

`inspect.signature(func).parameters` maps each parameter name to a `Parameter` object. Test a name with `in`. Each parameter's `.kind` says how it is passed. `inspect.Parameter.VAR_KEYWORD` is the kind of a `**kwargs` parameter, which accepts any keyword name.

```
1 import inspect
2
3 def f(a, b=1, **options): ...
4
5 params = inspect.signature(f).parameters
6 "a" in params                                                    # True
7 params["options"].kind is inspect.Parameter.VAR_KEYWORD         # True
```

Your code must pass these checks:

- Return the names the function does not accept, in order. Keyword-only parameters count as accepted.
- An empty list when every name exists.
- With **kwargs, any name is accepted by the signature. Return None.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
1 # Parameter names that a function does not accept.
2 def unknown_params(
3     func: Callable[..., object],
4     params: list[str],
5 ) -> list[str] | None: ...
```

Starter code:

```
1 def unknown_params(func, params):
2     return []
```

Exercise 5. Architecture design (level 4)

Set the team rule for APIs suggested by an assistant.

Options: Link the official docs page for our version in the pull request, Accept it if the assistant is confident.

1. Before code with a new function or parameter is merged
2. SQL statements such as row filters and masks
3. A function you cannot find in the docs

Sources

- PySpark 4.0.4: spark.read.delta and DataFrameWriter.saveAsDelta raise AttributeError; F.col('a').isnotnull is a Column and calling it raises TypeError; CREATE ROW ACCESS POLICY raises PARSE_SYNTAX_ERROR. (verified: Local run on PySpark 4.0.4, September 2026.)
- pandas 3.0.2: drop_duplicates(keep='latest') raises ValueError; read_csv(encodng=...) raises TypeError with a 'Did you mean' hint. (verified: Local run on pandas 3.0.2 with CPython 3.13, September 2026.)
- Databricks row filters use a SQL UDF and ALTER TABLE ... SET ROW FILTER; column masks use ALTER TABLE ... ALTER COLUMN ... SET MASK. (verified: https://docs.databricks.com/aws/en/data-governance/unity-catalog/filters-and-masks/manually-apply (checked 2026-09-25))

D5-L2 Deprecated APIs and version mismatches

Concept

An assistant learned from years of code written for older versions. It often suggests an API that worked in 2021 and has since been deprecated, renamed or removed. Your reference `observability.py` uses `datetime.utcnow()`, deprecated from Python 3.12.

Examples that matter for this course, checked in September 2026:

Old code	Current state	Use instead
`datetime.utcnow()`	Deprecated, Python 3.12	`datetime.now(timezone.utc)`
`df.append(other)`	Removed from pandas (gone in 3.0)	`pd.concat([df, other])`
`df.applymap(f)`	Removed in pandas 3.0	`df.map(f)`
`df["a"][0] = x`	Never updates `df` under pandas 3 Copy-on-Write	`df.loc[0, "a"] = x`
`import distutils`	Removed in Python 3.12	`packaging`, `setuptools`
`input_file_name()`		`_metadata.file_path`

Old code	Current state	Use instead
	Not supported from Databricks Runtime 17.3 LTS	
`mssparkutils` in Fabric	Being retired	`notebookutils`

The chained assignment row is the dangerous one. pandas 3.0.2 emits a `ChainedAssignmentError` warning and leaves the DataFrame unchanged. If warnings are hidden, the update simply does not happen.

Version mismatches cause the opposite problem: code for a **newer** version than yours. A function added in PySpark 4.1 fails on Databricks Runtime 17.3 LTS, which ships Spark 4.0. Pyodide in this app runs pandas 3.0.2. Your laptop might run 3.0.6.

Defend in three ways:

- **State the versions** in every prompt, and in the project: `VERSIONS.md`, `pyproject.toml` pins, the runtime in the job definition.
- **Show warnings** in tests. Run pytest with `-W error::DeprecationWarning` for your own code, so a deprecated call fails the build.
- **Compare versions as numbers**, never as strings. `"3.10" < "3.9"` is `True` as text.

Why it matters

An assistant fixed a data cleaning function by adding `df["weight"][mask] = 1.0` to cap weights. Tests passed on a developer laptop with pandas 1.5. In the pandas 3 environment, the cap never happened. Weights above 5 reached gold unchanged for four waves. The warning had been filtered out in the job's logging configuration.

Worked example: weak code

```
1 import pandas as pd
2
3 def cap_weights(df: pd.DataFrame, cap: float) -> pd.DataFrame:
4     """Cap weights at cap."""
❶     df["final_weight"][df["final_weight"] > cap] = cap
❷     return df.applymap(lambda v: v)
```

1. **Line 5.** Chained assignment. Under pandas 3 Copy-on-Write it never changes df.
2. **Line 6.** applymap no longer exists in pandas 3.0: AttributeError.

Worked example: strong code

```
1  import pandas as pd
2
3  def cap_weights(df: pd.DataFrame, cap: float) -> pd.DataFrame:
1      """Return a copy with final_weight capped at cap. Written for pandas
       3.0."""
5      out = df.copy()
2      out.loc[out["final_weight"] > cap, "final_weight"] = cap
7      return out
```

1. **Line 4.** The docstring states the target version.
2. **Line 6.** One .loc call: row mask and column together. Works in every pandas version.

Assume suggested code targets some other version. Check deprecations for yours, turn warnings into test failures, and compare versions numerically.

Recap card

- **Chained assignment in pandas 3** df['a'][i] = x never updates df (Copy-on-Write). Use df.loc[i, 'a'] = x.
- **pandas 3 removals** DataFrame.append (use pd.concat) and applymap (use DataFrame.map).
- **input_file_name() on DBR 17.3 LTS** Not supported. Use _metadata.file_path.
- **Comparing version strings** Never as text: '3.10' < '3.9'. Compare integer tuples or use packaging.version.

Practice

Exercise 1. Multiple choice (level 1)

An assistant adds `F.input_file_name()` to your bronze job on Databricks Runtime 17.3 LTS. What do you use instead?

- a) The _metadata.file_path column
- b) Keep input_file_name(), it is fine
- c) os.path.basename on the DataFrame
- d) A Python UDF reading the path

Why?

- r1) Databricks release notes state input_file_name is no longer supported from 17.3 LTS. The file metadata column replaces it.
- r2) input_file_name was renamed file_path.

Exercise 2. Predict the output (level 2)

What does this print with pandas 3.0?

```
import pandas as pd

df = pd.DataFrame({"w": [1.0, 9.0]})
df["w"][1] = 5.0
print(df["w"].tolist())
```

Exercise 3. Fill the gap (level 2)

Update this code from pandas 1.x to pandas 3.0.

```
combined = pd._____(1)([waves, new_wave], ignore_index=True)
labels = combined._____(2)(str.strip)
```

Exercise 4. Write the function (level 3)

Write `version_at_least(installed: str, required: str) -> bool` comparing dotted versions such as `"3.0.2"` and `"3.10"` numerically. Ignore anything after the numeric part of each segment (`"4.0.4rc1"` counts as 4.0.4). Missing segments count as 0, so `"3.0"` equals `"3.0.0"`. Raise `ValueError` if a segment has no leading digits.

Your code must pass these checks:

- 3.10 is newer than 3.9. Compare numbers, not strings.
- Pad missing segments with 0.
- Ignore suffixes after the digits in each segment.
- A segment without digits is invalid.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
SEGMENT = ...

# Numeric segments of a dotted version string.
def _parse(version: str) -> tuple[int, ...]: ...

# True if installed is the same as or newer than required.
def version_at_least(installed: str, required: str) -> bool: ...
```

Starter code:

```
def version_at_least(installed, required):
    return installed >= required
```

Exercise 5. Spot the bug (level 3)

This function caps weights, but under pandas 3 the weights come back unchanged. Tap the line, then fix it without changing the caller's DataFrame.

```python
import pandas as pd

def cap_weights(df: pd.DataFrame, cap: float) -> pd.DataFrame:
    """Return a copy with final_weight capped at cap."""
    out = df.copy()
    out["final_weight"][out["final_weight"] > cap] = cap
    return out
```

Write down the line number of the bug. Then write the fix.

Sources

- pandas 3.0.2: no DataFrame.append or applymap; DataFrame.map exists; default string dtype 'str'; chained assignment emits ChainedAssignmentError and leaves the frame unchanged. Python 3.13: distutils missing; utcnow deprecated. (verified: Local run on pandas 3.0.2 with CPython 3.13.12, September 2026.)
- input_file_name is no longer supported from Databricks Runtime 17.3 LTS. (verified: https://docs.databricks.com/aws/en/release-notes/runtime/17.3lts (checked 2026-09-25))
- mssparkutils is renamed NotebookUtils and will be retired. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/notebook-utilities (see lesson C5-L1))

D5-L3 Five review checks for AI-written code

Concept

Code from an assistant usually runs. The problems hide in what it leaves out. Review it with five checks.

1. Missing edge cases. Assistants write the happy path. Ask what happens with an empty DataFrame, a zero total, a null, a duplicate key, a single row. Your reference `check_weight_distribution` fails on an empty wave for exactly this reason (lesson D4-L3).

2. Self-confirming tests. A test that computes its expected value with the same formula as the code proves nothing:

```python
def test_share():
    assert share(3, 4) == 3 / 4   # repeats the code
```

If the formula is wrong, the test is wrong the same way. Write expected values by hand, from small data you understand: `assert share(3, 4) == 0.75`, `assert share(0, 0) is None`.

3. Secrets in code. Assistants fill in placeholders that look like real tokens, or keep the one you pasted into the prompt. A token in a notebook ends up in Git history. Read secrets at run time: `dbutils.secrets.get(scope, key)` on Databricks, `notebookutils.credentials.getSecret(vault_url, name)` in Fabric, or an environment variable on your laptop.

4. Over-engineering. A 12-line task arrives as an abstract base class, a factory and a config object. Every extra layer is code to test and review. Ask for the simplest version that meets the requirement, then add structure when a second use case appears.

5. The wrong filter context in DAX.
`CALCULATE(SUM(fact_reach[weighted_reach]), ALL(fact_reach))` removes **every** filter on the fact table, including wave and region slicers. A "share of total" measure then divides by the all-time, all-region total. Remove only the filter the measure means to ignore: `ALL(dim_brand)` for a share across brands.

Each check takes a minute. Each catches a class of bug that runs without error.

Why it matters

An assistant wrote a reach function and its tests in one reply. All 12 tests passed. Each test computed its expected value with the same expression as the function, including the same missing `is None` check. A reviewer replaced three expected values with hand-worked numbers, and two tests failed at once: the function returned 0.0 where it should have returned None.

Worked example: weak code

```
1  API_TOKEN = "paste-your-token-here-EXAMPLE"
2
3  def share(part: float, total: float) -> float:
2      return part / total
5
6  def test_share():
3      assert share(3, 4) == 3 / 4
```

1. **Line 1.** A secret in code. The real token replaces the placeholder and lands in Git.
2. **Line 4.** No guard for a zero total.
3. **Line 7.** The expected value repeats the code. It cannot catch a wrong formula.

Worked example: strong code

```
1  import os
```

```
 2
 3 def share(part: float, total: float) -> float | None:
 4     """part / total, or None when total is 0."""
 1     return None if total == 0 else part / total
 6
 7 def api_token() -> str:
 8     """Read the token at run time, never from source code."""
 2     return os.environ["SURVEY_API_TOKEN"]
10
11 def test_share():
12     assert share(3, 4) == 0.75
 3     assert share(0, 0) is None
```

1. **Line 5.** The edge case is handled and documented.
2. **Line 9.** The secret stays outside the code. On Databricks use dbutils.secrets.get.
3. **Line 13.** Hand-written expected values, including the edge case.

Review AI code for edge cases, self-confirming tests, secrets, needless structure and, in DAX, filters removed too widely.

Recap card

- **Five review checks for AI code** Edge cases, self-confirming tests, secrets, over-engineering, DAX filter context.
- **Self-confirming test** Expected value computed with the code's own formula. Use hand-worked values instead.
- **Where secrets live** Secret scopes or Key Vault, read at run time. Never in code or prompts.
- **ALL(fact_table) in a share measure** Removes every filter, including wave and region slicers. Use ALL on the one dimension you mean.

Practice

Exercise 1. Multiple choice (level 1)

An assistant writes `assert weighted_pct(r, b) == r / b * 100` as the test for `weighted_pct`. What is wrong with it?

- a) It repeats the implementation, so a wrong formula passes
- b) It should use pytest.approx
- c) Tests must not use multiplication
- d) Nothing: it checks the output

Why?

- r1) A test's expected value must come from outside the code: small data worked by hand.
- r2) Tests should never call the function they test.

Exercise 2. Spot the bug (level 2)

An assistant produced this Databricks helper. Tap the line that must never be committed, then fix it to read the value from the secret scope `survey-api`, key `token`.

```
1 import requests
2
3 API_TOKEN = "tok_EXAMPLE_do_not_use_0000"
4
5 def fetch_wave(wave_id: str) -> dict[str, object]:
6     """Fetch wave metadata from the survey platform."""
7     headers = {"Authorization": f"Bearer {API_TOKEN}"}
8     return requests.get(f"https://survey.example/api/waves/{wave_id}",
    headers=headers, timeout=30).json()
```

Write down the line number of the bug. Then write the fix.

Exercise 3. Multiple choice (level 2)

A report has a wave slicer set to W14. An assistant's measure is `Share = DIVIDE(SUM(fact_reach[weighted_reach]), CALCULATE(SUM(fact_reach[weighted_reach]), ALL(fact_reach)))`. What is the denominator?

- a) Weighted reach for all waves, all regions and all brands
- b) Weighted reach for W14 across all brands
- c) Weighted reach for W14 for the current brand
- d) An error, because ALL needs a column

Why?

- r1) ALL(fact_reach) removes every filter on the fact table, including the wave slicer.
- r2) CALCULATE always ignores slicers.

Exercise 4. Write the function (level 3)

An assistant wrote `weighted_mean(values, weights)` as `sum(v * w for v, w in zip(values, weights)) / sum(weights)`. Rewrite it with the edge cases it missed. Return `None` if the lists are empty or the weight total is 0. Skip pairs

where the value or the weight is `None`. Raise `ValueError` if the lists differ in length or any weight is negative.

Your code must pass these checks:

- (10 x 1 + 20 x 3) / 4 = 17.5.
- Empty lists and a zero weight total return None.
- Skip pairs with a None value or weight.
- zip silently truncates different lengths. Raise ValueError, and for negative weights too.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Weighted mean that skips missing pairs.
def weighted_mean(
    values: list[float | None],
    weights: list[float | None],
) -> float | None: ...
```

Starter code:

```python
def weighted_mean(values, weights):
    return sum(v * w for v, w in zip(values, weights)) / sum(weights)
```

Exercise 5. Refactor (level 4)

An assistant answered "flag weights above a cap" with the code below. Replace it with one function, `flag_high_weights(weights: list[float], cap: float) -> list[int]`, returning the indexes of weights above `cap`. No classes.

Your code must pass these checks:

- Return indexes of weights strictly above the cap.
- No weights, no flags.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Indexes of weights strictly above cap.
def flag_high_weights(
    weights: list[float],
    cap: float,
) -> list[int]: ...
```

Starter code:

```python
from abc import ABC, abstractmethod


class WeightRule(ABC):
    @abstractmethod
    def check(self, weight):
```

```
        ...


class CapRule(WeightRule):
    def __init__(self, cap):
        self.cap = cap

    def check(self, weight):
        return weight > self.cap


class RuleFactory:
    @staticmethod
    def create(kind, **kwargs):
        if kind == "cap":
            return CapRule(**kwargs)
        raise ValueError(kind)


def flag_high_weights(weights, cap):
    rule = RuleFactory.create("cap", cap=cap)
    return [i for i, w in enumerate(weights) if rule.check(w)]
```

Sources

- dbutils.secrets.get(scope, key) reads a secret and values are redacted in notebook output. (verified: https://docs.databricks.com/aws/en/security/secrets/ (checked 2026-09-25))
- In DAX, ALL(table) inside CALCULATE removes all filters on that table; ALL on one dimension keeps other filters. (verified: Taught and tested in lesson C6-L2; DAX CALCULATE and ALL semantics per Microsoft Learn DAX reference.)

D6 Verifying AI output

Roadmap: Every month. A workflow to check AI code before you trust it.

D6-L1 Read before you run, and check each API

Concept

Track D so far showed what goes wrong in AI-written code. Module D6 is the routine that catches it. Step one happens before the code runs at all.

Why read first. Running code is the fastest check for invented names, but running is not free. A suggested cell might overwrite a table, drop a column or read a whole year of data. Reading takes two minutes. Restoring a bronze table takes a morning, if you can do it at all.

The read-through:

1. **Restate the goal** in one sentence before you look at the code. "Append wave W14 deliveries to bronze with audit columns." You now have something to check against.
2. **List every external name**: imports, functions, methods, parameters, tables and columns. An `ast` walk lists the calls for you (exercise 4).
3. **Check each name** against the docs for your version and against the table schema. `DESCRIBE TABLE`, `df.columns`, `hasattr`, `inspect.signature` and `help()` answer most questions in seconds.
4. **Find side effects**: `mode("overwrite")`, `DROP`, `DELETE`, `TRUNCATE`, `VACUUM`, writes to production names. Each needs a reason that matches the goal.
5. **Check the scope**: filters on tracker and wave, no `collect()` on large data, no loop that calls `count()` per item.

Then run it, on development data first, in a dev schema: `research_dev`, not `research_bronze`.

Where assistants go wrong most in this course's work: the write mode, three-level table names, the filter keys, and parameters that exist but mean something else. `mode("overwrite")` on an append-only bronze table replaces every earlier delivery. It runs without error.

A read-through does not prove code correct. It removes the mistakes that are visible on the page, so testing (lesson D6-L2) can focus on the ones that are not.

Why it matters

An assistant rewrote a bronze ingestion cell to "simplify" it. The new cell used `mode("overwrite")`. It ran in production and succeeded, replacing 38 waves of append-only deliveries with one. Delta time travel recovered the table because retention had not yet expired. A two-minute read, looking for side effects, would have found the one word that mattered.

Worked example: weak code

```
1 # Suggested by an assistant: "load the new delivery"
2 delivery = spark.read.format("csv").option("header", True).load(path)
2 delivery.write.mode("overwrite").saveAsTable("research_bronze.nat_raw")
```

1. **Line 1.** No stated goal to check against.
2. **Line 3.** Overwrite on an append-only table replaces every earlier delivery. It runs cleanly.

Worked example: strong code

```
1 # Goal: append the W14 delivery to bronze with audit columns. Bronze is
    append-only.
2 # Checked: format("csv").option("header", True) and saveAsTable (PySpark
    4.0 API reference).
3 from pyspark.sql import functions as F
4
5 delivery = spark.read.format("csv").option("header", True).load(path)
6 delivery = delivery.withColumn("source_file",
    F.col("_metadata.file_path"))
2 delivery.write.mode("append").saveAsTable("research_dev.nat_raw")
```

1. **Line 1.** The goal, written first, makes the write mode checkable.
2. **Line 7.** Append, and into a dev schema until the run has been checked.

Restate the goal, list and check every name, hunt for side effects, then run on development data.

Recap card

- **Five steps of the read-through** Restate the goal, list names, check each name, find side effects, run on dev data.
- **Side effects to hunt for** mode('overwrite'), DROP, DELETE, TRUNCATE, VACUUM, writes to production names.
- **Quick existence checks in a notebook** hasattr for names, inspect.signature(...).parameters for parameters, DESCRIBE TABLE or df.columns for columns.
- **What a read-through does not do** Prove correctness. It clears visible mistakes so tests can find the rest.

Practice

Exercise 1. Multiple choice (level 1)

An assistant gives you a cell that writes to `research_bronze.nat_raw`. What do you check first?

- a) The write mode and the target, against the goal and the table's rules
- b) Whether the code is formatted with ruff
- c) How long it will take
- d) Nothing: run it and read the result

Why?

- r1) Side effects on a production table cannot always be undone. Bronze is append-only.
- r2) Assistants always pick the wrong write mode.

Exercise 2. Put in order (level 2)

Order the read-through of an AI-written cell.

- Restate the goal in one sentence
- Find side effects: write modes, DROP, DELETE
- Check each name against the docs for your version and the schema
- Run it on development data in a dev schema
- List every function, parameter, table and column

Exercise 3. Fill the gap (level 2)

In a notebook, confirm that a suggested function and parameter exist.

```
import inspect
from pyspark.sql import functions as F

_____(1)(F, "try_divide")
"validate" in inspect._____(2)(pd.DataFrame.merge).parameters
```

Exercise 4. Write the function (level 3)

Write `called_names(source: str) -> list[str]` that parses Python source with `ast` and returns the dotted names of every call, such as `spark.read.format` or `F.col`, unique and sorted. Ignore calls whose target is not a name or attribute chain (for example a call on a subscript). This gives you the checklist for step 2 of the read-through.

What you need to know.

`ast.parse(source)` turns Python code into a tree of nodes. `ast.walk(tree)` visits every node. Three node types matter here:

- `ast.Call` is a call. Its `.func` is the thing being called.
- `ast.Name` is a plain name such as `print`. Its `.id` is the text.
- `ast.Attribute` is `something.attr`. Its `.attr` is the last part and its `.value` the node before the dot.

```
import ast

func = ast.parse("os.path.join(a)").body[0].value.func
func.attr, func.value.attr, func.value.value.id   # ('join', 'path', 'os')
```

Test a node's type with `isinstance(node, ast.Call)`.

Your code must pass these checks:

- Return dotted names for name and attribute chains. A call on a call result (.load on format(...)) has no plain dotted name: skip it.
- Each name once, sorted.
- Skip calls on subscripts.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Dotted name for a Name or Attribute chain, else None.
def _dotted(node: ast.expr) -> str | None: ...

# Sorted unique dotted names of every call in the source.
def called_names(source: str) -> list[str]: ...
```

Starter code:

```
def called_names(source):
    return [line for line in source.splitlines() if "(" in line]
```

Exercise 5. Spot the bug (level 3)

Goal: append this wave's delivery to bronze, which is append-only. Tap the line that breaks the goal, then fix it.

```
from pyspark.sql import functions as F

delivery = spark.read.format("csv").option("header", True).load(path)
delivery = delivery.withColumn("ingestion_timestamp",
    F.current_timestamp())
delivery.write.mode("overwrite").saveAsTable("research_bronze.nat_raw")
```

Write down the line number of the bug. Then write the fix.

Sources

- PySpark DataFrameReader.format/option/load, DataFrameWriter.mode and saveAsTable, and the _metadata.file_path column exist as used. (verified: Local runs on PySpark 4.0.4 (see D1 and D5 claims); _metadata.file_path per Databricks Auto Loader docs (A8-L2).)

D6-L2 Small data with a known answer, failing test first

Concept

After the read-through, run the code where you already know the right answer. That means small data you built by hand, and a test written **before** you accept the code.

Build the known-answer data. Five to ten rows, chosen so every rule in the requirement matters:

- a normal case you can compute in your head,
- a duplicate key, a null, a zero total, an empty group,
- a value exactly on a boundary, such as a weight equal to the cap.

Write the expected output by hand, next to the input. If you cannot work out the answer by hand, the requirement is not clear enough yet, and neither the assistant nor you should write code.

Write the test first, and watch it fail. The order matters:

1. Write the test with the hand-worked expected values.
2. Run it against the current code, or a stub. It must **fail**. A test that passes before the feature exists tests nothing.
3. Accept or write the code. Run the test. It passes.
4. Break the code on purpose, say change `>` to `>=`. The test must fail again. If it still passes, it does not cover that rule.

Step 4 is a quick form of mutation testing. It tells you whether your tests would catch the mistakes an assistant makes.

For Spark code, run the same idea on a local SparkSession. `pyspark.testing.assertDataFrameEqual(actual, expected)` compares two DataFrames. In PySpark 4.0 it ignores row order by default (`checkRowOrder=False`) and compares floats with `rtol=1e-05`. `assertSchemaEqual` compares schemas.

Keep the known-answer tests in the repository. They check every later change, including the next suggestion from an assistant.

Why it matters

An assistant wrote a deduplication step for silver and a test for it. The test passed. A reviewer changed the dedup key from `(respondent_id, wave_id)` to `respondent_id` alone, a real bug, and the test still passed. Its fixture had only one wave. Adding a respondent who appears in two waves made the test fail on the broken key, as it should.

Worked example: weak code

```
1 def test_share():
2     result = share(30, 120)
(1)   assert result is not None
(2)   assert 0 <= result <= 1
```

1. **Line 3.** Passes for almost any implementation, including wrong ones.
2. **Line 4.** A range, not an answer. 0.3 would pass as well as 0.25.

Worked example: strong code

```
 1 import pytest
 2
 3 @pytest.mark.parametrize(
 4     ("part", "total", "expected"),
 5     [
(1)        (30, 120, 0.25),   # normal case, worked by hand
 7         (0, 50, 0.0),      # zero share is a real value
(2)        (5, 0, None),      # zero total has no share
 9         (50, 50, 1.0),     # boundary
10     ],
11 )
12 def test_share(part: float, total: float, expected: float | None) -> None:
13     assert share(part, total) == expected
```

1. **Line 6.** An exact answer you computed yourself.
2. **Line 8.** The edge case the requirement names.

Hand-built data, hand-worked answers, a test that fails first, and a deliberate break to prove the test bites.

Recap card

- **Known-answer data** 5 to 10 hand-built rows: normal case, duplicate, null, zero total, empty group, boundary.
- **Test-first order** Write the test, watch it fail, accept the code, watch it pass, break the code on purpose, watch it fail.
- **assertDataFrameEqual defaults (PySpark 4.0)** checkRowOrder=False, rtol=1e-05, atol=1e-08.

- **Why break code on purpose** If the test still passes, it misses that rule. A quick form of mutation testing.

Practice

Exercise 1. Multiple choice (level 1)

You write a test for a feature that does not exist yet, and it passes. What does that tell you?

- a) The test does not check the feature
- b) The feature already works
- c) The test is well written
- d) The code is fast

Why?

- r1) A test must fail when the behaviour it checks is missing. Passing without the feature means it asserts something else.
- r2) Tests always fail the first time they run.

Exercise 2. Multiple choice (level 2)

Your tests pass. You change `w > cap` to `w >= cap` in the code on purpose, and the tests still pass. What do you add?

- a) A case with a weight exactly equal to the cap
- b) More cases with weights far above the cap
- c) A timeout on the test
- d) Nothing: the tests pass

Why?

- r1) > and >= differ only on the boundary, so only a boundary case can tell them apart.
- r2) Mutations always need more data.

Exercise 3. Fill the gap (level 2)

Compare a Spark result with the expected rows in a local test, ignoring row order.

```
1 from pyspark.testing import _____(1)
2
3 expected = spark.createDataFrame([("W01", 0.25)], "wave_id string, share
    double")
4 assertDataFrameEqual(result, expected, _____(2)=False)
```

Exercise 4. Write the function (level 3)

Write the known-answer cases before the code. Write `share_cases() -> list[tuple[float, float, float | None]]` returning `(part, total, expected)` cases for `share(part, total)`, which returns `part / total`, or `None` when `total` is 0. Your cases must all pass against a correct `share`, and each of three wrong versions must fail at least one of them: one that raises on a zero total, one that returns 0.0 on a zero total, and one that returns a percentage (times 100).

Your code must pass these checks:

- Every case must pass against a correct share. Work the expected values out by hand.
- Add a case with total 0, expecting None. A version that divides by zero must fail it.
- Your zero-total case must expect None, not 0.0, so a version returning 0.0 fails.
- Add a normal case, such as (1, 4, 0.25). A version returning 25.0 must fail it.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Known-answer cases for share(part, total), worked out by hand.
def share_cases() -> list[tuple[float, float, float | None]]: ...
```

Starter code:

```
def share_cases():
    return [(1.0, 2.0, 0.5)]
```

Exercise 5. Architecture design (level 4)

An assistant offers a silver deduplication step keyed on respondent and wave. Plan the known-answer test.

Options: A duplicate in one wave, the same respondent in two waves, a null key, an unchanged respondent, 1,000 rows copied from production.

1. Fixture rows
2. Expected output
3. Proving the test bites

Sources

- pyspark.testing.assertDataFrameEqual signature in 4.0.4 has checkRowOrder=False, rtol=1e-05, atol=1e-08; assertSchemaEqual exists. (verified: inspect.signature on PySpark 4.0.4, September 2026.)

D6-L3 Ask for assumptions, then compare a second approach

Concept

Every piece of code assumes things about its input. Code from an assistant assumes things nobody wrote down. The last two steps of the verification routine bring them into the open.

Ask for the assumptions. After the assistant writes code, ask: "List every assumption this code makes about its input data, one per line." Typical answers for a pipeline step:

- `respondent_id` is unique within a wave.
- `final_weight` is never null and always positive.
- `wave_id` looks like `W` plus two digits.
- Timestamps are in UTC.
- The lookup table has one row per key.

Then **check each one against the real data**, and turn the important ones into checks that run with the pipeline. An assumption you have not checked is a bug waiting for the wave where it stops being true. The list is also a review aid: if an assumption surprises you, the code does something you did not ask for.

The assistant can miss assumptions too. Add your own from the D3 and D4 lessons: nulls in filters, duplicate keys in joins, empty groups, time zones.

Compare a second approach. Compute the same result by an independent route, and compare:

- Spark SQL against the PySpark DataFrame version.
- pandas on a small extract against Spark on the same rows.
- A total from the gold table against a direct aggregate of silver.

"Independent" matters. Asking the same assistant to rewrite the same logic tends to repeat the same mistake. Write the second version yourself, or derive it from a different table.

When the two disagree, do not pick the one you like. Run both on your known-answer data (lesson D6-L2). The one that matches the hand-worked answer is right, and the other has a bug worth understanding.

Why it matters

An assistant's gold aggregation and an analyst's hand-written SQL disagreed by 0.8 points of reach for one region. On the known-answer fixture, the SQL matched the hand calculation. The assistant's version used an inner join to the region lookup, which dropped 212 respondents with a region code missing from the lookup. Asked for its assumptions, the assistant had listed "every region code exists in the lookup". Nobody had checked it.

Worked example: weak code

```
1  # Assistant's code, accepted after it ran
1  gold = silver.join(regions,
       "region_code").groupBy("region").agg(F.sum("final_weight"))
3  gold.write.mode("overwrite").saveAsTable("research_gold.nat_region_totals")
```

1. **Line 2.** An inner join assumes every code is in the lookup. Nobody listed or checked that.

Worked example: strong code

```
1  # Assumption: every region_code in silver exists in regions. Checked:
1  missing = silver.join(regions, "region_code", "left_anti").count()
3  if missing:
4      raise ValueError(f"{missing} respondents have a region code missing
       from the lookup")
5
6  gold = silver.join(regions, "region_code",
       "left").groupBy("region").agg(F.sum("final_weight").alias("w"))
7  # Second approach: the grand total must equal silver's weight total.
8  gold_total = gold.agg(F.sum("w")).first()[0]
9  silver_total = silver.agg(F.sum("final_weight")).first()[0]
2  if abs(gold_total - silver_total) > 1e-6 * silver_total:
11     raise ValueError(f"gold {gold_total} != silver {silver_total}")
```

1. **Line 2.** The assumption, turned into a check that runs every time.
2. **Line 10.** An independent total. A dropped or duplicated row makes them disagree.

Make the assistant state its assumptions, check them on real data, keep the important ones as checks, and confirm results by an independent route.

Recap card

- **Prompt for assumptions** 'List every assumption this code makes about its input data, one per line.' Then check each on real data.
- **Common data assumptions to check** Unique keys per grain, non-null positive weights, id formats, UTC timestamps, one lookup row per key.
- **An independent second approach** SQL vs DataFrame code, pandas vs Spark on an extract, gold totals vs silver aggregates. Not the same assistant rewriting itself.
- **When two methods disagree** Run both on known-answer data. The one matching the hand-worked answer is right.

Practice

Exercise 1. Multiple choice (level 1)

An assistant lists "respondent_id is unique within a wave" as an assumption of its code. What do you do?

- a) Check it on the real data, and add a uniqueness check if it matters
- b) Accept it: the assistant knows the data
- c) Delete duplicates so the assumption holds
- d) Ignore it: assumptions are documentation only

Why?

- r1) An assumption is a claim about data. Only the data can confirm it, and it can stop holding in a later wave.
- r2) Assistants list false assumptions on purpose.

Exercise 2. Predict the output (level 2)

Two methods compute a region's weighted total. What does this print?

```
method_a = 1250.0   # assistant's PySpark, inner join
method_b = 1310.0   # your SQL, left join
known_answer = 1310.0
print("a" if method_a == known_answer else "b")
```

Exercise 3. Fill the gap (level 2)

Check the assumption "every region_code in silver exists in regions".

```
missing = silver.join(regions, "region_code", "_____(1)")._____(2)()
```

Exercise 4. Write the function (level 3)

Write `violated_assumptions(rows: list[dict[str, object]]) -> list[str]` that checks three assumptions and returns the names of those that fail, in this order: `"unique_respondent_per_wave"` (no two rows share `respondent_id` and `wave_id`), `"positive_weight"` (every `final_weight` is a number above 0, not None), `"wave_format"` (every `wave_id` is a string matching `W` followed by exactly two digits). An empty list of rows violates nothing.

Your code must pass these checks:

- The same respondent in two waves is fine. Uniqueness is per wave.
- A duplicate key, a None weight and 'W1' violate all three, reported in order.
- A weight of 0.0 is not positive. Check with > 0, and None with is None.
- No rows, no violations.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
WAVE_PATTERN = ...

# Names of data assumptions that the rows break.
def violated_assumptions(
    rows: list[dict[str, object]],
) -> list[str]: ...
```

Starter code:

```
def violated_assumptions(rows):
    return []
```

Exercise 5. Architecture design (level 4)

Choose the second approach for verifying an assistant's PySpark gold aggregation of weighted reach by region and wave.

Options: Your own Spark SQL query over silver, written without looking at the PySpark code, Ask the same assistant to rewrite it in SQL.

1. Independent method
2. The two results differ for one region
3. After the fix

Sources

- left_anti joins return left rows with no match; used here to check a lookup assumption. (verified: Local run on PySpark 4.0.4 (see D3-L1 claims), September 2026.)

D7 Review practice

Roadmap: Every month. 25 AI-style snippets with seeded bugs and a hidden answer key.

D7-L1 Review practice: Python and pandas

Concept

This lesson is practice. Each exercise shows a short snippet in the style an assistant writes: tidy, plausible and typed. Each holds one seeded bug, drawn from your reference pipeline and from the lessons in Track D. The answer key stays hidden until you submit.

How to review each snippet:

1. Read the prompt's goal first. The bug is a gap between the goal and the code.
2. Walk the lines in order. For each, ask: what does this assume, and what input breaks it?
3. Tap the line you would change. Then write the fix, and run it against the checker.

What to look for in Python and pandas code:

- Names used but never imported, and imports that shadow built-ins.
- Deprecated calls: `datetime.utcnow()`, chained assignment, `applymap`.
- Type hints that lie: `str = None`.
- Truthiness where 0, 0.0 or an empty string is a valid value.
- Mutable default arguments.
- Joins that drop or duplicate rows, and nulls that behave unexpectedly in filters.
- Text compared where numbers were meant, such as wave ids.

Your score counts two things: bugs found (the right line) and fixes proposed (a fix that passes the checker). Aim to find the bug before you open a hint. When a hint helps, note which one: that is the check you skipped.

Work at your own pace. A snippet you get wrong is worth more than one you get right, as long as you read the explanation.

Why it matters

Reviewing is a separate skill from writing. Engineers who review AI output well do it by habit: goal first, then line by line, asking what each line assumes. Short, deliberate practice on seeded bugs builds that habit faster than waiting for real bugs to appear in production.

Worked example: weak code

```
1 def latest_wave(wave_ids: list[str]) -> str:
2     """Return the most recent wave."""
1     return max(wave_ids)
```

1. **Line 3.** Text comparison: max(['W9', 'W10']) is 'W9'.

Worked example: strong code

```
1 def latest_wave(wave_ids: list[str]) -> str:
2     """Return the most recent wave. Wave ids look like 'W' plus a
    number."""
1     return max(wave_ids, key=lambda w: int(w[1:]))
```

1. **Line 3.** Compares the numbers, so W10 beats W9.

Goal first, then line by line: what does each line assume, and which input breaks it?

Recap card

- **Review routine for a snippet** Goal first, then each line: what does it assume, and which input breaks it?
- **Python bugs seeded in this lesson** Missing import, mutable default, utcnow, str = None, truthiness of 0.0, inner join, text max.
- **When truthiness is fine** Testing a list or dict for emptiness. Not for numbers where 0 is valid.

Practice

Exercise 1. Spot the bug (level 1)

Goal: return a new unique run id for the run log. Find the bug and fix it.

```
1 def new_run_id() -> str:
2     """Return a new unique run id."""
3     return str(uuid.uuid4())
```

Write down the line number of the bug. Then write the fix.

Exercise 2. Predict the output (level 1)

Goal: each call starts with an empty list of warnings. What does this print?

```python
def add_warning(msg, warnings=[]):
    warnings.append(msg)
    return warnings

add_warning("weight sum low")
print(len(add_warning("null ids")))
```

Exercise 3. Spot the bug (level 2)

Goal: record the run time as an aware UTC timestamp. Find the bug and fix it.

```python
from datetime import datetime, timezone

def run_timestamp() -> datetime:
    """Current time in UTC for the run log."""
    return datetime.utcnow()
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Spot the bug (level 2)

Goal: `error_message` is optional. The signature must say so for mypy. Find the bug and fix the signature.

```python
def log_run(
    pipeline_name: str,
    status: str,
    error_message: str = None,
) -> None:
    """Append one record to the run log."""
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Spot the bug (level 3)

Goal: count respondents with a recorded weight. A weight of 0.0 is recorded and valid for this count. Find the bug and fix it.

```python
def count_weighted(weights: list[float | None]) -> int:
    """Number of respondents with a recorded weight."""
    recorded = [w for w in weights if w]
    return len(recorded)
```

Write down the line number of the bug. Then write the fix.

Exercise 6. Multiple choice (level 3)

Goal: keep every respondent except those in region `N`, including respondents with no region. Does `df[df["region"] != "N"]` meet the goal in pandas 3?

- a) Yes: in pandas, != keeps missing regions
- b) No: missing regions are dropped
- c) No: it raises on missing values

- d) Only if the column is a string dtype

Why?

- r1) pandas treats missing != 'N' as True, so those rows stay. Spark would drop them.
- r2) pandas fills missing values with empty strings before comparing.

Exercise 7. Spot the bug (level 4)

Goal: attach region names to every respondent. Respondents whose code is missing from the lookup must stay, with no name. Find the bug and fix it.

```
1 import pandas as pd
2
3 def add_region(resp: pd.DataFrame, regions: pd.DataFrame) -> pd.DataFrame:
4     """Attach region names. One row per respondent."""
5     lookup = regions.dropna(subset=["code"])
6     return resp.merge(lookup, on="code", how="inner",
    validate="many_to_one")
```

Write down the line number of the bug. Then write the fix.

Exercise 8. Spot the bug (level 5)

Goal: return the most recent wave from ids like `W9`, `W10`, `W11`. Find the bug and fix it.

```
1 def latest_wave(wave_ids: list[str]) -> str:
2     """Return the most recent wave id."""
3     if not wave_ids:
4         raise ValueError("no waves")
5     return max(wave_ids)
```

Write down the line number of the bug. Then write the fix.

Sources

- Snippet behaviours (NameError without import, mutable default sharing, utcnow naive and deprecated, pandas != keeps missing, max of strings) match local runs. (verified: Local runs on CPython 3.13.12 and pandas 3.0.2, September 2026 (see D1, D3 and D5 claims).)

D7-L2 Review practice: PySpark and Spark SQL

Concept

Nine snippets of PySpark and Spark SQL, each with one seeded bug. Most come straight from your reference pipeline's known defects. The answer key stays hidden until you submit.

Use the same routine as lesson D7-L1: read the goal, walk the lines, ask what each line assumes. Spark adds its own questions:

- **Imports.** Is `sum`, `min`, `max`, `round` or `filter` imported by name from `pyspark.sql.functions`? Is `functions as F` used instead?
- **Filters.** Does a comparison with a nullable column drop nulls? Is a boolean compared with `== True`?
- **Grain.** Does every `groupBy` include each grain column the docstring names? Does every join keep the grain?
- **Aggregates.** Can a `sum(when(...))` return null? Is a division guarded?
- **Passes over the data.** Are there several `count()` calls on an uncached DataFrame, where one `agg` would do?
- **Writes.** Does a MERGE source hold one row per key? Is a rename silently skipped when the source column is missing?
- **SQL.** Does `NOT IN` meet a nullable subquery?

For each snippet, tap the line, then write the fix. The checker tests the fix statically: it looks for the pattern the standard requires. Your score counts bugs found and fixes proposed.

When you finish, compare your list of misses with the checklist above. The item you missed most is the one to add to your personal review routine.

Why it matters

Spark code fails quietly more often than Python code. A shadowed import fails loudly. A missing grain column, a null sum or a duplicate MERGE key can pass every run and still publish wrong numbers. Practice on the defects from your own pipeline trains your eye for the ones that will appear in its next version.

Worked example: weak code

```
1 from pyspark.sql.functions import col, sum
2
3 metrics = silver.filter(col("in_scope") ==
      True).groupBy("wave_id").agg(sum("final_weight"))
```

1. **Line 1.** Shadows Python's sum for the whole module.
2. **Line 3.** `== True` trips ruff E712. Filter on the column directly.

Worked example: strong code

```
1 from pyspark.sql import functions as F
2
```

```
❷ metrics =
      silver.filter(F.col("in_scope")).groupBy("wave_id").agg(F.sum("final_weight"))
```

1. **Line 1.** Python built-ins keep their meaning.
2. **Line 3.** A boolean column is already a condition.

For Spark code, check imports, null behaviour in filters and sums, grain in groupBy and joins, passes over data and MERGE keys.

Recap card

- **Spark review checklist** Imports, null behaviour in filters and sums, grain in groupBy, passes over data, MERGE keys, silent renames, NOT IN.
- **MERGE with repeated source keys** Fails when several source rows match one target row. Deduplicate the source on the merge keys first.
- **withColumnRenamed on a missing column** A silent no-op. Check df.columns and raise.

Practice

Exercise 1. Spot the bug (level 1)

Goal: a gold aggregation module that also uses Python's built-in `sum` in a helper. Find the bug and fix the import.

```
1 from pyspark.sql.functions import col, count, sum, when
2
3 def total_checks(results: list[dict[str, str]]) -> int:
4     """Number of passed checks."""
5     return sum(1 for r in results if r["status"] == "PASS")
```

Write down the line number of the bug. Then write the fix.

Exercise 2. Spot the bug (level 1)

Goal: keep in-scope respondents. Find the line that breaks the standard and fix it.

```
1 from pyspark.sql import functions as F
2
3 in_scope = silver.filter(F.col("in_scope") == True)
4 in_scope.write.mode("overwrite").saveAsTable("research_dev.nat_in_scope")
```

Write down the line number of the bug. Then write the fix.

Exercise 3. Spot the bug (level 2)

Goal: drop test panel respondents and keep everyone else, including respondents with no panel recorded. Find the bug and fix it.

```
1 from pyspark.sql import functions as F
```

```
2
3 live = respondents.filter(F.col("panel_type") != "test")
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Multiple choice (level 2)

Goal: gold shows 0 unweighted reach for a subgroup where nobody was reached. The reference code uses `sum(when(col("reach_flag") == 1, 1)).alias("unweighted_reach")`. What does gold show?

- a) NULL
- b) 0
- c) The subgroup's row count
- d) An error

Why?

- r1) when without otherwise is null for non-matching rows, and a sum of nulls is null.
- r2) Spark drops zero sums.

Exercise 5. Spot the bug (level 3)

Goal (from the docstring): reach metrics at tracker x wave x subgroup grain. Find the bug and fix it.

```
1 from pyspark.sql import functions as F
2
3 metrics = (
4     silver.filter(F.col("in_scope"))
5     .groupBy("wave_id", "age_group", "gender", "region")
6     .agg(F.sum("final_weight").alias("weighted_base"))
7 )
```

Write down the line number of the bug. Then write the fix.

Exercise 6. Spot the bug (level 3)

Goal: table health counts for the run log, computed efficiently. Find the line that shows the problem and rewrite the counts as one aggregation.

```
1 from pyspark.sql import functions as F
2
3 df = spark.table(table_name)
4 health = {
5     "row_count": df.count(),
6     "null_ids": df.filter(F.col("respondent_id").isNull()).count(),
7     "null_weights": df.filter(F.col("final_weight").isNull()).count(),
8 }
```

Write down the line number of the bug. Then write the fix.

Exercise 7. Spot the bug (level 4)

Goal: MERGE a redelivery into silver on respondent and wave. Redeliveries sometimes repeat a respondent. Find the bug and fix it.

```
from delta.tables import DeltaTable

target = DeltaTable.forName(spark, "research_silver.nat_respondents")
(target.alias("t")
    .merge(delivery.alias("s"), "t.respondent_id = s.respondent_id AND
   t.wave_id = s.wave_id")
    .whenMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute())
```

Write down the line number of the bug. Then write the fix.

Exercise 8. Predict the output (level 4)

Goal: list respondents outside the excluded regions. `excluded` holds `'N'` and one NULL. How many rows does this return from 1,000 respondents, 300 of them in N?

```
SELECT COUNT(*) FROM respondents
WHERE region_code NOT IN (SELECT code FROM excluded);
```

Exercise 9. Spot the bug (level 5)

Goal: rename delivery columns to canonical names. If a source column is missing, stop with an error naming it. Find the bug and fix it.

```
def rename_columns(bronze_df, column_map: dict[str, str]):
    """Rename source columns to canonical names."""
    for src, tgt in column_map.items():
        bronze_df = bronze_df.withColumnRenamed(src, tgt)
    return bronze_df
```

Write down the line number of the bug. Then write the fix.

Sources

- Snippet behaviours (shadowed sum NOT_COLUMN_OR_STR, != drops nulls, sum(when) null, NOT IN with NULL returns no rows, withColumnRenamed no-op, MERGE multiple-source-rows failure) match local runs. (verified: Local runs on PySpark 4.0.4 and delta-spark 4.0.1 (see D1, D3, D4 claims and practice/tests/test_a3_l1_merge.py), September 2026.)

D7-L3 Review practice: Databricks SQL and DAX

Concept

Eight snippets of Databricks SQL and DAX, each with one seeded bug. SQL and DAX look simple, which makes their bugs easy to read past. The answer key stays hidden until you submit.

What to look for in Databricks SQL:

- **Syntax from another platform.** `CREATE ROW ACCESS POLICY` and `CREATE MASKING POLICY` are Snowflake. Databricks uses a SQL UDF with `SET ROW FILTER` or `SET MASK`.
- **Grants to individuals.** Your standard grants to groups, never to one user's email.
- **Group checks.** `is_member()` checks workspace groups. Unity Catalog guidance favours `is_account_group_member()`.
- **Names.** Two-part names resolve against the current catalog. Use three-level names: `catalog.schema.table`.
- **Ratios.** An `AVG` of percentages weights every subgroup equally. Recompute ratios from their sums.
- **MERGE sources.** One row per key, or the statement fails.

What to look for in DAX:

- **Division.** `/` returns infinity or an error on a zero denominator. `DIVIDE` returns BLANK.
- **Filter context.** `ALL(fact_table)` removes every filter on the table, wave and region slicers included. Name the one dimension whose filter you mean to remove.

Review each snippet with the same routine: goal, then line by line. For SQL, also check each statement against the Databricks SQL reference in your head: does this statement exist here? The checker tests your fix against the pattern the standard requires. Your score counts bugs found and fixes proposed.

This lesson closes the practice module. The capstone review in Phase 6 uses the same skills on a whole pull request.

Why it matters

Security SQL and report measures carry outsized risk. A grant to one person survives their move to another team. A row filter written in the wrong dialect

leaves a table open. A DAX measure with the wrong filter context shows a plausible number to every viewer of a dashboard. Reviewing these few lines carefully protects many readers.

Worked example: weak code

```
(1) SELECT region, AVG(weighted_reach_pct) AS reach_pct
 2  FROM research_gold.nat_metrics
 3  GROUP BY region;
```

1. **Line 1.** An average of subgroup percentages. A subgroup of 12 counts as much as one of 1,200.

Worked example: strong code

```
 1  SELECT region,
(1)         SUM(weighted_reach) / NULLIF(SUM(weighted_base), 0) AS reach_pct
 3  FROM research_gold.nat_metrics
 4  GROUP BY region;
```

1. **Line 2.** Total over total, with the zero base guarded.

For SQL, check the dialect, grants, group functions, names and ratios. For DAX, check DIVIDE and the exact filter each ALL removes.

Recap card

- **SQL review checklist** Dialect, grants to groups, is_account_group_member, three-level names, ratios from sums, one source row per MERGE key.
- **DAX review checklist** DIVIDE for every ratio. ALL on the one dimension you mean, never the whole fact table.
- **Deduplicating a MERGE source in SQL** ROW_NUMBER() OVER (PARTITION BY keys ORDER BY ingestion_timestamp DESC), keep rn = 1.

Practice

Exercise 1. Spot the bug (level 1)

Goal: restrict `research_silver.nat_respondents` rows by region with a Databricks row filter. Find the bug and fix it.

```
 1  CREATE FUNCTION research_governance.region_filter(region STRING)
 2    RETURN is_account_group_member(concat('analysts_', region));
 3  CREATE ROW ACCESS POLICY region_policy ON research_silver.nat_respondents
      (region);
```

Write down the line number of the bug. Then write the fix.

Exercise 2. Spot the bug (level 1)

Goal: let the analysts read the gold metrics table, following the standard. Find the bug and fix it.

```
GRANT USE CATALOG ON CATALOG research TO `analysts`;
GRANT USE SCHEMA ON SCHEMA research.research_gold TO `analysts`;
GRANT SELECT ON TABLE research.research_gold.nat_metrics TO
    `ana.lopez@example.com`;
```

Write down the line number of the bug. Then write the fix.

Exercise 3. Spot the bug (level 2)

Goal: a column mask that shows postcodes only to members of the account-level group `analysts`, as Unity Catalog guidance recommends. Find the bug and fix it.

```
CREATE FUNCTION research_governance.mask_postcode(p STRING)
  RETURN CASE WHEN is_member('analysts') THEN p ELSE '***' END;
ALTER TABLE research_silver.nat_respondents ALTER COLUMN postcode SET
    MASK research_governance.mask_postcode;
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Multiple choice (level 2)

Goal: a scheduled query must read production gold. It runs `SELECT * FROM research_gold.nat_metrics`. The job's session has `research_dev` as its current catalog. What does it read?

- a) research_dev.research_gold.nat_metrics, if it exists
- b) research.research_gold.nat_metrics, because gold is production
- c) An error: two-part names are not allowed
- d) Both tables, unioned

Why?

- r1) A two-part name resolves against the current catalog. Only a three-level name is unambiguous.
- r2) Databricks always prefers dev catalogs.

Exercise 5. Spot the bug (level 3)

Goal: weighted reach percentage per region from the gold table, which has one row per subgroup with `weighted_reach` and `weighted_base`. Find the bug and fix it, guarding a zero base.

```
SELECT region,
       AVG(weighted_reach / weighted_base) AS reach_pct
FROM research.research_gold.nat_metrics
```

```
4 GROUP BY region;
```

Write down the line number of the bug. Then write the fix.

Exercise 6. Fill the gap (level 3)

Goal: a reach percentage measure that shows an empty cell, not infinity, where the base is zero. The assistant wrote `RETURN reach_w / base_w`. Complete the fix.

```
1 Reach % =
2 VAR reach_w = SUM ( fact_reach[weighted_reach] )
3 VAR base_w = SUM ( fact_reach[weighted_base] )
4 RETURN _____(1) ( reach_w, _____(2) )
```

Exercise 7. Spot the bug (level 4)

Goal: each brand's share of weighted reach across all brands, respecting the wave and region slicers. Find the bug and fix it.

```
1 Brand share =
2 VAR this_brand = SUM ( fact_reach[weighted_reach] )
3 VAR all_brands = CALCULATE ( SUM ( fact_reach[weighted_reach] ), ALL
     ( fact_reach ) )
4 RETURN DIVIDE ( this_brand, all_brands )
```

Write down the line number of the bug. Then write the fix.

Exercise 8. Spot the bug (level 5)

Goal: MERGE a redelivery into silver. The delivery can repeat a respondent within a wave. Keep the row with the latest `ingestion_timestamp`. Find the bug and fix it.

What you need to know.

`MERGE INTO target AS t USING source AS s ON <condition>` is the SQL form of the Delta merge from lesson A3-L1. Each target row is matched against the source rows that meet the `ON` condition:

- `WHEN MATCHED THEN UPDATE SET *` overwrites the matched target row with the source row's columns.
- `WHEN NOT MATCHED THEN INSERT *` inserts source rows with no match.

The `USING` part can be a table or a subquery in brackets. In Databricks SQL, `SELECT * EXCEPT (col)` returns every column except `col`.

```
1 MERGE INTO research.research_silver.nat_respondents AS t
2 USING research.research_bronze.nat_raw_w14 AS s
3 ON t.respondent_id = s.respondent_id AND t.wave_id = s.wave_id
4 WHEN MATCHED THEN UPDATE SET *
5 WHEN NOT MATCHED THEN INSERT *;
```

Write down the line number of the bug. Then write the fix.

Sources

- Databricks row filters and masks, is_account_group_member, three-level namespace and grants follow the docs verified in Track A. (verified: https://docs.databricks.com/aws/en/data-governance/unity-catalog/filters-and-masks/manually-apply and related pages (checked 2026-09-25))
- SELECT * EXCEPT (col) is supported in Databricks SQL. (verified: https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-syntax-qry-star (checked 2026-09-25))

D8 Prompting for correct code

Roadmap: Every month. State versions, constraints, expected outputs and tests.

D8-L1 State versions and constraints in every prompt

Concept

Most wrong code from an assistant answers a question you did not ask. It targets a version you do not run, a platform you do not use, or a standard you did not mention. Track D showed the results: `CREATE ROW ACCESS POLICY`, `datetime.utcnow()`, `applymap`. The cheapest fix happens before the code exists: a better prompt.

Name the versions. The assistant cannot see your environment. Say it:

```
Environment: Databricks Runtime 17.3 LTS (Spark 4.0, Python 3.12),
PySpark DataFrame API, Delta tables in Unity Catalog.
Local tests: pandas 3.0, pytest 9.
```

Name the platform and dialect. "SQL" means nothing. Write "Databricks SQL" or "Fabric Warehouse T-SQL". The same goes for "a notebook": Databricks or Fabric?

State the constraints your standard sets. Paste the few that apply, not the whole file:

- Import `functions as F`. Never import `sum`, `min` or `max` by name.
- Type hints and a Google-style docstring on every function.
- Bronze is append-only. Silver uses MERGE on `(respondent_id, wave_id)`.
- Three-level table names. No secrets in code.
- Test for None with `is None`.

State what not to do. "Do not use RDDs, Python UDFs or `collect()` on the full table." Negative constraints stop the common wrong turns.

Ask for a list of assumptions at the end of the answer (lesson D6-L3).

Keep a prompt header with these lines in your repository, next to `CLAUDE.md`, and paste it at the start of each coding conversation. A prompt that states versions and constraints does not guarantee correct code. It removes the mistakes that come from guessing, so your review can focus on the rest.

Why it matters

An engineer asked for "a PySpark function to cap survey weights". The answer used `pandas_udf` with a type-hint style from Spark 3.0 docs, imported `max` from `pyspark.sql.functions`, and wrote to a two-part table name. The same request with a four-line header naming Databricks Runtime 17.3 LTS, the import rule and three-level names produced code that passed review on the first read.

Worked example: weak code

```
❶ Write a PySpark function that caps survey weights at 5 and saves the
    result.
```

1. **Line 1.** No version, no platform, no standard, no table rules. The assistant guesses all of them.

Worked example: strong code

```
❶ Environment: Databricks Runtime 17.3 LTS (Spark 4.0, Python 3.12), Unity
    Catalog.
2 Standards: from pyspark.sql import functions as F; type hints and Google
    docstrings;
❷  no Python UDFs, RDDs or collect() on full tables; three-level table
    names.
4 Task: write cap_weights(df, cap: float) -> DataFrame that caps
    final_weight at cap.
5  Do not write to any table. Return the DataFrame.
❸ Finish by listing the assumptions your code makes about the input.
```

1. **Line 1.** Versions and platform, stated once.
2. **Line 3.** Negative constraints close the common wrong turns.
3. **Line 6.** Makes hidden assumptions reviewable.

Every coding prompt names versions, platform and dialect, the constraints from your standard, and what not to do.

Recap card

- **Four things every coding prompt states** Versions and platform, the dialect, the relevant standards, and what not to do.
- **Why name the SQL dialect** Syntax differs by platform. 'SQL' alone invites Snowflake or T-SQL statements.
- **Where to keep the prompt header** In the repository next to CLAUDE.md, updated when versions change.

Practice

Exercise 1. Multiple choice (level 1)

You ask for "SQL to add a row filter by region". Which line most improves the prompt?

- a) Target: Databricks SQL, Unity Catalog, Databricks Runtime 17.3 LTS
- b) Please write clean code
- c) Make it efficient
- d) Explain your answer in detail

Why?

- r1) Row filter syntax differs by platform. Naming the dialect rules out Snowflake's CREATE ROW ACCESS POLICY.
- r2) Longer prompts always give better answers.

Exercise 2. Put in order (level 2)

Order the parts of a coding prompt header.

- Environment: runtime, language and library versions
- What not to do: no UDFs, no RDDs, no secrets
- Standards that apply: imports, typing, table rules
- The task, with inputs, outputs and side effects
- Ask for a list of assumptions

Exercise 3. Fill the gap (level 2)

Complete the environment line for this course's Databricks lessons.

```
Environment: Databricks Runtime _____(1) LTS (Spark _____(2), Python
    3.12), Unity Catalog.
```

Exercise 4. Write the function (level 3)

Write `unstated_versions(prompt: str, libraries: list[str]) -> list[str]`. Return the libraries, in the order given, that the prompt does not mention followed by a version number, such as `pandas 3.0`, `PySpark 4.0` or `Python 3.12`. Match library names case-insensitively. A version is digits and dots after optional whitespace and an optional `v`. A library mentioned without a version counts as unstated.

What you need to know.

`re.escape(text)` adds backslashes before characters that have a meaning in patterns, such as `.` or `+`, so the text matches literally. `re.search(pattern,`

`text, flags=re.IGNORECASE)` looks for a match anywhere in the text, ignoring case, and returns `None` when there is none.

```
import re

name = re.escape("C++")   # 'C\+\+'
re.search(name + r"\s*\d+", "I use c++ 20", flags=re.IGNORECASE)   # a
    match
```

An `rf"..."` string combines a raw string (backslashes stay as typed) with an f-string (`{...}` inserts a value).

Your code must pass these checks:

- Python has a version (case-insensitive). pandas appears without one. PySpark is absent.
- Allow an optional v before the version.
- pandas-stubs 2.2 is not a pandas version. Match the library name as a whole word.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Libraries a prompt names without a version number.
def unstated_versions(
    prompt: str,
    libraries: list[str],
) -> list[str]: ...
```

Starter code:

```
def unstated_versions(prompt, libraries):
    return [lib for lib in libraries if lib not in prompt]
```

Exercise 5. Architecture design (level 4)

Build the prompt header your team pastes into every coding conversation for the survey pipeline.

Options: Databricks Runtime 17.3 LTS, Spark 4.0, Python 3.12, pandas 3.0 for local tests, Use the latest versions of everything.

1. Versions line
2. Standards
3. Where the header lives

Sources

- Databricks Runtime 17.3 LTS ships Spark 4.0.0 and Python 3.12.3; Runtime 18 is not yet LTS. (verified: https://docs.databricks.com/aws/en/release-notes/runtime/17.3lts and /18 (checked 2026-09-25))

D8-L2 State expected outputs and tests in the prompt

Concept

"Write a function that computes weighted reach" leaves the assistant to decide what correct means. It will decide something plausible, and you will find out later whether it matches your meaning. Put your meaning in the prompt instead: examples with expected outputs, the edge cases, and the tests the code must pass.

Give a small input and the exact output. A table of five rows and the result you worked out by hand removes most ambiguity at once:

```
Input (respondent_id, reach_flag, final_weight):
  R1, 1, 2.0   R2, 0, 1.0   R3, 1, 1.0   R4, null, 1.0
Expected: weighted_reach = 3.0, weighted_base = 4.0, pct = 0.75
R4 has no reach answer: exclude it from the base.
```

List the edge cases and their answers. Empty input returns None. A zero base returns None. A share of 0.0 is valid. A duplicate key raises ValueError. Each line is a decision the assistant would otherwise make for you.

Paste the tests, or ask for them in a fixed form. Two options:

- Paste your own failing tests (lesson D6-L2) and ask for code that makes them pass without changing them.
- Ask for tests first, "using only hand-worked expected values, not the function's formula", review them, then ask for the code.

State the output format. Return type, column names and order, rounding, and whether the function writes anywhere. "Return a DataFrame with columns wave_id, weighted_reach_pct. Do not write to a table."

A prompt with examples and tests turns review from "does this look right?" into "do the tests pass, and are they the tests I wrote?" The second question takes seconds.

Why it matters

Asked for "weighted reach by wave", an assistant included respondents with a null reach answer in the base. Another conversation, given the same task with a five-row example that excluded the null, produced code that matched on the first try. The example took two minutes to write and settled a question that had caused a four-point difference in the dashboard.

Worked example: weak code

```
(1) Write a function to compute the weighted reach percentage by wave.
```

1. **Line 1.** Nulls? Empty waves? Output shape? The assistant decides all of it.

Worked example: strong code

```
1  Task: weighted_reach_pct(rows) -> float | None.
2  rows: list of (reach_flag: int | None, final_weight: float).
(1) Example: [(1, 2.0), (0, 1.0), (1, 1.0), (None, 1.0)] -> 0.75
4    (the None row is excluded from the base).
5  Edge cases: [] -> None; all reach_flag None -> None; [(0, 1.0)] -> 0.0.
(2) Make these tests pass without editing them:
7    assert weighted_reach_pct([(1, 2.0), (0, 1.0), (1, 1.0), (None, 1.0)])
     == 0.75
8    assert weighted_reach_pct([]) is None
9    assert weighted_reach_pct([(0, 1.0)]) == 0.0
```

1. **Line 3.** A worked example settles how nulls are treated.
2. **Line 6.** The acceptance criteria, stated before any code exists.

Give a worked example, the edge cases with answers, the output format and the tests. Then review against them.

Recap card

- **Four things that specify correct code in a prompt** A worked example, edge cases with answers, the output format, and fixed tests.
- **Instruction that protects your tests** 'Make these tests pass without editing them. If a test seems wrong, say so.'
- **Why work the example out yourself** If you cannot compute the answer by hand, the rule is not clear enough to ask for code.

Practice

Exercise 1. Multiple choice (level 1)

Which addition to a prompt best settles how null reach answers are treated?

- a) A five-row example with a null and the expected result worked by hand
- b) 'Handle nulls properly'
- c) 'Be careful with edge cases'
- d) 'Use best practices'

Why?

- r1) An example states the rule precisely. Words like 'properly' leave the rule to the assistant.

- r2) Assistants ignore instructions written in words.

Exercise 2. Predict the output (level 2)

Work out the expected value for your prompt's example. Rows are `(reach_flag, final_weight)`, and rows with a `None` flag are excluded from the base. What does this print?

```
rows = [(1, 2.0), (0, 1.0), (1, 1.0), (None, 1.0)]
valid = [(f, w) for f, w in rows if f is not None]
print(sum(w for f, w in valid if f == 1) / sum(w for _, w in valid))
```

Exercise 3. Fill the gap (level 2)

Complete the instruction that stops an assistant from weakening your tests.

```
Make these tests pass _____(1) editing them. If a test seems wrong,
    _____(2) instead of changing it.
```

Exercise 4. Write the function (level 3)

Write `examples_to_asserts(func_name: str, examples: list[tuple[tuple[object, ...], object]]) -> list[str]` that turns worked examples into assert lines for a prompt. Each example is `(args, expected)`. Produce `assert func(arg1, arg2) == expected` using `repr` for every argument and value, except that an expected `None` produces `assert func(...) is None`.

Your code must pass these checks:

- Join args with ', ' using repr, and use `is None` for a None result.
- repr keeps strings quoted, so the line is valid Python.
- The generated lines must run as Python and pass for a correct function.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Turn worked examples into assert lines for a prompt or a test file.
def examples_to_asserts(
    func_name: str,
    examples: list[tuple[tuple[object, ...], object]],
) -> list[str]: ...
```

Starter code:

```
def examples_to_asserts(func_name, examples):
    return [f"assert {func_name}({args}) == {expected}" for args,
    expected in examples]
```

Exercise 5. Architecture design (level 4)

Plan the prompt for an assistant to write a silver deduplication function.

Options: Six input rows with two duplicates and the expected four output rows, No example: the assistant knows what deduplication means.

1. Example in the prompt
2. Rule for duplicates
3. Tests

Sources

- Generated assert lines using repr and 'is None' execute as valid Python. (verified: Checked by the exercise tests in this repo, September 2026.)

D8-L3 Iterate on a failing answer

Concept

The first answer often fails a test. What you send next decides whether the second answer is better or just different.

Send the evidence, not a feeling. "It doesn't work" invites a rewrite from scratch. Send:

- the **exact failing test** and its assertion,
- the **full error**, or the actual versus expected values,
- the **smallest input** that shows the problem,
- what you already checked.

```
test_zero_share fails:
  assert compare_waves({"a": 0.0}, {"a": 0.12}) == ["a"]
  got [] instead of ["a"]
Line 6 uses `if current_pct and prior_pct:`. I think 0.0 is treated as
   missing.
Fix only this. Do not change the tests or other behaviour.
```

One problem per round. Ask for a fix to one failure at a time, and say what must not change. Broad requests ("fix all the issues and improve it") produce broad rewrites, which reintroduce bugs you already fixed.

Watch the tests, not only the code. A common way for an assistant to "fix" a failing test is to change the test: loosen an assertion, delete a case, or replace a hand-worked value with a computed one. Diff the test file after every answer. Any removed or changed assertion needs your approval.

Know when to stop. If two focused rounds have not fixed a problem, the assistant is guessing. Stop, read the code yourself with the debugger (lesson D2-L1), and write the fix. Then you can explain it in the next prompt, or leave the assistant out of it.

Start fresh when the thread goes wrong. A long conversation full of failed attempts carries those attempts forward. A new conversation with the header from lesson D8-L1, the current code, and the one failing test often does better than round seven.

Why it matters

After four rounds of "still failing, please fix", an assistant's version of a stability check passed every test. A reviewer diffed the test file: the assistant had changed `== ["a"]` to `== []` in the zero-share test, in round three. The real bug, `if current_pct and prior_pct:`, was still there. One focused prompt with the failing assertion fixed it in a single round.

Worked example: weak code

```
❶ Still not working. Please fix everything and make it better.
```

1. **Line 1.** No test, no error, no scope. The assistant rewrites everything, including what already worked.

Worked example: strong code

```
❶ test_missing_metric fails with KeyError: 'new' at line 7, `prior_pct =
    prior[metric]`.
  2 Input: compare_waves({"new": 0.4}, {}). Expected: [].
  3 Fix only the lookup so a metric missing from prior is skipped.
❷ Do not edit the tests. Keep every other behaviour the same.
```

1. **Line 1.** The exact test, error and line.
2. **Line 4.** Scope limits stop regressions and test tampering.

Send the failing test, the error and the smallest input. Fix one thing per round, diff the tests, and stop after two failed rounds.

Recap card

- **What to send when AI code fails a test** The failing assertion, actual vs expected, the smallest input, what you checked, and 'fix only this'.
- **The test-tampering check** Diff the test file after every answer. Removed or changed assertions need your approval.
- **When to stop iterating** After two focused rounds without progress: debug it yourself, or restart in a fresh conversation.

Practice

Exercise 1. Multiple choice (level 1)

An assistant's function fails one test. What do you send next?

- a) The failing test, the actual and expected values, and 'fix only this'
- b) 'It doesn't work, try again'
- c) 'Rewrite it more carefully'
- d) The whole project so it has more context

Why?

- r1) Precise evidence and a narrow scope lead to a targeted fix instead of a rewrite.
- r2) Assistants only respond to test output.

Exercise 2. Spot the bug (level 2)

This follow-up prompt has one line that invites a rewrite instead of a fix. Tap it, then rewrite the prompt so it limits the scope.

```
1 test_zero_share fails: expected ["a"], got [].
2 Input: compare_waves({"a": 0.0}, {"a": 0.12}).
3 While you are at it, refactor the whole module and improve the naming.
```

Write down the line number of the bug. Then write the fix.

Exercise 3. Put in order (level 2)

Order your steps after an assistant's code fails a test.

- If two rounds fail, debug and fix it yourself
- Diff both the code and the test file in the answer
- Collect the failing assertion, the error and the smallest input
- Run the full test suite
- Send them with 'fix only this, do not edit the tests'

Exercise 4. Write the function (level 3)

Write `removed_asserts(before: str, after: str) -> list[str]`. Given two versions of a test file, return the `assert` statements present in `before` but missing from `after`, compared after stripping whitespace at both ends, in the order they appear in `before`. An assert that was changed counts as removed. Only lines that start with `assert` after stripping count.

Your code must pass these checks:

- A changed assertion counts as removed. Compare stripped lines.

- Report deleted asserts, and ignore non-assert lines.
- Reordering or re-indenting an assert is not a removal.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Assert statements in the old test file that the new one lacks.
def removed_asserts(before: str, after: str) -> list[str]: ...
```

Starter code:

```
def removed_asserts(before, after):
    return []
```

Exercise 5. Architecture design (level 4)

Round three of a conversation still fails the same test, and the thread holds four earlier attempts. Decide the next steps.

Options: Debug it yourself with pdb on the failing test, Send 'still failing' again.

1. Next move
2. If you use the assistant again
3. Before accepting any answer

Sources

- The stop-after-two-rounds rule is this course's working guideline, not a published standard. (verified: Course guideline stated as such in the lesson text.)

Track D best-practice checklist

☐ Read a traceback from the bottom up. Find the first frame in your own code.

☐ Reproduce the error on the smallest data that shows it.

☐ Check row counts before and after every join.

☐ Test numbers with `is None`, never with truthiness, when zero is a valid value.

☐ Never use a mutable object as a default argument.

☐ Check every function and parameter an AI suggests against the official docs for your version.

☐ Run AI code on a small input with a known answer before you trust it.

☐ Write the failing test first. A test that only repeats the code proves nothing.

☐ Ask the model to list its assumptions. Check each one.

☐ State versions, constraints, expected outputs and tests in every prompt.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

D0-L1 Bugs, errors and failures, the words and the method

D0-L1 exercise 1

Hints. (1) Did anything stop the job? (2) Is the result right? (3) It ran and gave the wrong number.

Answer. c) A wrong result, a silent bug Because: r1) The code ran to the end and produced a wrong number, with no error to point at it.

Code that runs and reports the wrong number is a silent bug, the most expensive kind, because nothing tells you to look.

Common wrong answers.

- `a`: A syntax error stops the code before it runs. This job finished.
- `b`: An exception stops the run with a traceback. There was none.
- `d`: A wrong total is a failure, even when nothing crashed.
- `c|r2`: Right kind, wrong reason. Silent bugs come from code as often as from data.

D0-L1 exercise 2

Hints. (1) You cannot study what you cannot make happen. (2) Evidence, then one idea, then a check of that idea. (3) The test must fail before the fix, so it comes first.

Answer.

1. Reproduce the failure and shrink the input to the smallest one that still fails.
2. Read the evidence: the last traceback line or the first wrong value.
3. Form one hypothesis that explains everything you see.
4. Test the hypothesis with one small check.
5. Write a test that fails because of the bug.
6. Fix the root cause and watch the test pass.

Reproduce, read, hypothesise and check. Then write a failing test, and fix the cause until it passes.

Common wrong answers.

- Reproduce first. Evidence you cannot reproduce cannot confirm a fix.
- Test the hypothesis before you change code, or you may fix the wrong thing.
- Write the test first. A test written after the fix never proved it could fail.

D0-L1 exercise 3

Hints. (1) What happens when you index a dict with a key it does not have? (2) The except block catches it and prints the error. (3) The error shows the key as Python writes it.

Answer.

```
missing key: 'R2'
```

`weights["R2"]` raises `KeyError`, and the error prints as the key's repr, `'R2'`, which tells you exactly which key was missing.

Common wrong answers.

- `missing key: R2`: A KeyError's message is the key's repr, so a string key prints in quotes.
- `None`: Indexing a dict with a missing key raises KeyError. Only `.get` returns None.

D0-L1 exercise 4

Hints. (1) Check `fails(rows)` first, then work on `current = list(rows)`. (2) `candidate = current[:i] + current[i + 1:]` is the list without element i. (3) Loop while the last pass removed something. After each successful removal, break and start again from the left.

Answer.

```python
from collections.abc import Callable


def minimise(rows: list[int], fails: Callable[[list[int]], bool]) -> list[int]:
    """Shrink a failing input by removing one element at a time.

    Args:
        rows: An input that makes fails return True.
        fails: Returns True when the bug shows on the given input.

    Returns:
        A smaller list that still fails, where no single removal still fails.

```

```
    Raises:
        ValueError: If the full input does not fail.
    """
    if not fails(rows):
        raise ValueError("the input does not fail, so there is nothing to shrink")
    current = list(rows)
    shrunk = True
    while shrunk:
        shrunk = False
        for i in range(len(current)):
            candidate = current[:i] + current[i + 1 :]
            if fails(candidate):
                current = candidate
                shrunk = True
                break
    return current
```

Repeated single removals keep the failing behaviour while the input shrinks. A small failing input makes the evidence easy to read and the test easy to write.

D0-L1 exercise 5

Hints. (1) What happens when `float("n/a")` runs? (2) Where does that error go? (3) The catch-all and the default it returns.

Answer. The bug is on line 5, 6.

Fixed version:

```
def load_total(lines: list[str]) -> float:
    """Total weight from the lines of a delivery file."""
    return sum(float(line) for line in lines)
```

`except Exception` swallows the ValueError, and `return 0.0` replaces it with a believable number. Remove both, so a corrupted file stops the run with a message naming the bad text.

Common wrong answers.

- Line 4: Summing the parsed lines is correct. The problem is what happens when parsing fails.
- Line 6: Returning 0.0 is half of it. Which line catches the error in the first place?
- Line 5: Catching everything is half of it. Which line hides the failure behind a plausible number?

D0-L2 Read code you did not write

D0-L2 exercise 1

Hints. (1) What do you compare the code against? (2) What the function promises. (3) Name, signature, docstring.

Answer. a) Its name, signature and docstring: what it promises. Because: r1) You cannot judge whether code is right until you know what it is meant to do.

The contract defines what "correct" means. Every later step compares the code with it.

Common wrong answers.

- `b`: Without the contract, you cannot tell a bug from intended behaviour.
- `c`: Running unread code on production data can change or delete real data.
- `d`: The return line makes sense only once you know what the function promises.
- `a|r2`: Right first step, wrong reason. It comes first because it defines what right means.

D0-L2 exercise 2

Hints. (1) Keep a table with v, best and out after each step. (2) Step 1: v 3, best 3, out [3]. Step 2: v 1, best stays 3. (3) out grows by one value on every step.

Answer.

```
1 [3, 3, 4, 4, 5]
```

`best` changes only when a larger value arrives, and `out` records it on every step: 3, 3, 4, 4, 5.

Common wrong answers.

- `[3, 4, 5]`: `out.append(best)` runs on every step, not only when best changes, so the list has five values.
- `[3, 1, 4, 1, 5]`: Each step appends the best value so far, not the current value.
- `5`: The function returns the whole list `out`, not only the final maximum.

D0-L2 exercise 3

Hints. (1) Start from total = 0. (2) Step 1: 0 x 2 + 2. (3) Step 2: 2 x 2 + 5. Step 3: 9 x 2 + 1.

Answer.

```
step 1: v = 2, total = 2
step 2: v = 5, total = 9
step 3: v = 1, total = 19
```

Each step doubles the previous total and adds v: 2, then 9, then 19.

Common wrong answers.

- g1 = `2`; g2 = `7`; g3 = `8`: That is a plain running sum. Each step doubles the old total before adding v.
- g1 = `4`; g2 = `13`; g3 = `27`: The doubling applies to the old total, not to v: 0 x 2 + 2 is 2.

D0-L2 exercise 4

Hints. (1) `for num, den in zip(numerators, denominators, strict=True)` pairs the values. (2) Keep a pair when `num is not None and den > 0`. (3) Put it in one list comprehension and return it.

Answer.

```python
def ratios(numerators: list[float | None], denominators: list[float]) -> list[float]:
    """Ratio of each pair, skipping missing numerators and non-positive denominators.

    Args:
        numerators: Values to divide, None when missing.
        denominators: Values to divide by.

    Returns:
        One ratio per usable pair, in order.

    Raises:
        ValueError: If the lists differ in length.
    """
    return [
        num / den
        for num, den in zip(numerators, denominators, strict=True)
        if num is not None and den > 0
    ]
```

The restated function says what it does in its name, types and docstring, and `strict=True` turns a silent assumption into an error.

D0-L2 exercise 5

Hints. (1) Trace it: which index does the loop look at after removing -1? (2) Removing an element shifts the rest left, so the loop skips the next one. (3) Build a new list with a comprehension instead.

Answer. The bug is on line 5.

Fixed version:

```
1 def drop_negative(weights: list[float]) -> list[float]:
2     """Return the weights without negative values."""
3     return [w for w in weights if w >= 0]
```

`remove` shifts the remaining elements left while the loop moves on, so -2 is never checked. It also changes the caller's list, a side effect the name does not promise. A comprehension fixes both.

Common wrong answers.

- Line 3: Looping over the list is fine on its own. The trouble starts when the loop body changes the same list.
- Line 4: The condition is right: -1 and -2 are negative.

D1-L1 Read a Python traceback from the bottom up

D1-L1 exercise 1

Hints. (1) Read the header: 'most recent call last'. (2) Where is the most recent call? (3) At the bottom.

Answer. a) The last line: exception type and message Because: r1) Frames run oldest first. The exception and the frame that raised it are at the bottom.

The bottom holds what went wrong. The frames above say where and how you got there.

Common wrong answers.

- `b`: The first frame is the oldest call, often your entry point or a scheduler. The error happened at the other end.
- `c`: Length says nothing about relevance. Start at the bottom.
- `d`: Library frames rarely hold the cause. Find the last frame in your own code.
- `a|r2`: Right answer, wrong reason. The order is chronological: 'most recent call last'.

D1-L1 exercise 2

Hints. (1) Read the last line first. (2) Which frame is closest to the error and inside /repo/pipelines/? (3) The frame for qa_statistical.py, line 19.

Answer. The bug is on line 4.

`stats["mean"]` is None when the DataFrame is empty, and `round(None, 4)` raises TypeError. The frame for `qa_statistical.py` line 19 is where you add a guard.

Common wrong answers.

- Line 2: That is the entry point that called the function. The error happened deeper.
- Line 7: That line says what went wrong: round() received None. The frame above it says where.

D1-L1 exercise 3

Hints. (1) Which sentence does `from err` print? (2) This one is the other sentence. (3) The handler failed.

Answer. a) The except block that handled the KeyError has its own bug Because: r1) The second exception was raised while handling the first, without an explicit chain. Both need a fix.

"During handling" means your error handler raised. "Direct cause" means someone chained on purpose.

Common wrong answers.

- `b`: `from err` prints 'The above exception was the direct cause of the following exception'.
- `c`: The KeyError started it. Fix the handler and the missing key.
- `d`: Python does not retry. It records that the handler failed.
- `a|r2`: Right answer, wrong reason. The two errors are unrelated types. The handler raised a new one.

D1-L1 exercise 4

Hints. (1) Split on 'Traceback (most recent call last):' and keep the last part. (2) Match frame lines with a regular expression. (3) Keep overwriting the result, so the last match wins.

Answer.

```
1 import re
2
3 FRAME_PATTERN = re.compile(r'^\s*File "(?P<path>[^"]+)", line (?
    P<line>\d+)')
4
5
6 def first_own_frame(traceback_text: str, project_dir: str) -> tuple[str,
    int] | None:
7     """Find the frame to start from when reading a traceback bottom up.
```

```

    Args:
        traceback_text: Full traceback text, possibly chained.
        project_dir: Path prefix of your own code.

    Returns:
        (path, line) of the last frame inside project_dir, or None.
    """
    last_block = traceback_text.split("Traceback (most recent call last):")[-1]
    found: tuple[str, int] | None = None
    for line in last_block.splitlines():
        match = FRAME_PATTERN.match(line)
        if match and match["path"].startswith(project_dir):
            found = (match["path"], int(match["line"]))
    return found
```

Reading bottom up is the same as keeping the last matching frame. Library frames never match your prefix.

D1-L1 exercise 5

Hints. (1) Catch `KeyError as err`, nothing wider. (2) `err.args[0]` is the missing key. (3) End the raise with `from err`.

Answer.

```
def row_count_bounds(contract: dict[str, object]) -> tuple[object, object]:
    """Row count bounds from a data contract.

    Args:
        contract: Parsed contract with contract_id, row_count_min and row_count_max.

    Returns:
        (row_count_min, row_count_max).

    Raises:
        ValueError: If a bound is missing.
    """
    try:
        return contract["row_count_min"], contract["row_count_max"]
    except KeyError as err:
        contract_id = contract.get("contract_id", "<unknown>")
        raise ValueError(f"contract {contract_id} is missing key {err.args[0]}") from err
```

A narrow except, a message with the facts, and an explicit chain. The next reader sees both tracebacks joined by "direct cause".

D1-L2 Spark errors, the condition and the real cause

D1-L2 exercise 1

Hints. (1) What does SQLSTATE class 42 mean? (2) Compare the name in the message with the suggestions. (3) A typo.

Answer. a) The column name in the code: final_wieght is a typo for final_weight Because: r1) Class 42 errors mean the code names something that does not exist. The message suggests the right name.

Unresolved names are code errors. Read the suggestion list, then fix the name.

Common wrong answers.

- `b`: Missing values are nulls, not an unresolved column. The name itself is wrong.
- `c`: The error is in the query plan. A restart runs the same plan.
- `d`: A permission error has a different condition. This one says the name cannot be resolved.
- `a|r2`: Right answer, wrong reason. The suggestions are the closest names, not always the right one. Check them.

D1-L2 exercise 2

Hints. (1) Which sum is being called? (2) Look at the module's imports. (3) Import functions as F.

Answer. a) `sum` is PySpark's sum, imported by name, which shadows Python's built-in Because: r1) `from pyspark.sql.functions import sum` replaces the built-in in that module, so the generator goes to Spark's sum.

Import `functions as F`. Python's `sum`, `min`, `max`, `round`, `abs` and `filter` then keep their meaning.

Common wrong answers.

- `b`: self.checks is a list of CheckResult objects. The function receiving it is the problem.
- `c`: Generators work anywhere. Look at the imports.
- `d`: No column is involved. The code never meant to call Spark.
- `a|r2`: Right answer, wrong reason. PySpark does not disable anything. The import replaced the name.

D1-L2 exercise 3

Hints. (1) Actions trigger the work. Where was the failing expression written? (2) The message names the origin: cast. (3) try_cast returns null instead of failing.

Answer. The bug is on line 3.

Fixed version:

```
1 from pyspark.sql import functions as F
2
3 silver = bronze.withColumn("age", F.col("age").try_cast("int"))
4 silver = silver.withColumn("wave_id", F.lit(wave_id))
5 silver.write.mode("append").saveAsTable("research_silver.nat_respondents")
```

The write triggers the plan, so the error surfaces there. The origin line points at the cast. Use `try_cast`, then count the nulls it creates.

Common wrong answers.

- Line 5: The error appears at the write because that is the action. The origin line says cast, on line 3.
- Line 4: Adding a literal cannot fail on 'n/a'.

D1-L2 exercise 4

Hints. (1) Match the condition with `^\s*\[([A-Z0-9_.]+)\]`. (2) Search for `SQLSTATE:\s*([0-9A-Z]{5})`. (3) The class is the first two characters of the SQLSTATE.

Answer.

```
1  import re
2
3  CONDITION_PATTERN = re.compile(r"^\s*\[([A-Z0-9_.]+)\]")
4  SQLSTATE_PATTERN = re.compile(r"SQLSTATE:\s*([0-9A-Z]{5})")
5  CLASS_LABELS = {"42": "code", "22": "data"}
6
7
8  def triage(message: str) -> tuple[str | None, str]:
9      """Error condition and a first-pass category for a Spark error
       message.
10
11     Args:
12         message: The error message text.
13
14     Returns:
15         (condition or None, "code", "data" or "unknown").
16     """
17     condition_match = CONDITION_PATTERN.match(message)
18     condition = condition_match.group(1) if condition_match else None
19     state_match = SQLSTATE_PATTERN.search(message)
```

```
    if state_match is None:
        return condition, "unknown"
    return condition, CLASS_LABELS.get(state_match.group(1)[:2],
"unknown")
```

A first-pass triage sends class 42 to the code and class 22 to the data. Parse the structured parts, never loose substrings.

D1-L2 exercise 5

Hints. (1) Which part of the message is structured? (2) What did ANSI mode expose? (3) What stays stable across releases?

Answer.

- The condition and SQLSTATE give the category in seconds. The origin line gives the place.
- Class 22 means real bad data. Switching ANSI off hides it again.
- Conditions are stable identifiers. Message wording changes between releases.

Condition, SQLSTATE class, origin line. Then fix code or investigate data, depending on the class.

Common wrong answers.

- first = `java`: The JVM stack is Spark's internals. The condition and origin line point at your code.
- class22 = `ansi_off`: That returns to silent nulls. The bad values are still there, uncounted.
- catch = `text`: Message text changes between versions. Conditions do not.

D1-L3 Py4J and Spark Connect errors

D1-L3 exercise 1

Hints. (1) Wrapped exceptions nest. (2) Look for 'Caused by:'. (3) The last one.

Answer. a) After the last 'Caused by:', in the Python traceback it contains Because: r1) The JVM wraps the Python worker's exception. The innermost cause holds the Python traceback.

Search the message for the last "Caused by:" and read its Python traceback bottom up.

Common wrong answers.

- `b`: Java frames show Spark's internals, not your code.

- `c`: That line names the JVM method Python called, such as collectAndServe.
- `d`: The Python traceback is inside the message, after 'Caused by: org.apache.spark.api.python.PythonException'.
- `a|r2`: Right answer, wrong reason. The cause comes last, nested inside the outer exception.

D1-L3 exercise 2

Hints. (1) When does Spark Connect resolve column names? (2) At execution, or when you read the schema. (3) No action runs here.

Answer.

```
1 select ok
```

Spark Connect builds the plan without resolving names, so the `select` succeeds. `out.count()` or `out.columns` would raise.

Common wrong answers.

- `caught`: That is Spark Classic, which analyses eagerly. Spark Connect defers analysis to the action.

D1-L3 exercise 3

Hints. (1) Which package is shared by both modes? (2) pyspark.errors. (3) AnalysisException.

Answer. a) except pyspark.errors.AnalysisException Because: r1) Both modes raise a subclass of pyspark.errors.AnalysisException.

Catch the base classes in `pyspark.errors`. Your code then works in both modes.

Common wrong answers.

- `b`: Py4J exists only in Spark Classic, and PySpark converts most errors before you see them.
- `c`: That class exists only under Spark Connect.
- `d`: Column errors are not Python KeyErrors.
- `a|r2`: Right answer, wrong reason. It is a pyspark.errors class, with Classic and Connect subclasses.

D1-L3 exercise 4

Hints. (1) `message.rsplit(MARKER, 1)[1]` gives the text after the last marker. (2) Skip the rest of the marker line, then indented lines. (3) Java frames start with a tab, which is whitespace.

Answer.

```python
MARKER = "Caused by: org.apache.spark.api.python.PythonException:"


def python_cause(message: str) -> str | None:
    """Python exception line inside the last PythonException of a Py4J error.

    Args:
        message: Full error text.

    Returns:
        The exception line, such as "ZeroDivisionError: division by zero", or None.
    """
    if MARKER not in message:
        return None
    section = message.rsplit(MARKER, 1)[1]
    for line in section.splitlines()[1:]:
        if not line.strip() or line[0].isspace():
            continue
        if line.startswith(("Traceback", "File")):
            continue
        return line
    return None
```

The innermost cause is last. `rsplit` with a limit of 1 finds it directly.

D1-L3 exercise 5

Hints. (1) When does Connect resolve names? (2) Which package exists in both modes? (3) What does serverless support?

Answer.

- Spark Connect defers analysis, so a try around a select catches nothing. An explicit schema check works in both modes.
- pyspark.errors classes exist in both modes. Py4J exists only in Classic.
- Serverless supports only Spark Connect APIs, not RDDs.

Explicit schema checks, pyspark.errors classes, and the DataFrame API only.

Common wrong answers.

- optional = `try_select`: Under Spark Connect the select succeeds and the error moves to the action.
- except_class = `py4j`: Serverless has no Py4J.
- rdd = `keep`: Spark RDD APIs are not supported on serverless compute.

D2-L1 breakpoint() and pdb, stop and look

D2-L1 exercise 1

Hints. (1) The variable controls what breakpoint() calls. (2) 0 means disabled. (3) The job continues.

Answer. a) Nothing: the call does nothing and the job continues Because: r1) PYTHONBREAKPOINT=0 makes breakpoint() a no-op.

Set PYTHONBREAKPOINT=0 in job environments as a safety net, and still remove breakpoints before committing.

Common wrong answers.

- `b`: That is what happens without the variable. With 0, the call is ignored.
- `c`: No error. The call returns immediately.
- `d`: pdb never starts.
- `a|r2`: Right answer, wrong reason. Without the variable, a scheduled job would stop at the breakpoint.

D2-L1 exercise 2

Hints. (1) Print is one letter. (2) Up is one letter. (3) `p` and `u`.

Answer.

```
(Pdb) p reach, base        # print the inputs
(Pdb) u                    # move to the caller's frame
(Pdb) p rows
```

`p` shows values. `u` moves to the caller, where the bad value came from.

Common wrong answers.

- g2 = `d`: `d` moves down, towards the newest frame. You are already there. Use `u`.

D2-L1 exercise 3

Hints. (1) What makes each run fast? (2) Find the first wrong value. (3) Clean up last.

Answer.

1. Reproduce with the smallest input that gives the wrong number
2. Stop in the function with breakpoint() or post-mortem pdb
3. Print the inputs, walk up with u until a value is first wrong
4. Fix that frame and add a test with the small input
5. Remove the breakpoint and run the tests

Small reproduction, stop, walk up to the first wrong value, fix with a test, clean up.

Common wrong answers.

- Reproduce first, with small data, so each run is fast and you know the right answer.
- Fix and add the test while you know the cause, then clean up.

D2-L1 exercise 4

Hints. (1) `err.__traceback__` is the outermost traceback entry. (2) Follow `tb_next` until it is None. (3) `tb.tb_frame.f_locals` holds that frame's variables.

Answer.

```python
from collections.abc import Callable


def crash_locals(fn: Callable[..., object], *args: object) -> dict[str, object] | None:
    """Local variables of the frame that raised, as pdb post-mortem shows them.

    Args:
        fn: Function to call.
        *args: Arguments for fn.

    Returns:
        A copy of the innermost frame's locals, or None if fn did not raise.
    """
    try:
        fn(*args)
    except Exception as err:
        tb = err.__traceback__
        while tb is not None and tb.tb_next is not None:
            tb = tb.tb_next
        return dict(tb.tb_frame.f_locals) if tb is not None else None
    return None
```

A traceback is a linked list from your call down to the raise. pdb post-mortem opens at its last entry.

D2-L1 exercise 5

Hints. (1) Which line would stop an unattended job? (2) A debugging call. (3) Line 4.

Answer. The bug is on line 4.

Fixed version:

```
1 def unweighted_base(rows: list[dict[str, object]]) -> int:
2     """Count in-scope respondents."""
3     in_scope = [r for r in rows if r["in_scope"] is True]
4     return len(in_scope)
```

A leftover `breakpoint()` blocks a job waiting for input that never comes. ruff rule T100 flags it.

Common wrong answers.

- Line 3: `is True` is correct for a Python bool flag in a dict. The problem is the next line.

D2-L2 Logging levels that help at 3 a.m.

D2-L2 exercise 1

Hints. (1) What is the root logger's default level? (2) WARNING. (3) Only the warning appears, as bare text.

Answer.

```
1 weight sum low
```

With no configuration, records at WARNING and above go to stderr as the bare message. INFO is dropped.

Common wrong answers.

- `rows written weight sum low`: The default effective level is WARNING, so the INFO record is dropped.
- `WARNING:qa:weight sum low`: That is basicConfig's default format. Without configuration, Python's last-resort handler prints the message only.

D2-L2 exercise 2

Hints. (1) Does basicConfig reconfigure? (2) Only the first call counts. (3) force=True changes that.

Answer. a) WARNING: the second call does nothing Because: r1) basicConfig does nothing if the root logger already has handlers, unless force=True.

Pass `force=True`, or set levels on named loggers with `setLevel`.

Common wrong answers.

- `b`: Only with force=True, or with logger.setLevel.
- `c`: It returns silently, which is why this surprises people.
- `d`: No second handler is added.

- `a|r2`: Right answer, wrong reason. setLevel or force=True lower the level fine.

D2-L2 exercise 3

Hints. (1) Which logger method adds the traceback? (2) A bare statement re-raises the current exception. (3) `exception` and `raise`.

Answer.

```
try:
    validate(df)
except ContractError:
    logger.exception("contract failed for wave %s", wave_id)
    raise
```

`logger.exception` logs at ERROR with the traceback. A bare `raise` re-raises the same exception.

Common wrong answers.

- g1 = `error`: error logs the message without the traceback. exception adds it.
- g2 = `pass`: Swallowing the error lets the pipeline continue with invalid data.

D2-L2 exercise 4

Hints. (1) A dict from status to level keeps the branches short. (2) `logger.log(level, msg, *args)` logs at any level. (3) Handle the CRITICAL case first: log, then raise.

Answer.

```
import logging

STATUS_LEVELS = {"PASS": logging.INFO, "WARN": logging.WARNING, "FAIL": logging.ERROR}


def log_check(
    logger: logging.Logger, check: str, status: str, severity: str, wave_id: str
) -> None:
    """Log one check result at a level matching its outcome.

    Args:
        logger: Module logger.
        check: Check name.
        status: "PASS", "WARN" or "FAIL".
        severity: Severity of the check, such as "CRITICAL".
        wave_id: Wave being checked.
```

```
17
18      Raises:
19          ValueError: If status is unknown.
20          RuntimeError: If a CRITICAL check failed.
21      """
22      if status not in STATUS_LEVELS:
23          raise ValueError(f"unknown status: {status}")
24      if status == "FAIL" and severity == "CRITICAL":
25          logger.critical("wave %s check %s failed (CRITICAL)", wave_id,
    check)
26          raise RuntimeError(f"CRITICAL check {check} failed for wave
    {wave_id}")
27      logger.log(STATUS_LEVELS[status], "wave %s check %s %s", wave_id,
    check, status)
```

Levels describe. Exceptions stop. A CRITICAL failure needs both.

D2-L2 exercise 5

Hints. (1) What do you read at 3 a.m.? (2) Which identifiers find the data? (3) Where does history for dashboards live?

Answer.

- INFO keeps logs readable. A parameter enables DEBUG without a code change.
- Identifiers and counts let you find the step that dropped rows.
- Structured history belongs in a table. Text logs are for people.

INFO by default, identifiers everywhere, and a structured run log alongside the text.

Common wrong answers.

- level = `debug`: DEBUG every night buries the lines you need under detail you rarely read.
- content = `done`: 'Done' cannot tell you which wave or how many rows.
- record = `parse`: Log text changes. A run log table with a fixed schema is queryable.

D2-L3 explain(), the Spark UI and the query profile

D2-L3 exercise 1

Hints. (1) What does Cartesian mean? (2) Every pair of rows. (3) Check the join condition.

Answer. a) A join without an equality condition, so every row meets every row Because: r1) A Cartesian product pairs every row with every row. It comes from a cross join or a join condition Spark cannot use as an equi-join.

Find the join that produced it and give it an equality condition on the keys.

Common wrong answers.

- `b`: Broadcast joins show as BroadcastHashJoin.
- `c`: Python UDFs show as BatchEvalPython or ArrowEvalPython.
- `d`: Pruning shows as PartitionFilters on the scan.
- `a|r2`: Right answer, wrong reason. Small tables get BroadcastHashJoin when there is an equality condition.

D2-L3 exercise 2

Hints. (1) Which operator moves data across the cluster? (2) Its name means swap. (3) Exchange.

Answer. The bug is on line 3.

`Exchange hashpartitioning(wave_id, 200)` shuffles partial sums so each wave's rows meet on one executor for the final sum.

Common wrong answers.

- Line 6: The scan reads files. The shuffle moves data between executors, which is an Exchange.
- Line 2: HashAggregate aggregates. The final aggregate needs the data shuffled first, by the Exchange below it.

D2-L3 exercise 3

Hints. (1) Is the Spark UI available on serverless? (2) No. (3) Its replacement.

Answer. a) The query profile Because: r1) The Spark UI is not available on serverless. Databricks points you to the query profile.

On serverless, use the query profile. On classic compute, the Spark UI's SQL tab shows the same kind of detail.

Common wrong answers.

- `b`: Correct for classic compute. Serverless has no Spark UI.
- `c`: explain shows the plan before running, not what happened.
- `d`: Logs do not show per-operator metrics.
- `a|r2`: Right answer, wrong reason. Classic compute has the Spark UI too.

D2-L3 exercise 4

Hints. (1) Keep the operator names in constants. (2) A negative lookbehind, `(?<!Broadcast)`, excludes BroadcastExchange. (3) Append warnings in the stated order.

Answer.

```python
import re

CROSS_JOIN_OPERATORS = ("CartesianProduct", "BroadcastNestedLoopJoin")
PYTHON_UDF_OPERATORS = ("BatchEvalPython", "ArrowEvalPython")
SHUFFLE_PATTERN = re.compile(r"(?<!Broadcast)\bExchange\b")


def plan_warnings(plan: str, max_exchanges: int = 2) -> list[str]:
    """Warnings for common problems in a Spark physical plan.

    Args:
        plan: Text of df.explain().
        max_exchanges: Number of shuffles allowed before warning.

    Returns:
        Warnings in a fixed order, each at most once.
    """
    warnings: list[str] = []
    if any(op in plan for op in CROSS_JOIN_OPERATORS):
        warnings.append("cross join")
    if any(op in plan for op in PYTHON_UDF_OPERATORS):
        warnings.append("python udf")
    if len(SHUFFLE_PATTERN.findall(plan)) > max_exchanges:
        warnings.append("many shuffles")
    return warnings
```

A plan linter in a test catches a cross join the day it appears, not the day the dimension grows.

D2-L3 exercise 5

Hints. (1) What is free to check? (2) Where are rows and task times? (3) What does one slow task mean?

Answer.

- The plan shows join types and shuffles for free, before you spend compute.
- Rows per operator show where rows multiply. Task times show skew.
- One very slow task among fast ones is the signature of a skewed key.

Plan first, then runtime metrics, then the data behind any outlier.

Common wrong answers.

- first = `size`: A bigger cluster makes a cross join or a skewed key slightly less slow, at double the cost.
- runtime = `rerun`: A rerun tells you it is still slow, not why.
- skew = `slow_node`: Skew follows the data, not the machine. It returns after a restart.

D3-L1 Join row explosion and nulls in filters

D3-L1 exercise 1

Hints. (1) How many rows in regions match code N? (2) How many respondents have code N? (3) 2 x 2 + 1.

Answer.

```
1 5
```

Each N respondent matches two lookup rows. 4 rows for N, 1 for S. The left join multiplied the grain.

Common wrong answers.

- `3`: A left join keeps every left row, but each one appears once per matching right row. Code N matches twice.
- `4`: Two respondents have code N, and each matches two rows: 2 x 2 + 1 = 5.

D3-L1 exercise 2

Hints. (1) What is null != 'N' in SQL logic? (2) Null, which is not true. (3) 1,000 - 300 - 50.

Answer. a) 650 Because: r1) null != 'N' evaluates to null, and a filter keeps only rows where the condition is true.

Three-valued logic: comparisons with null give null. Add `| F.col("region_code").isNull()` to keep them.

Common wrong answers.

- `b`: That is pandas behaviour. In Spark the 50 null rows are dropped too.
- `c`: That keeps N and null. The filter removes N.
- `d`: The filter removes at least the 300 N rows.
- `a|r2`: Right answer, wrong reason. Spark keeps nulls in general. Only the comparison yields null here.

D3-L1 exercise 3

Hints. (1) Expand NOT IN into ANDs. (2) What is x <> NULL? (3) Use NOT EXISTS or a left_anti join.

Answer. a) x NOT IN (..., NULL) is never true, because x <> NULL is null Because: r1) NOT IN expands to x <> 'N' AND x <> NULL. The second term is null, so the whole condition is never true.

Use `NOT EXISTS` or a `left_anti` join. Both treat a null in the exclusion list as "no match".

Common wrong answers.

- `b`: It runs. It just returns no rows when the list contains a null.
- `c`: Two rows is fine. The null is the problem.
- `d`: Even an S respondent fails the null comparison.
- `a|r2`: Right answer, wrong reason. The subquery keeps the null, and that is what breaks NOT IN.

D3-L1 exercise 4

Hints. (1) `lookup.dropna(subset=[key])` removes null keys. (2) `value_counts()` then `counts[counts > 1]` finds duplicates. (3) Check `len(joined) == len(facts)` before returning.

Answer.

```python
import pandas as pd


def safe_left_join(facts: pd.DataFrame, lookup: pd.DataFrame, key: str) -> pd.DataFrame:
    """Left join that keeps the grain of facts.

    Args:
        facts: Rows to keep, one per output row.
        lookup: Attributes to attach, at most one row per key.
        key: Join column.

    Returns:
        The joined frame with len(facts) rows.

    Raises:
        ValueError: If lookup holds a key more than once, or the grain changes.
    """
    clean = lookup.dropna(subset=[key])
    counts = clean[key].value_counts()
    duplicated = sorted(counts[counts > 1].index.tolist())
    if duplicated:
```

```
22          raise ValueError(f"duplicate keys in lookup: {duplicated}")
23      joined = facts.merge(clean, on=key, how="left")
24      if len(joined) != len(facts):
25          raise ValueError(f"join changed rows: {len(facts)} ->
    {len(joined)}")
26      return joined
```

Report duplicates, never drop them silently: which "North" is right is a business decision. Null keys leave the lookup before the merge.

D3-L1 exercise 5

Hints. (1) What is null != 'test' in Spark? (2) Null, so the row is dropped. (3) eqNullSafe treats null as a comparable value.

Answer. The bug is on line 3.

Fixed version:

```
1 from pyspark.sql import functions as F
2
3 live = respondents.filter(~F.col("panel").eqNullSafe("test"))
4 live.write.mode("overwrite").saveAsTable("research_silver.nat_live")
```

`~F.col("panel").eqNullSafe("test")` is true for nulls, so respondents with no panel stay.

Common wrong answers.

- Line 4: The write stores what the filter kept. The rows were lost one line earlier.

D3-L2 Type coercion and timezone drift

D3-L2 exercise 1

Hints. (1) Compare 'W10' and 'W2' character by character. (2) '1' < '2'. (3) W10 first.

Answer.

```
1 ['W10', 'W2', 'W9']
```

Text sorts by character. Pad ids (`W02`, `W09`, `W10`) or sort by the number: `sorted(waves, key=lambda w: int(w[1:]))`.

Common wrong answers.

- `['W2', 'W9', 'W10']`: That is numeric order. Strings compare character by character, and '1' comes before '2'.
- `['W9', 'W2', 'W10']`: Strings sort ascending by character. 'W1...' comes first.

D3-L2 exercise 2

Hints. (1) What is 23:30 UTC in London in summer? (2) BST is UTC+1. (3) 00:30 the next day.

Answer. a) 2026-07-01 Because: r1) TIMESTAMP is an instant. to_date uses the session time zone, where 23:30 UTC is 00:30 on 1 July in BST.

Date boundaries depend on the zone. Set the session time zone deliberately, or convert with a named zone in the rule itself.

Common wrong answers.

- `b`: That is the date in UTC. The session time zone here is London, one hour ahead in summer.
- `c`: Spark uses the session time zone setting, not the OS clock.
- `d`: The conversion succeeds. It just gives a different day.
- `a|r2`: Right answer, wrong reason. Spark stores the instant. It applies the zone when converting or displaying.

D3-L2 exercise 3

Hints. (1) Which method takes a time zone argument? (2) Pass the UTC zone object. (3) `now` and `timezone.utc`.

Answer.

```
from datetime import datetime, timezone

run_timestamp = datetime.now(timezone.utc)
```

`datetime.now(timezone.utc)` returns an aware datetime in UTC.

Common wrong answers.

- g1 = `utcnow`: utcnow() is deprecated from Python 3.12 and returns a naive datetime.

D3-L2 exercise 4

Hints. (1) Pass `dtype=` to read_csv for the id columns. (2) Read the weight as a string first, then `pd.to_numeric(..., errors="coerce")`. (3) Failed = present before, missing after.

Answer.

```
import io

import pandas as pd

ID_COLUMNS = {"respondent_id": "string", "postcode_area": "string"}

```

```python

def load_delivery(csv_text: str) -> pd.DataFrame:
    """Read a delivery CSV with identifiers as strings and weights as floats.

    Args:
        csv_text: CSV text with respondent_id, postcode_area and final_weight.

    Returns:
        The delivery as a DataFrame.

    Raises:
        ValueError: If any non-empty weight cannot be parsed.
    """
    df = pd.read_csv(io.StringIO(csv_text), dtype={**ID_COLUMNS, "final_weight": "string"})
    raw = df["final_weight"]
    parsed = pd.to_numeric(raw, errors="coerce")
    failed = int((raw.notna() & parsed.isna()).sum())
    if failed:
        raise ValueError(f"{failed} weights could not be parsed")
    return df.assign(final_weight=parsed.astype("float64"))
```

Read as text, convert explicitly, and count what the conversion lost. Silent coercion hides bad data.

D3-L2 exercise 5

Hints. (1) How does union match columns? (2) By position. (3) unionByName matches by name.

Answer. The bug is on line 3.

Fixed version:

```python
from pyspark.sql import functions as F

combined = original.unionByName(redelivery)
combined = combined.withColumn("loaded_at", F.current_timestamp())
```

`union` pairs the first column with the first column, whatever their names. `unionByName` pairs by name, and fails if a name is missing.

Common wrong answers.

- Line 4: Adding a timestamp cannot move ids into the wave column. The union already mixed them.

D3-L3 Mutable defaults and the truthiness of zero

D3-L3 exercise 1

Hints. (1) When is the default `[]` created? (2) Both calls use the default. Is it the same list? (3) The list is created once, when Python defines the function.

Answer.

```
1 ['null_weights', 'row_count']
```

The default list is created when Python defines the function. Both calls append to that one list, so the second call returns both flags.

Common wrong answers.

- `['row_count']`: That is what the author intended. The default list is created once, so the second call appends to the list the first call used.
- `['row_count', 'null_weights']`: `append` adds to the end. The first call's flag is already in the list.

D3-L3 exercise 2

Hints. (1) Evaluate `0.0 and 0.12` in your head. (2) `0.0` is falsy, like `None`. (3) The whole block, including `results.append`, is skipped.

Answer. c) Leaves it out of the results. Because: r1) `if current_pct and prior_pct:` is False when current_pct is 0.0, so the block never runs.

`0.0 and 0.12` evaluates to `0.0`, which is falsy. The block that compares and appends never runs, so the metric vanishes from the results and no warning appears.

Common wrong answers.

- `a`: That is what the check should do. Look at the `if` line before the comparison: 0.0 is falsy.
- `b`: Nothing is appended for this metric, so it cannot be marked PASS. The `if` skips it.
- `d`: The check subtracts, it does not divide. No error appears, which is the problem.
- `c|r2`: Right outcome, wrong reason. There is no 1% filter. The truthiness test treats 0.0 as missing.

D3-L3 exercise 3

Hints. (1) The first default is None, so the type must allow None. (2) The second default must be immutable. (3) `str | None` for the first gap and `None` for the second.

Answer.

```
1 def log_run(status: str, error_message: str | None = None) -> None: ...
2
3 def collect(flag: str, flags: list[str] | None = None) -> list[str]: ...
```

`error_message: str | None = None` states that the value may be missing. `flags: list[str] | None = None` avoids a shared list. Create the list inside the function when it is `None`.

Common wrong answers.

- g1 = `str: str = None` claims the value is always a string. mypy in strict mode rejects it. Write `str | None`.
- g2 = `[]`: A list default is created once and shared by every call. Use `None` and create the list inside the function.

D3-L3 exercise 4

Hints. (1) Which values make `if current_pct and prior_pct:` False? (2) 0.0 is one of them. (3) Replace the truthiness test with `is not None` on both values.

Answer. The bug is on line 3.

Fixed version:

```
1 def stability_status(current_pct: float | None, prior_pct: float | None,
     tolerance: float = 0.10) -> str:
2     """Return WARN, PASS, or SKIP when a share is missing."""
3     if current_pct is not None and prior_pct is not None:
4         change = abs(current_pct - prior_pct)
5         return "WARN" if change > tolerance else "PASS"
6     return "SKIP"
```

Line 3 treats 0.0 as missing. Use `if current_pct is not None and prior_pct is not None:` so that only missing shares skip the comparison.

Common wrong answers.

- Line 4: The subtraction is correct. The code never reaches it when a share is 0.0.
- Line 5: The comparison with the tolerance is correct. Look at what stops the code reaching it.

D3-L3 exercise 5

Hints. (1) Start with `flags: list[str] = []` inside the function. (2) Read the prior share with `prior.get(metric)`, which gives None for a missing metric. (3) Skip with `if current_pct is None or prior_pct is None: continue`, then compare with the tolerance.

Answer.

```python
def compare_waves(
    current: dict[str, float | None],
    prior: dict[str, float | None],
    tolerance: float = 0.10,
) -> list[str]:
    """Flag metrics whose share moved more than tolerance between waves.

    A metric is skipped only when a share is missing. A share of 0.0 is compared.
    """
    flags: list[str] = []
    for metric, current_pct in current.items():
        prior_pct = prior.get(metric)
        if current_pct is None or prior_pct is None:
            continue
        if abs(current_pct - prior_pct) > tolerance:
            flags.append(metric)
    return flags
```

Create the list inside the function. Use `prior.get` so a new metric gives `None`. Skip only on `None`, then compare the absolute change with the tolerance.

D3-L3 exercise 6

Hints. (1) Default `results` to None. (2) Create a new list only when `results is None`. (3) `results or []` looks similar but replaces an empty list the caller passed. Use `is None`.

Answer.

```python
def add_result(
    name: str, passed: bool, results: list[dict[str, object]] | None = None
) -> list[dict[str, object]]:
    """Append one check result and return the list.

    Args:
        name: Check name.
        passed: Whether the check passed.
        results: List to append to. A new list is created when omitted.

    Returns:
        The list with the new result.
    """
```

```
    results = [] if results is None else results
    results.append({"check": name, "passed": passed})
    return results
```

Default to `None`, create the list when `results is None`, then append and return. Avoid `results or []`: an empty list the caller passed is falsy, so the function would append to a new list and the caller would never see the result.

D4-L1 Row counts across layers

D4-L1 exercise 1

Hints. (1) What is silver's grain? (2) One row per respondent per wave. (3) Count distinct keys in bronze.

Answer. a) 10,000 Because: r1) Silver's grain is one row per respondent per wave, so it holds one row per distinct key.

The grain turns a vague expectation into an exact rule: silver rows = distinct bronze keys.

Common wrong answers.

- `b`: That keeps the 400 duplicates. Silver deduplicates on the key.
- `c`: The grain fixes the number exactly. A vague rule catches nothing.
- `d`: Weights do not change row counts.
- `a|r2`: Right answer, wrong reason. The number follows the grain, not a fixed rate.

D4-L1 exercise 2

Hints. (1) Which metric adds rows? (2) Inserts. (3) 10,000 + 120.

Answer. a) 10,120 Because: r1) Updates replace existing rows. Only inserts add rows.

Check after every MERGE: rows after = rows before + numTargetRowsInserted - rows deleted.

Common wrong answers.

- `b`: 380 source rows updated existing rows, so they add nothing.
- `c`: Updated rows were already counted in the 10,000.
- `d`: 120 new respondents were inserted.
- `a|r2`: Right answer, wrong reason. Updated rows replace existing ones and add nothing.

D4-L1 exercise 3

Hints. (1) Which function counts distinct values? (2) Then fetch the single result row. (3) `count_distinct` and `first`.

Answer.

```python
row = df.agg(
    F.count("*").alias("rows"),
    F.count_distinct("respondent_id").alias("keys"),
).first()
```

One `agg` returns both numbers in one scan. `first()` brings the single row back to the driver.

Common wrong answers.

- g1 = `count`: count counts non-null values, duplicates included. You need distinct keys.

D4-L1 exercise 4

Hints. (1) Validate inputs first. (2) List the rules as (name, expected, actual) tuples. (3) Silver is compared with distinct bronze keys.

Answer.

```python
def reconcile(
    delivered: int,
    bronze_rows: int,
    bronze_keys: int,
    silver_rows: int,
    gold_n: int,
    in_scope: int,
) -> list[dict[str, object]]:
    """Row-count rules across layers for one tracker and wave.

    Args:
        delivered: Rows in the delivery manifest.
        bronze_rows: Bronze rows for the delivery.
        bronze_keys: Distinct respondent keys in bronze.
        silver_rows: Silver rows for the wave.
        gold_n: Sum of gold unweighted_n.
        in_scope: Silver rows in scope.

    Returns:
        One structured record per rule.

    Raises:
        ValueError: If inputs are impossible.
    """
    counts = (delivered, bronze_rows, bronze_keys, silver_rows, gold_n,
in_scope)
    if min(counts) < 0:
```

```
        raise ValueError("counts must not be negative")
    if in_scope > silver_rows:
        raise ValueError("in_scope cannot exceed silver_rows")
    rules = [
        ("bronze_vs_delivery", delivered, bronze_rows),
        ("silver_vs_bronze_keys", bronze_keys, silver_rows),
        ("gold_vs_silver_scope", in_scope, gold_n),
    ]
    return [
        {"rule": name, "expected": expected, "actual": actual,
         "status": "PASS" if expected == actual else "FAIL"}
        for name, expected, actual in rules
    ]
```

Structured records with expected and actual values go straight into the run log, and a dashboard can chart them.

D4-L1 exercise 5

Hints. (1) Compare the filters on both sides. (2) Which side has no tracker filter? (3) Line 5.

Answer. The bug is on line 5.

Fixed version:

```
bronze_waves = (
    spark.table(bronze_table).filter(F.col("tracker_id") == tracker_id)
    .select("wave_id").distinct()
)
silver_waves = spark.table(silver_table).filter(F.col("tracker_id") ==
   tracker_id).select("wave_id").distinct()
new_waves = [r.wave_id for r in
   bronze_waves.subtract(silver_waves).collect()]
```

Both sides of a comparison need the same keys. Silver waves for all trackers hide this tracker's missing waves.

Common wrong answers.

- Line 6: subtract is the right operation. Its right side holds every tracker's waves.
- Line 2: The bronze side is filtered correctly. The silver side is not.

D4-L2 Distribution drift between waves

D4-L2 exercise 1

Hints. (1) Compare the labels, not the counts. (2) North vanished, Northern appeared. (3) A recode.

Answer. a) A recode by the supplier Because: r1) A category that vanishes while a similar new one appears is a label change until proven otherwise.

Check new and missing categories before comparing shares.

Common wrong answers.

- `b`: A population does not rename itself. Check the labels first.
- `c`: Weights change shares, not category names.
- `d`: Sampling variation moves shares. It does not create new labels.
- `a|r2`: Right answer, wrong reason. Most waves have no recode, which is why a check must catch the one that does.

D4-L2 exercise 2

Hints. (1) Both totals are 100. (2) 0.45 - 0.30. (3) In points, times 100.

Answer.

```
15
```

A move from 30% to 45% is 15 percentage points, and a 50% relative increase. State which one a tolerance means.

Common wrong answers.

- `50`: That is the relative change (45/30 - 1). The check uses percentage points: 45% - 30%.
- `0`: The shares differ by 0.15, which is 15 points.

D4-L2 exercise 3

Hints. (1) Natural logarithm. (2) Of actual over expected. (3) `log` and `e`.

Answer.

```
import math

e = max(expected_share, FLOOR)
a = max(actual_share, FLOOR)
term = (a - e) * math.log(a / e)
```

`(a - e) * ln(a / e)` is never negative, so every category adds to the index.

Common wrong answers.

- g1 = `log10`: PSI uses the natural logarithm, math.log.

D4-L2 exercise 4

Hints. (1) Totals first, and guard zero. (2) Loop over `expected.keys() | actual.keys()`. (3) Floor each share with `max(share, floor)`.

Answer.

```python
import math


def psi(expected: dict[str, int], actual: dict[str, int], floor: float = 0.0001) -> float:
    """Population stability index between two categorical distributions.

    Args:
        expected: Category counts in the reference wave.
        actual: Category counts in the current wave.
        floor: Minimum share, so a zero share does not break the logarithm.

    Returns:
        PSI, 0 when the shares are identical.

    Raises:
        ValueError: If either wave has no observations.
    """
    e_total = sum(expected.values())
    a_total = sum(actual.values())
    if e_total == 0 or a_total == 0:
        raise ValueError("a wave has no observations")
    index = 0.0
    for category in expected.keys() | actual.keys():
        e = max(expected.get(category, 0) / e_total, floor)
        a = max(actual.get(category, 0) / a_total, floor)
        index += (a - e) * math.log(a / e)
    return index
```

Shares, the union of categories and a floor. Without the union, the recode that matters most adds nothing to the index.

D4-L2 exercise 5

Hints. (1) What is total for an empty delivery? (2) Zero. (3) Check it before dividing.

Answer. The bug is on line 3.

Fixed version:

```python
def distribution_status(value_count: int, total: int, min_pct: float, max_pct: float) -> str:
    """PASS if the value's share is within bounds, WARN if not, NO_DATA if empty."""
    if total == 0:
        return "NO_DATA"
    pct = value_count / total
    return "PASS" if min_pct <= pct <= max_pct else "WARN"
```

Guard every division. An empty delivery is a status to report, not a crash.

Common wrong answers.

- Line 4: The bounds check is fine. The division above it fails first.

D4-L3 Weight totals and survey checks

D4-L3 exercise 1

Hints. (1) What does when() return when no otherwise is given? (2) Null. (3) Sum of nulls.

Answer. a) NULL Because: r1) when without otherwise gives null for non-matching rows, and the sum of only nulls is null.

Add `.otherwise(0)` inside, or `F.coalesce(..., F.lit(0))` outside.

Common wrong answers.

- `b`: That is what you want, and what coalesce(..., 0) gives you. The raw sum is null.
- `c`: No error: the null passes silently into the table.
- `d`: groupBy keeps the subgroup. Its sum is null.
- `a|r2`: Right answer, wrong reason. Zeros sum normally. Only an all-null input gives null.

D4-L3 exercise 2

Hints. (1) sum(w^2) = 4, (sum w)^2 = 16. (2) 4 x 4 / 16. (3) 1.0.

Answer.

```
1.0
```

Equal weights give deff 1 and n_eff = n. Any variation raises deff above 1.

Common wrong answers.

- `4.0`: 4 x 4 / 16 = 1. Equal weights cost no precision.
- `0.0`: deff is at least 1. Equal weights give exactly 1.

D4-L3 exercise 3

Hints. (1) Which function returns the first non-null argument? (2) The replacement value. (3) `coalesce` and `0`.

Answer.

```
F.coalesce(F.sum(F.when(F.col("reach_flag") == 1, 1)),
    F.lit(0)).alias("unweighted_reach")
```

`coalesce` returns its first non-null argument, so a null sum becomes 0.

Common wrong answers.

- g1 = `isnull`: isnull tests for null and returns a boolean. coalesce replaces it.

D4-L3 exercise 4

Hints. (1) Check emptiness and invalid weights first. (2) Test `w is not None and w > 0`, not truthiness. (3) n_eff = total**2 / sum of squares; deff = n / n_eff.

Answer.

```python
def weight_report(weights: list[float | None], max_ratio: float = 20.0) -> dict[str, object]:
    """Weight checks with Kish's effective sample size and design effect.

    Args:
        weights: One weight per respondent. None means missing.
        max_ratio: Largest allowed max/min weight ratio before WARN.

    Returns:
        status ("PASS", "WARN" or "CRITICAL"), n, n_eff and deff.
    """
    n = len(weights)
    valid = [w for w in weights if w is not None and w > 0]
    if n == 0 or len(valid) != n:
        return {"status": "CRITICAL", "n": n, "n_eff": None, "deff": None}
    total = sum(valid)
    n_eff = total**2 / sum(w * w for w in valid)
    status = "WARN" if max(valid) / min(valid) > max_ratio else "PASS"
    return {"status": status, "n": n, "n_eff": round(n_eff, 4), "deff": round(n / n_eff, 4)}
```

A missing or non-positive weight is a data error to report, never a row to drop quietly.

D4-L3 exercise 5

Hints. (1) Which result makes the estimate undefined? (2) Which result needs judgement? (3) Which standard deviation does Kish use?

Answer.

- A null weight makes every estimate for that respondent undefined: halt.
- A high deff is a signal for a person, not proof of an error: warn.
- Kish's identity uses the population standard deviation. Guard the empty case.

CRITICAL for undefined estimates, WARN for signals, and the right statistic for the formula.

Common wrong answers.

- nulls = `warn`: Dropping rows changes the sample silently. Halt and fix the weighting.
- deff = `critical`: Real populations shift. Halting on every efficiency change blocks legitimate waves.
- sd = `samp`: F.stddev is the sample version, null for one row, and does not match 1 + CV^2.

D5-L1 Invented functions, parameters and syntax

D5-L1 exercise 1

Hints. (1) Is delta() a DataFrameReader method? (2) Check with hasattr(spark.read, 'delta'). (3) No.

Answer. a) AttributeError: DataFrameReader has no attribute 'delta'
Because: r1) The reader has format('delta').load(path) and table(name), but no delta() method.

Plausible, and invented. `hasattr` or the PySpark API reference answers in seconds.

Common wrong answers.

- `b`: The method does not exist. Use spark.read.format("delta").load(path) or spark.read.table(name).
- `c`: No read happens. The attribute lookup fails first.
- `d`: Python raises AttributeError at once.
- `a|r2`: Right answer, wrong reason. Reading by path works with format('delta').load(path).

D5-L1 exercise 2

Hints. (1) What does Column do with an unknown attribute? (2) It assumes a struct field. (3) The result is another Column.

Answer.

```
1 Column
```

`F.col("region").isnotnull` means "the field isnotnull inside the struct region". Calling it raises `TypeError: 'Column' object is not callable`.

Common wrong answers.

- `method`: The real method is isNotNull, with capitals. isnotnull is treated as a struct field name.

- `AttributeError`: Column turns unknown attributes into field lookups, so no error appears here.

D5-L1 exercise 3

Hints. (1) Which statement comes from another platform? (2) Databricks binds masks with ALTER TABLE. (3) ALTER COLUMN ... SET MASK.

Answer. The bug is on line 3.

Fixed version:

```
1 CREATE FUNCTION research_governance.mask_postcode(p STRING)
2   RETURN CASE WHEN is_account_group_member('analysts') THEN p ELSE '***'
    END;
3 ALTER TABLE research_silver.nat_respondents ALTER COLUMN postcode SET
    MASK research_governance.mask_postcode;
```

Databricks column masks: a SQL UDF, then `ALTER TABLE ... ALTER COLUMN ... SET MASK`.

Common wrong answers.

- Line 1: A SQL UDF is exactly what Databricks column masks use.
- Line 2: is_account_group_member is the recommended check in Unity Catalog.

D5-L1 exercise 4

Hints. (1) `inspect.signature(func).parameters` maps names to Parameter objects. (2) `p.kind is inspect.Parameter.VAR_KEYWORD` marks **kwargs. (3) Keep the order of params.

Answer.

```
1  import inspect
2  from collections.abc import Callable
3
4
5  def unknown_params(func: Callable[..., object], params: list[str]) ->
     list[str] | None:
6      """Parameter names that a function does not accept.
7
8      Args:
9          func: Function to check.
10         params: Names suggested for it, for example by an assistant.
11
12     Returns:
13         Unknown names in order, or None if func takes **kwargs.
14     """
15     parameters = inspect.signature(func).parameters
16     if any(p.kind is inspect.Parameter.VAR_KEYWORD for p in
       parameters.values()):
```

```
        return None
    return [name for name in params if name not in parameters]
```

A signature check catches an invented parameter before the code runs. With **kwargs, only the docs can answer.

D5-L1 exercise 5

Hints. (1) What proves an API exists for your version? (2) Which reference covers Databricks SQL? (3) What does 'undocumented' guarantee?

Answer.

- A docs link for the right version turns "looks right" into "checked".
- SQL dialects share keywords and differ in statements. Only the platform reference counts.
- Undocumented behaviour can change or disappear without notice.

Docs for your version, the platform's own SQL reference, and no undocumented APIs.

Common wrong answers.

- check = `trust`: Assistants state invented APIs with the same confidence as real ones.
- sql = `generic`: CREATE ROW ACCESS POLICY appears in tutorials. It is Snowflake, not Databricks.
- unverified = `try`: Running is necessary, not sufficient. Undocumented behaviour has no guarantee.

D5-L2 Deprecated APIs and version mismatches

D5-L2 exercise 1

Hints. (1) Check the 17.3 LTS release notes. (2) File details live in a hidden metadata column. (3) _metadata.file_path.

Answer. a) The _metadata.file_path column Because: r1) Databricks release notes state input_file_name is no longer supported from 17.3 LTS. The file metadata column replaces it.

Select `F.col("_metadata.file_path")` when reading files.

Common wrong answers.

- `b`: It is no longer supported on 17.3 LTS and above.
- `c`: os.path works on strings, not on DataFrame columns.
- `d`: A UDF cannot see the source file. The metadata column can.

- `a|r2`: Right answer, wrong reason. It was not renamed. A different mechanism, the _metadata column, replaces it.

D5-L2 exercise 2

Hints. (1) What does pandas 3 do with chained assignment? (2) The intermediate object behaves as a copy. (3) The original keeps 9.0.

Answer.

```
[1.0, 9.0]
```

pandas 3 warns with ChainedAssignmentError and leaves df unchanged. Use `df.loc[1, "w"] = 5.0`.

Common wrong answers.

- `[1.0, 5.0]`: That was pandas 1.x behaviour. Under Copy-on-Write, df['w'] behaves as a copy, so the original is unchanged.

D5-L2 exercise 3

Hints. (1) DataFrame.append is gone. What joins frames end to end? (2) applymap was renamed. (3) `concat` and `map`.

Answer.

```
combined = pd.concat([waves, new_wave], ignore_index=True)
labels = combined.map(str.strip)
```

`pd.concat` replaces `append`. `DataFrame.map` replaces `applymap`.

Common wrong answers.

- g2 = `applymap`: applymap was removed in pandas 3.0. DataFrame.map replaces it.

D5-L2 exercise 4

Hints. (1) Split on dots and read the leading digits of each part. (2) Compare tuples of ints. (3) Pad the shorter tuple with zeros.

Answer.

```
import re

SEGMENT = re.compile(r"^(\d+)")


def _parse(version: str) -> tuple[int, ...]:
    """Numeric segments of a dotted version string."""
    parts = []
    for segment in version.split("."):
        match = SEGMENT.match(segment)
        if match is None:
```

```
            raise ValueError(f"invalid version segment {segment!r} in
    {version!r}")
        parts.append(int(match.group(1)))
    return tuple(parts)


def version_at_least(installed: str, required: str) -> bool:
    """True if installed is the same as or newer than required.

    Args:
        installed: Installed version, such as "3.0.2".
        required: Minimum version, such as "3.0".

    Returns:
        Whether installed >= required, compared numerically.

    Raises:
        ValueError: If a version has a segment without digits.
    """
    a, b = _parse(installed), _parse(required)
    width = max(len(a), len(b))
    return a + (0,) * (width - len(a)) >= b + (0,) * (width - len(b))
```

Tuples of integers compare element by element, which is version order. In real projects, use `packaging.version.Version`.

D5-L2 exercise 5

Hints. (1) Two indexing steps in one assignment. (2) Chained assignment. (3) out.loc[mask, 'final_weight'] = cap.

Answer. The bug is on line 6.

Fixed version:

```
import pandas as pd

def cap_weights(df: pd.DataFrame, cap: float) -> pd.DataFrame:
    """Return a copy with final_weight capped at cap."""
    out = df.copy()
    out.loc[out["final_weight"] > cap, "final_weight"] = cap
    return out
```

Index once with `.loc[rows, column]`. Chained indexing sets values on a temporary copy.

Common wrong answers.

- Line 5: The copy is right: it keeps the caller's frame unchanged.

D5-L3 Five review checks for AI-written code

D5-L3 exercise 1

Hints. (1) Where does the expected value come from? (2) From the same formula. (3) Self-confirming.

Answer. a) It repeats the implementation, so a wrong formula passes Because: r1) A test's expected value must come from outside the code: small data worked by hand.

Write `assert weighted_pct(25, 200) == 12.5` and `assert weighted_pct(1, 0) is None`: numbers you worked out yourself.

Common wrong answers.

- `b`: approx helps with floats, but the test still repeats the formula.
- `c`: Arithmetic is fine. Copying the implementation is the problem.
- `d`: It checks that the code equals itself.
- `a|r2`: Right answer, wrong reason. Tests call the function. The expected value must be independent.

D5-L3 exercise 2

Hints. (1) Which line would leak through Git history? (2) A literal credential. (3) Read it at run time from the scope.

Answer. The bug is on line 3.

Fixed version:

```
1 import requests
2
3 def fetch_wave(wave_id: str) -> dict[str, object]:
4     """Fetch wave metadata from the survey platform."""
5     token = dbutils.secrets.get("survey-api", "token")
6     headers = {"Authorization": f"Bearer {token}"}
7     return requests.get(f"https://survey.example/api/waves/{wave_id}",
    headers=headers, timeout=30).json()
```

Secrets come from a secret scope at run time. Databricks also redacts values read with `dbutils.secrets.get` in notebook output.

Common wrong answers.

- Line 7: The header format is right. The value it uses comes from a literal on line 3.
- Line 8: The request is fine, with a timeout even. The secret is the problem.

D5-L3 exercise 3

Hints. (1) What does ALL on a whole table remove? (2) Every filter on it. (3) Including the wave.

Answer. a) Weighted reach for all waves, all regions and all brands Because: r1) ALL(fact_reach) removes every filter on the fact table, including the wave slicer.

Name the dimension whose filter you mean to remove: `ALL(dim_brand)` keeps the wave and region filters.

Common wrong answers.

- `b`: That is what ALL(dim_brand) would give. ALL on the fact table removes the wave filter too.
- `c`: That is the numerator's context.
- `d`: ALL accepts a table. That is exactly the problem here.
- `a|r2`: Right answer, wrong reason. CALCULATE keeps slicer filters unless a modifier such as ALL removes them.

D5-L3 exercise 4

Hints. (1) Check lengths before zip: zip stops at the shorter list. (2) Filter pairs with `is not None`. (3) Guard the total before dividing.

Answer.

```python
def weighted_mean(values: list[float | None], weights: list[float | None]) -> float | None:
    """Weighted mean that skips missing pairs.

    Args:
        values: Values, None for missing.
        weights: Weights, None for missing.

    Returns:
        The weighted mean, or None if there is nothing to average.

    Raises:
        ValueError: If lengths differ or a weight is negative.
    """
    if len(values) != len(weights):
        raise ValueError("values and weights differ in length")
    pairs = [(v, w) for v, w in zip(values, weights) if v is not None and w is not None]
    if any(w < 0 for _, w in pairs):
        raise ValueError("weights must not be negative")
    total = sum(w for _, w in pairs)
    if total == 0:
        return None
```

```
    return sum(v * w for v, w in pairs) / total
```

Five edge cases the one-liner missed: different lengths, None values, negative weights, empty input and a zero total.

D5-L3 exercise 5

Hints. (1) What does the code do, stripped of structure? (2) One comparison per weight. (3) A list comprehension with enumerate.

Answer.

```
def flag_high_weights(weights: list[float], cap: float) -> list[int]:
    """Indexes of weights strictly above cap.

    Args:
        weights: Respondent weights.
        cap: Largest allowed weight.

    Returns:
        Indexes of weights greater than cap.
    """
    return [i for i, w in enumerate(weights) if w > cap]
```

Three classes and a factory wrapped one comparison. Add structure when a second rule actually arrives.

D6-L1 Read before you run, and check each API

D6-L1 exercise 1

Hints. (1) What can this cell change? (2) The table. (3) Which rule does bronze follow?

Answer. a) The write mode and the target, against the goal and the table's rules Because: r1) Side effects on a production table cannot always be undone. Bronze is append-only.

Side effects first: write mode, target name, DROP, DELETE. Then the rest.

Common wrong answers.

- `b`: Formatting matters later. A destructive write matters now.
- `c`: Speed is secondary to what the cell changes.
- `d`: The result of an overwrite is a replaced table.
- `a|r2`: Right answer, wrong reason. Often the mode is right. You check because the cost of a wrong one is high.

D6-L1 exercise 2

Hints. (1) What do you check the code against? (2) Names before side effects. (3) Run last.

Answer.

1. Restate the goal in one sentence
2. List every function, parameter, table and column
3. Check each name against the docs for your version and the schema
4. Find side effects: write modes, DROP, DELETE
5. Run it on development data in a dev schema

Goal, names, checks, side effects, then a dev run.

Common wrong answers.

- Without a goal stated first, you have nothing to check the code against.
- Find side effects before running, even on dev data.

D6-L1 exercise 3

Hints. (1) A built-in that returns True or False for an attribute. (2) inspect's function for parameters. (3) `hasattr` and `signature`.

Answer.

```
import inspect
from pyspark.sql import functions as F

hasattr(F, "try_divide")
"validate" in inspect.signature(pd.DataFrame.merge).parameters
```

`hasattr` checks a name. `inspect.signature(...).parameters` checks a parameter.

Common wrong answers.

- g1 = `getattr`: getattr raises if the name is missing. hasattr returns True or False.

D6-L1 exercise 4

Hints. (1) Walk the tree with ast.walk and keep ast.Call nodes. (2) Build the dotted name recursively from Attribute.value down to a Name. (3) Return None when the chain hits anything else.

Answer.

```
import ast


def _dotted(node: ast.expr) -> str | None:
```

```
    """Dotted name for a Name or Attribute chain, else None."""
    if isinstance(node, ast.Name):
        return node.id
    if isinstance(node, ast.Attribute):
        base = _dotted(node.value)
        return None if base is None else f"{base}.{node.attr}"
    return None


def called_names(source: str) -> list[str]:
    """Sorted unique dotted names of every call in the source.

    Args:
        source: Python source code.

    Returns:
        Names such as "F.col" or "spark.read.format".
    """
    names = set()
    for node in ast.walk(ast.parse(source)):
        if isinstance(node, ast.Call):
            name = _dotted(node.func)
            if name is not None:
                names.add(name)
    return sorted(names)
```

A list of every call is a checklist: each name gets a docs link or a question mark.

D6-L1 exercise 5

Hints. (1) Which line changes existing data? (2) Look at the write mode. (3) Append.

Answer. The bug is on line 5.

Fixed version:

```
from pyspark.sql import functions as F

delivery = spark.read.format("csv").option("header", True).load(path)
delivery = delivery.withColumn("ingestion_timestamp",
   F.current_timestamp())
delivery.write.mode("append").saveAsTable("research_bronze.nat_raw")
```

One word, `overwrite`, turns an append into a replacement of all history.

Common wrong answers.

- Line 4: Adding an ingestion timestamp is part of the bronze standard.
- Line 3: The read is fine. The write replaces history.

D6-L2 Small data with a known answer, failing test first

D6-L2 exercise 1

Hints. (1) What should a test do when the behaviour is missing? (2) Fail. (3) This one did not.

Answer. a) The test does not check the feature Because: r1) A test must fail when the behaviour it checks is missing. Passing without the feature means it asserts something else.

Watch every new test fail once. Then you know it can.

Common wrong answers.

- `b`: The feature does not exist. The test passed anyway, so it is not checking it.
- `c`: A test that cannot fail is not well written.
- `d`: Speed is unrelated.
- `a|r2`: Right answer, wrong reason. Tests do not fail by default. This one should have.

D6-L2 exercise 2

Hints. (1) Where do > and >= differ? (2) Only at equality. (3) Add that case.

Answer. a) A case with a weight exactly equal to the cap Because: r1) > and >= differ only on the boundary, so only a boundary case can tell them apart.

A deliberate break shows which rule your tests miss. Boundaries are the usual gap.

Common wrong answers.

- `b`: Both operators agree far from the cap. Those cases cannot catch this change.
- `c`: Timing has nothing to do with the comparison.
- `d`: They pass with a known bug, so they are incomplete.
- `a|r2`: Right answer, wrong reason. One well-chosen case is enough: the boundary.

D6-L2 exercise 3

Hints. (1) The function you call on the last line. (2) The parameter about row order. (3) `assertDataFrameEqual` and `checkRowOrder`.

Answer.

```
1 from pyspark.testing import assertDataFrameEqual
```

```
2
3 expected = spark.createDataFrame([("W01", 0.25)], "wave_id string, share
    double")
4 assertDataFrameEqual(result, expected, checkRowOrder=False)
```

`checkRowOrder=False` is the default. Writing it out documents the intent.

Common wrong answers.

- g2 = `ignoreRowOrder`: The parameter is checkRowOrder. It is False by default in PySpark 4.0.

D6-L2 exercise 4

Hints. (1) A normal case whose share is not 0 or 1, so a percentage differs. (2) A zero total expecting None. (3) Work each expected value out by hand.

Answer.

```
1 def share_cases() -> list[tuple[float, float, float | None]]:
2     """Known-answer cases for share(part, total), worked out by hand.
3
4     Returns:
5         (part, total, expected) tuples covering a normal share, a zero
    share and a zero total.
6     """
7     return [
8         (1.0, 4.0, 0.25),
9         (0.0, 10.0, 0.0),
10        (5.0, 0.0, None),
11    ]
```

Good cases are chosen to break wrong code. A case where every version agrees, such as (1, 1, 1.0) for a percentage bug, adds nothing.

D6-L2 exercise 5

Hints. (1) Which rows exercise each rule? (2) Who decides the right answer? (3) How do you prove a test can fail?

Answer.

- Each fixture row exercises one rule of the requirement.
- Expected rows come from you, not from the code under test.
- A deliberate wrong key must fail, or the fixture misses the multi-wave case.

Designed rows, hand-written expectations, and a deliberate break.

Common wrong answers.

- fixture = `sample`: You cannot work out 1,000 rows by hand, and production rarely has every edge case.

- expected = `run`: That makes the test self-confirming: it records the code's output, bugs included.
- prove = `rerun`: A rerun gives the same result. Only a deliberate break shows the test can fail.

D6-L3 Ask for assumptions, then compare a second approach

D6-L3 exercise 1

Hints. (1) Who knows whether it is true? (2) The data. (3) Check it, then keep the check.

Answer. a) Check it on the real data, and add a uniqueness check if it matters Because: r1) An assumption is a claim about data. Only the data can confirm it, and it can stop holding in a later wave.

Check each assumption with a query, then keep the important ones as pipeline checks.

Common wrong answers.

- `b`: The assistant has never seen your data. It guessed.
- `c`: Silently deleting rows hides a delivery problem. Check first, then decide.
- `d`: An unchecked assumption is a future bug.
- `a|r2`: Right answer, wrong reason. The assumptions are usually sensible guesses. You check because guesses can be wrong.

D6-L3 exercise 2

Hints. (1) Which method equals the known answer? (2) 1310. (3) Method b.

Answer.

```
1 b
```

The known answer settles disagreements. Method a lost rows, most likely in its inner join.

Common wrong answers.

- `a`: Method a gives 1250, which does not match the hand-worked 1310.

D6-L3 exercise 3

Hints. (1) Which join keeps left rows without a match? (2) Then count them. (3) `left_anti` and `count`.

Answer.

```
missing = silver.join(regions, "region_code", "left_anti").count()
```

A left anti join returns silver rows with no match in regions. Zero means the assumption holds.

Common wrong answers.

- g1 = `inner`: An inner join keeps the matches. You want the rows with no match.

D6-L3 exercise 4

Hints. (1) Uniqueness uses the pair (respondent_id, wave_id). (2) Positive means > 0 and not None. (3) Use re.fullmatch so 'W145' does not pass.

Answer.

```
import re

WAVE_PATTERN = re.compile(r"W\d{2}")


def violated_assumptions(rows: list[dict[str, object]]) -> list[str]:
    """Names of data assumptions that the rows break.

    Args:
        rows: Records with respondent_id, wave_id and final_weight.

    Returns:
        Violated assumption names in a fixed order.
    """
    violated: list[str] = []
    keys = [(r["respondent_id"], r["wave_id"]) for r in rows]
    if len(keys) != len(set(keys)):
        violated.append("unique_respondent_per_wave")
    weights = [r["final_weight"] for r in rows]
    if any(w is None or not isinstance(w, (int, float)) or w <= 0 for w in weights):
        violated.append("positive_weight")
    waves = [r["wave_id"] for r in rows]
    if any(not isinstance(w, str) or WAVE_PATTERN.fullmatch(w) is None for w in waves):
        violated.append("wave_format")
    return violated
```

Each assumption becomes one small check with a name. The names go into the run log, so a failure says which assumption broke.

D6-L3 exercise 5

Hints. (1) What makes a second method independent? (2) What decides who is right? (3) What catches the next regression?

Answer.

- Independent code avoids repeating the same reasoning, and the same mistake.
- Hand-worked answers decide disagreements.
- A cheap reconciliation total stays in the pipeline to catch the next regression.

Independent method, known answer as referee, and a lasting reconciliation check.

Common wrong answers.

- route = `rewrite`: The same assistant tends to repeat its own assumptions, such as the inner join.
- disagree = `average`: An average of a right and a wrong number is wrong.
- keep = `drop_sql`: The next change can break it again. Keep a cheap independent check.

D7-L1 Review practice: Python and pandas

D7-L1 exercise 1

Hints. (1) Where does uuid come from? (2) It is never imported. (3) Add import uuid.

Answer. The bug is on line 3.

Fixed version:

```
1 import uuid
2
3
4 def new_run_id() -> str:
5     """Return a new unique run id."""
6     return str(uuid.uuid4())
```

Reference defect 1. NameError on the first run. ruff rule F821 flags undefined names before you run anything.

Common wrong answers.

- Line 1: The signature is fine. The body uses a name that was never imported.

D7-L1 exercise 2

Hints. (1) When is the default list created? (2) Once, at definition. (3) Both calls share it.

Answer.

```
2
```

Mutable default argument. Default to None and create the list inside.

Common wrong answers.

- `1`: The default list is created once and shared, so the second call sees the first call's warning.

D7-L1 exercise 3

Hints. (1) Is the returned datetime aware? (2) utcnow() returns a naive one, and is deprecated. (3) datetime.now(timezone.utc).

Answer. The bug is on line 5.

Fixed version:

```python
from datetime import datetime, timezone

def run_timestamp() -> datetime:
    """Current time in UTC for the run log."""
    return datetime.now(timezone.utc)
```

Reference defect 6. Naive timestamps cannot be compared with aware ones and hide their zone.

Common wrong answers.

- Line 1: The import is right, and timezone is even imported already. It is just not used.

D7-L1 exercise 4

Hints. (1) Can error_message be None? (2) Then the type must allow None. (3) str | None = None.

Answer. The bug is on line 4.

Fixed version:

```python
def log_run(
    pipeline_name: str,
    status: str,
    error_message: str | None = None,
) -> None:
    """Append one record to the run log."""
```

Reference defect 7. `str = None` claims the value is always a string. mypy --strict rejects it.

Common wrong answers.

- Line 3: status is required and always a string. The optional parameter is the next one.

D7-L1 exercise 5

Hints. (1) Is 0.0 truthy? (2) No. (3) Use `is not None`.

Answer. The bug is on line 3.

Fixed version:

```
1 def count_weighted(weights: list[float | None]) -> int:
2     """Number of respondents with a recorded weight."""
3     recorded = [w for w in weights if w is not None]
4     return len(recorded)
```

Truthiness treats 0.0 like None. Your standard: test for None with `is None`.

Common wrong answers.

- Line 4: len is right. The filter above drops 0.0.

D7-L1 exercise 6

Hints. (1) Lesson D3-L1: nulls in filters. (2) pandas and Spark differ here. (3) pandas keeps them.

Answer. a) Yes: in pandas, != keeps missing regions Because: r1) pandas treats missing != 'N' as True, so those rows stay. Spark would drop them.

The same filter in PySpark would drop the null rows. Know which engine you are reviewing.

Common wrong answers.

- `b`: That is Spark and SQL behaviour. pandas keeps them.
- `c`: The comparison runs without error.
- `d`: The result is the same for object and str dtypes.
- `a|r2`: Right answer, wrong reason. pandas does not fill. Missing compares as not-equal.

D7-L1 exercise 7

Hints. (1) Which join type keeps unmatched left rows? (2) Inner drops them. (3) how='left'.

Answer. The bug is on line 6.

Fixed version:

```
1 import pandas as pd
2
3 def add_region(resp: pd.DataFrame, regions: pd.DataFrame) -> pd.DataFrame:
4     """Attach region names. One row per respondent."""
5     lookup = regions.dropna(subset=["code"])
6     return resp.merge(lookup, on="code", how="left",
      validate="many_to_one")
```

An inner join quietly removes respondents with unknown codes. The grain promise, one row per respondent, needs a left join.

Common wrong answers.

- Line 5: Dropping null lookup keys is right: pandas would otherwise match null to null.

D7-L1 exercise 8

Hints. (1) What is max(['W9', 'W10'])? (2) 'W9': text comparison. (3) Use key=lambda w: int(w[1:]).

Answer. The bug is on line 5.

Fixed version:

```
1 def latest_wave(wave_ids: list[str]) -> str:
2     """Return the most recent wave id."""
3     if not wave_ids:
4         raise ValueError("no waves")
5     return max(wave_ids, key=lambda w: int(w[1:]))
```

Text sorts character by character. The truthiness test on line 3 is fine: an empty list is exactly what it should catch.

Common wrong answers.

- Line 3: `if not wave_ids` is correct for a list: an empty list is the case to catch.

D7-L2 Review practice: PySpark and Spark SQL

D7-L2 exercise 1

Hints. (1) Which sum does line 5 call? (2) The one imported on line 1. (3) Import functions as F.

Answer. The bug is on line 1.

Fixed version:

```
1 from pyspark.sql import functions as F
2
3 def total_checks(results: list[dict[str, str]]) -> int:
4     """Number of passed checks."""
```

```
    return sum(1 for r in results if r["status"] == "PASS")
```

Reference defects 2 and 3. The helper raises NOT_COLUMN_OR_STR because it calls Spark's sum on a generator.

Common wrong answers.

- Line 5: This line is correct Python. It fails only because line 1 replaced sum.

D7-L2 exercise 2

Hints. (1) What does ruff E712 flag? (2) Comparison to True. (3) The column is already boolean.

Answer. The bug is on line 3.

Fixed version:

```
from pyspark.sql import functions as F

in_scope = silver.filter(F.col("in_scope"))
in_scope.write.mode("overwrite").saveAsTable("research_dev.nat_in_scope")
```

Reference defect 12. `F.col("in_scope")` is the condition.

Common wrong answers.

- Line 4: Overwriting a dev table is fine here. The standard issue is the comparison.

D7-L2 exercise 3

Hints. (1) What is null != 'test' in Spark? (2) Null, which the filter drops. (3) eqNullSafe.

Answer. The bug is on line 3.

Fixed version:

```
from pyspark.sql import functions as F

live = respondents.filter(~F.col("panel_type").eqNullSafe("test"))
```

Spark's three-valued logic drops null rows from a != filter.

Common wrong answers.

- Line 1: The import follows the standard.

D7-L2 exercise 4

Hints. (1) What does when() give for non-matching rows? (2) Null. (3) Sum of nulls.

Answer. a) NULL Because: r1) when without otherwise is null for non-matching rows, and a sum of nulls is null.

Reference defect 16. Wrap with `F.coalesce(..., F.lit(0))`.

Common wrong answers.

- `b`: That is the goal. It needs coalesce(..., lit(0)) or otherwise(0).
- `c`: Non-matching rows contribute null, not 1.
- `d`: No error. The null passes silently.
- `a|r2`: Right answer, wrong reason. A sum of real zeros is 0. A sum of only nulls is null.

D7-L2 exercise 5

Hints. (1) Compare the groupBy with the stated grain. (2) Which grain column is missing? (3) tracker_id.

Answer. The bug is on line 5.

Fixed version:

```
1 from pyspark.sql import functions as F
2
3 metrics = (
4     silver.filter(F.col("in_scope"))
5     .groupBy("tracker_id", "wave_id", "age_group", "gender", "region")
6     .agg(F.sum("final_weight").alias("weighted_base"))
7 )
```

Reference finding 15. Without tracker_id, two trackers' respondents merge into one row per wave and subgroup.

Common wrong answers.

- Line 4: The in-scope filter follows the standard.
- Line 6: The aggregate is fine. The grouping misses a grain column.

D7-L2 exercise 6

Hints. (1) How many times is the table scanned? (2) Once per count(). (3) One agg with conditional sums.

Answer. The bug is on line 5, 6, 7.

Fixed version:

```
1 from pyspark.sql import functions as F
2
3 df = spark.table(table_name)
4 row = df.agg(
5     F.count("*").alias("row_count"),
6     F.sum(F.col("respondent_id").isNull().cast("int")).alias("null_ids"),
```

```
7
      F.sum(F.col("final_weight").isNull().cast("int")).alias("null_weights"),
8 ).first()
9 health = row.asDict()
```

Reference defect 11. Three actions mean three scans of an uncached table. One `agg` does it in one.

Common wrong answers.

- Line 3: Reading the table once is right. The three actions below each scan it again.

D7-L2 exercise 7

Hints. (1) What happens when two source rows match one target row? (2) MERGE fails. (3) Deduplicate the source on the keys.

Answer. The bug is on line 5.

Fixed version:

```
1 from delta.tables import DeltaTable
2
3 source = delivery.dropDuplicates(["respondent_id", "wave_id"])
4 target = DeltaTable.forName(spark, "research_silver.nat_respondents")
5 (target.alias("t")
6     .merge(source.alias("s"), "t.respondent_id = s.respondent_id AND
    t.wave_id = s.wave_id")
7     .whenMatchedUpdateAll()
8     .whenNotMatchedInsertAll()
9     .execute())
```

Reference finding 14. Delta MERGE fails when several source rows match one target row. Deduplicate first, ideally keeping the latest delivery.

Common wrong answers.

- Line 6: Updating matched rows is right. The failure comes from two source rows matching one target row.

D7-L2 exercise 8

Hints. (1) Expand NOT IN into AND terms. (2) x <> NULL is null. (3) No row passes.

Answer.

```
1 0
```

Use `NOT EXISTS` or a `left_anti` join when the list can hold nulls.

Common wrong answers.

- `700`: That is what NOT EXISTS returns. NOT IN with a NULL in the list is never true.

D7-L2 exercise 9

Hints. (1) What does withColumnRenamed do with a missing column? (2) Nothing, silently. (3) Check bronze_df.columns first.

Answer. The bug is on line 4.

Fixed version:

```
1 def rename_columns(bronze_df, column_map: dict[str, str]):
2 
      """Rename source columns to canonical names. Raises if a source column
      is missing."""
3       missing = sorted(set(column_map) - set(bronze_df.columns))
4       if missing:
5           raise ValueError(f"delivery is missing columns: {missing}")
6       for src, tgt in column_map.items():
7           bronze_df = bronze_df.withColumnRenamed(src, tgt)
8       return bronze_df
```

Reference finding 18. A renamed column in a delivery vanishes without error, and every later step sees a null or a missing field.

Common wrong answers.

- Line 3: Looping over the map is fine. The rename inside it hides a missing column.

D7-L3 Review practice: Databricks SQL and DAX

D7-L3 exercise 1

Hints. (1) Does line 3 exist in Databricks SQL? (2) It is Snowflake syntax. (3) ALTER TABLE ... SET ROW FILTER.

Answer. The bug is on line 3.

Fixed version:

```
1 CREATE FUNCTION research_governance.region_filter(region STRING)
2   RETURN is_account_group_member(concat('analysts_', region));
3 ALTER TABLE research_silver.nat_respondents SET ROW FILTER
      research_governance.region_filter ON (region);
```

Reference defect 4. The UDF is right. The binding statement comes from another platform.

Common wrong answers.

- Line 1: A SQL UDF is exactly what Databricks row filters use.

D7-L3 exercise 2

Hints. (1) Who receives each grant? (2) One line names a person. (3) Grant to the group.

Answer. The bug is on line 3.

Fixed version:

```
1 GRANT USE CATALOG ON CATALOG research TO `analysts`;
2 GRANT USE SCHEMA ON SCHEMA research.research_gold TO `analysts`;
3 GRANT SELECT ON TABLE research.research_gold.nat_metrics TO `analysts`;
```

Grants to groups survive team changes. A grant to one email is forgotten when that person moves.

Common wrong answers.

- Line 1: USE CATALOG to the group is needed and correct.

D7-L3 exercise 3

Hints. (1) Which group function does the goal name? (2) Account-level groups. (3) is_account_group_member.

Answer. The bug is on line 2.

Fixed version:

```
1 CREATE FUNCTION research_governance.mask_postcode(p STRING)
2   RETURN CASE WHEN is_account_group_member('analysts') THEN p ELSE '***'
    END;
3 ALTER TABLE research_silver.nat_respondents ALTER COLUMN postcode SET
    MASK research_governance.mask_postcode;
```

Reference finding 22. is_member checks workspace-local groups. Account groups are the ones Unity Catalog uses.

Common wrong answers.

- Line 3: ALTER COLUMN ... SET MASK is the correct Databricks statement.

D7-L3 exercise 4

Hints. (1) How many parts does the name have? (2) Which part is missing? (3) The current catalog fills it in.

Answer. a) research_dev.research_gold.nat_metrics, if it exists Because: r1) A two-part name resolves against the current catalog. Only a three-level name is unambiguous.

Use three-level names in jobs: `research.research_gold.nat_metrics`.

Common wrong answers.

- `b`: The name does not say which catalog. The session decides.
- `c`: Two-part names work. That is why they are risky.
- `d`: One table is resolved.
- `a|r2`: Right answer, wrong reason. The current catalog decides, whatever it is.

D7-L3 exercise 5

Hints. (1) Does each subgroup weigh the same in AVG? (2) Should it? (3) Total over total.

Answer. The bug is on line 2.

Fixed version:

```
1 SELECT region,
2        SUM(weighted_reach) / NULLIF(SUM(weighted_base), 0) AS reach_pct
3 FROM research.research_gold.nat_metrics
4 GROUP BY region;
```

Recompute ratios from sums at the level you report. NULLIF turns a zero base into a null result instead of an error.

Common wrong answers.

- Line 4: Grouping by region matches the goal.

D7-L3 exercise 6

Hints. (1) What does / return when base_w is 0? (2) Infinity or an error. (3) `DIVIDE` with the base as its second argument.

Answer.

```
1 Reach % =
2 VAR reach_w = SUM ( fact_reach[weighted_reach] )
3 VAR base_w = SUM ( fact_reach[weighted_base] )
4 RETURN DIVIDE ( reach_w, base_w )
```

DIVIDE returns BLANK on a zero or blank denominator, so the cell stays empty (lesson C6-L2).

Common wrong answers.

- g1 = `IFERROR`: IFERROR hides every error, not only a zero base. DIVIDE returns BLANK for a zero or blank denominator.

D7-L3 exercise 7

Hints. (1) Which filters does ALL(fact_reach) remove? (2) All of them. (3) Name the brand dimension.

Answer. The bug is on line 3.

Fixed version:

```
1 Brand share =
2 VAR this_brand = SUM ( fact_reach[weighted_reach] )
3 VAR all_brands = CALCULATE ( SUM ( fact_reach[weighted_reach] ), ALL
     ( dim_brand ) )
4 RETURN DIVIDE ( this_brand, all_brands )
```

The denominator must keep the wave and region filters and drop only the brand filter.

Common wrong answers.

- Line 4: DIVIDE is right. The denominator's context is wrong.

D7-L3 exercise 8

Hints. (1) Can the source hold two rows for one key? (2) Yes, and MERGE fails then. (3) Deduplicate in the USING clause with ROW_NUMBER.

Answer. The bug is on line 2.

Fixed version:

```
1  MERGE INTO research.research_silver.nat_respondents AS t
2  USING (
3    SELECT * EXCEPT (rn) FROM (
4      SELECT *, ROW_NUMBER() OVER (PARTITION BY respondent_id, wave_id
     ORDER BY ingestion_timestamp DESC) AS rn
5      FROM research.research_bronze.nat_raw_w14
6    ) WHERE rn = 1
7  ) AS s
8  ON t.respondent_id = s.respondent_id AND t.wave_id = s.wave_id
9  WHEN MATCHED THEN UPDATE SET *
10 WHEN NOT MATCHED THEN INSERT *;
```

Deduplicate the source on the merge keys, keeping the latest row, before MERGE. Reference finding 14 in SQL form.

Common wrong answers.

- Line 3: The join condition uses both merge keys. The source is the problem.

D8-L1 State versions and constraints in every prompt

D8-L1 exercise 1

Hints. (1) Which fact can the assistant not guess? (2) The platform. (3) Name the dialect.

Answer. a) Target: Databricks SQL, Unity Catalog, Databricks Runtime 17.3 LTS Because: r1) Row filter syntax differs by platform. Naming the dialect rules out Snowflake's CREATE ROW ACCESS POLICY.

The dialect decides the syntax. Name it every time.

Common wrong answers.

- `b`: Vague quality words do not change which dialect the assistant picks.
- `c`: Efficiency does not matter if the statement does not exist on your platform.
- `d`: An explanation of the wrong dialect is still the wrong dialect.
- `a|r2`: Right answer, wrong reason. Length does not help. Specific facts do.

D8-L1 exercise 2

Hints. (1) Context before the task. (2) Rules, then prohibitions. (3) Assumptions at the end.

Answer.

1. Environment: runtime, language and library versions
2. Standards that apply: imports, typing, table rules
3. What not to do: no UDFs, no RDDs, no secrets
4. The task, with inputs, outputs and side effects
5. Ask for a list of assumptions

Environment, standards, prohibitions, task, assumptions.

Common wrong answers.

- Context first. An assistant starts shaping the answer as soon as it reads the task.
- The assumptions request comes last, after the task it refers to.

D8-L1 exercise 3

Hints. (1) Check VERSIONS.md. (2) The LTS runtime this course pins. (3) 17.3 LTS with Spark 4.0.

Answer.

```
Environment: Databricks Runtime 17.3 LTS (Spark 4.0, Python 3.12), Unity Catalog.
```

Databricks Runtime 17.3 LTS ships Apache Spark 4.0.0 and Python 3.12.3.

Common wrong answers.

- g1 = `18`: Runtime 18 is not LTS yet. This course targets 17.3 LTS.

D8-L1 exercise 4

Hints. (1) Escape the library name with re.escape. (2) Follow it with `\s*v?\d+(\.\d+)*`. (3) Use lookarounds so pandas-stubs does not match pandas.

Answer.

```python
import re


def unstated_versions(prompt: str, libraries: list[str]) -> list[str]:
    """Libraries a prompt names without a version number.

    Args:
        prompt: Prompt text.
        libraries: Library names that need a stated version.

    Returns:
        Libraries without a version, in the order given.
    """
    missing = []
    for lib in libraries:
        pattern = rf"(?<![\w-]){re.escape(lib)}(?![\w-])\s*v?\d+(\.\d+)*"
        if re.search(pattern, prompt, flags=re.IGNORECASE) is None:
            missing.append(lib)
    return missing
```

A small lint on your own prompts. Run it on the header you paste, and you never forget the versions line.

D8-L1 exercise 5

Hints. (1) Which versions will the code run on? (2) Which rules apply to this task? (3) Where do shared standards live?

Answer.

- Pinned versions match what the code will run on.
- A short list of relevant rules gets followed. A long file gets skimmed, by people and models.
- A header in the repository is shared, versioned and updated when the runtime changes.

Pinned versions, relevant rules, and a header kept in the repository.

Common wrong answers.

- versions = `latest`: The latest PySpark is not what Databricks Runtime 17.3 LTS runs.
- standards = `whole`: The rules that matter get lost among the ones that do not apply.
- storage = `memory`: Unwritten headers drift and differ between people.

D8-L2 State expected outputs and tests in the prompt

D8-L2 exercise 1

Hints. (1) Which option states the rule exactly? (2) An example with the answer. (3) Option a.

Answer. a) A five-row example with a null and the expected result worked by hand Because: r1) An example states the rule precisely. Words like 'properly' leave the rule to the assistant.

A worked example is a specification the assistant cannot misread.

Common wrong answers.

- `b`: Properly how? Excluded from the base, counted as 0, or an error?
- `c`: Which edge cases, and with what answers?
- `d`: Best practice does not say whether a null counts in the base.
- `a|r2`: Right answer, wrong reason. Instructions in words work when they are specific. An example is the most specific form.

D8-L2 exercise 2

Hints. (1) Which rows are valid? (2) Reach 3.0, base 4.0. (3) 0.75.

Answer.

```
1 0.75
```

Working the example out yourself is part of writing the prompt. If you cannot, the rule is not clear yet.

Common wrong answers.

- `0.6`: That includes the None row in the base: 3 / 5. The rule excludes it: 3 / 4.

D8-L2 exercise 3

Hints. (1) Tests are fixed. (2) Disagreement goes to you, not into the test. (3) `without` and `say so`.

Answer.

```
Make these tests pass without editing them. If a test seems wrong, say so
    instead of changing it.
```

Fixed tests are the acceptance criteria. An assistant that edits them to pass has changed the requirement.

Common wrong answers.

- g1 = `by`: Editing the tests to pass is exactly what the instruction must forbid.

D8-L2 exercise 4

Hints. (1) `', '.join(repr(a) for a in args)` builds the argument list. (2) Use `{expected!r}` for the expected value. (3) Special-case None with `is None`.

Answer.

```python
def examples_to_asserts(
    func_name: str, examples: list[tuple[tuple[object, ...], object]]
) -> list[str]:
    """Turn worked examples into assert lines for a prompt or a test file.

    Args:
        func_name: Name of the function under test.
        examples: (args, expected) pairs worked out by hand.

    Returns:
        One assert statement per example.
    """
    lines = []
    for args, expected in examples:
        call = f"{func_name}({', '.join(repr(a) for a in args)})"
        if expected is None:
            lines.append(f"assert {call} is None")
        else:
            lines.append(f"assert {call} == {expected!r}")
    return lines
```

Worked examples become executable tests in one step. Paste them into the prompt and into the test file.

D8-L2 exercise 5

Hints. (1) What settles the key and the grain? (2) What happens with two rows for one key? (3) When are tests written?

Answer.

- A worked example fixes the grain and which columns form the key.

- An explicit tie rule removes a decision the assistant would otherwise make silently.
- Fixed tests written first are acceptance criteria, not a mirror of the code.

Example, explicit rules, and tests written first.

Common wrong answers.

- example = `none_ex`: Deduplicate on which key, keeping which row? The example answers both.
- tie = `any`: 'Any' makes the output depend on execution order, and it changes between runs.
- tests = `after`: Tests written after the code tend to confirm what the code does, bugs included.

D8-L3 Iterate on a failing answer

D8-L3 exercise 1

Hints. (1) What does the assistant need to know? (2) What failed, and how. (3) Test, values, scope.

Answer. a) The failing test, the actual and expected values, and 'fix only this' Because: r1) Precise evidence and a narrow scope lead to a targeted fix instead of a rewrite.

Evidence plus scope. The same rule you would follow when asking a colleague.

Common wrong answers.

- `b`: Without evidence, the assistant guesses what failed and often rewrites working parts.
- `c`: 'More carefully' gives no information about what is wrong.
- `d`: More context is not more evidence. Send the failing test and the smallest input.
- `a|r2`: Right answer, wrong reason. Assistants respond to anything. Evidence makes the response useful.

D8-L3 exercise 2

Hints. (1) Which line widens the task? (2) An open-ended refactor. (3) Replace it with a scope limit.

Answer. The bug is on line 3.

Fixed version:

```
test_zero_share fails: expected ["a"], got [].
Input: compare_waves({"a": 0.0}, {"a": 0.12}).
Fix only this failure. Do not edit the tests or change other behaviour.
```

One problem per round. Refactoring can come later, as its own request with its own tests.

Common wrong answers.

- Line 1: The failing test and values are exactly what the prompt needs.
- Line 2: The smallest failing input is useful evidence.

D8-L3 exercise 3

Hints. (1) What goes into the prompt? (2) What must you check before trusting a pass? (3) When do you stop asking?

Answer.

1. Collect the failing assertion, the error and the smallest input
2. Send them with 'fix only this, do not edit the tests'
3. Diff both the code and the test file in the answer
4. Run the full test suite
5. If two rounds fail, debug and fix it yourself

Evidence, focused prompt, diff, full suite, and a limit on rounds.

Common wrong answers.

- Diff before running. Passing tests mean little if the tests were edited.
- Collect the evidence first. It is what the prompt sends.

D8-L3 exercise 4

Hints. (1) Build a set of stripped lines from after. (2) Walk before in order and keep stripped assert lines not in that set. (3) Counting asserts misses a changed one.

Answer.

```python
def removed_asserts(before: str, after: str) -> list[str]:
    """Assert statements in the old test file that the new one lacks.

    Args:
        before: Test file text before the assistant's change.
        after: Test file text after the change.

    Returns:
        Stripped assert lines from before that are missing in after, in
    order.
    """
    kept = {line.strip() for line in after.splitlines()}
```

```
    removed = []
    for line in before.splitlines():
        stripped = line.strip()
        if stripped.startswith("assert") and stripped not in kept:
            removed.append(stripped)
    return removed
```

Run it on every answer that touches tests. A changed assertion needs your approval, never a silent accept.

D8-L3 exercise 5

Hints. (1) What is faster after two failed rounds? (2) What does a long failed thread carry? (3) What can make a suite green without a fix?

Answer.

- Two focused rounds without progress means the assistant is guessing. Your debugger is faster.
- A fresh thread drops the failed attempts and starts from your diagnosis.
- A green suite with edited tests proves nothing.

Stop and debug, restart clean with a diagnosis, and diff the tests every time.

Common wrong answers.

- next = `round4`: A fourth vague round tends to produce a fourth guess.
- fresh = `same_thread`: The failed attempts stay in context and pull later answers towards them.
- tests = `trust_green`: Green can mean the tests were weakened.